財務管理

波動率（Volatility）、風險（Risk）與人工智慧（AI）

以問題導向學習探索現代風險分析

VolTech Analytics

亞 洲 量 化 風 險 解 決 方 案

工 程 與 金 融 的 交 匯 點

授課教師： 賴奕豪

所屬系所： 大葉大學財務金融學系

課程類別： 新型專班國際學生學程

學期： 2026 春季（第二學期）

版本： 初版

前言

為什麼寫這本書？

金融市場本質上充滿不確定性。2008 年全球金融危機、2020 年 COVID-19 市場崩盤，以及 2022 年利率衝擊，都揭示了一個共同的教訓：**假設常態分配的風險模型，在你最需要它們的時刻必然失靈**。

本教科書專為已完成一學期財務管理基礎課程的工程與理科學生設計。你們擁有堅實的數學能力和程式設計技術——正是現代風險分析所需要的。你們可能欠缺的是金融直覺。本書將為你們搭建這座橋樑。

如何使用本書

這不是一本傳統的教科書。這是一場**問題導向學習（Problem-Based Learning, PBL）**體驗，圍繞一條連續的故事線展開。

你將加入**VolTech Analytics**——一家位於台北的量化風險顧問新創公司，擔任初級分析師 Alex Chen。在八週的時間裡，你將協助團隊為第一個重要客戶——一家在 COVID-19 市場動盪中遭受重大損失的日本退休基金——建立完整的風險管理系統。

每一章都遵循一致的結構：

1. **故事**——設定本週挑戰的敘事情境
2. **理論**——你需要的數學與金融概念
3. **Python 實作**——使用真實資料的實作練習
4. **任務**——整合所有內容的實務任務
5. **重點摘要**——你應該記住的要點
6. **懸念**——接下來是什麼（以及你為什麼應該關心）

你將學到什麼

完成本課程後，你將能夠：

- 分析金融報酬分配（Return Distribution），並辨識偏離常態分配的現象
- 使用 EWMA、GARCH 和 GJR-GARCH 模型衡量和預測波動率（Volatility）
- 使用多種方法論計算和回溯測試風險值（Value-at-Risk）
- 將機器學習（Machine Learning）技術應用於金融風險預測
- 在 Python 中建立完整的風險管理流程
- 向非技術背景的利害關係人溝通風險分析結果

先修知識

- 一學期的財務管理（報酬、風險和投資組合理論的基礎知識）
- 基礎統計學（平均數、變異數、常態分配、假設檢定）
- 基礎 Python 程式設計（變數、迴圈、函式、pandas 基礎）
- 熟悉數學符號（求和、微分、矩陣代數）

致謝

本教科書專為大葉大學財務金融學系新型專班國際學生學程開發。特別感謝那些以回饋意見塑造 PBL 體驗的學生們，以及讓人人都能進行金融分析的開源 Python 生態系統。

賴奕豪
財務金融學系副教授
大葉大學
2026 春季

認識團隊

你將在 *VolTech Analytics* 共事的夥伴們

Alex Chen——初級量化分析師

背景：電機工程系應屆畢業生。數學和程式設計能力優異，但對金融領域尚屬新手。

角色：這就是你。Alex 代表你從工程領域踏入量化金融世界的旅程。Alex 提出的每一個問題，都可能是你心中的疑問。Alex 面臨的每一個挑戰，都是為了培養你的技能而設計的。

「我可以從零開始寫一個神經網路，但我完全不知道 *GARCH* 模型是什麼。不過，那只是目前而已。」

Dr. Mei Lin——資深風險管理經理

背景：哥倫比亞大學金融工程博士。在高盛工作十年後返回亞洲，加入 VolTech。

角色：你的導師。Dr. Lin 透過真實世界的類比和蘇格拉底式提問來講解理論。她親歷過每一次市場危機，深知教科書模型在哪裡會崩潰。

「公式很簡單。理解它何時失效——那才是真正學習的開始。」

Kenji Tanaka——日本生命退休基金交易部主管

背景：二十年日本機構投資經驗。態度保守但樂於創新——2020 年的經驗暴露了他們風險模型的弱點。

角色：嚴格要求的客戶。Kenji 帶來了現實世界的約束條件：監理要求、董事會報告，以及為數百萬人管理退休儲蓄的壓力。

「我不需要優雅的數學。我需要一個在市場崩潰時仍值得信賴的數字。」

Priya Sharma——AI/ML 工程師

背景：印度理工學院孟買分校計算機科學碩士。深度學習和時間序列分析專家。
角色：技術橋樑。Priya 將傳統金融模型與現代 AI 方法連結在一起。她挑戰傳統方法，但尊重那些行之有效的工具。
「你的 *GARCH* 模型不錯，但你試過用神經網路來處理嗎？」

David Wu——創辦人暨執行長

背景：曾任台北某避險基金的投資組合經理人。創辦 VolTech，致力於將機構等級的風險分析帶到亞洲市場。
角色：商業視角。David 會問出你未來老闆也會問的問題：「商業價值在哪裡？」「我們能向非技術背景的董事會解釋這個嗎？」「我們如何保持領先優勢？」「再好的分析，如果賣不出去，就一文不值。」

公司簡介：VolTech Analytics

VolTech Analytics

亞 洲 量 化 風 險 解 決 方 案

關於我們

VolTech Analytics是一家位於台北的量化風險顧問新創公司，由前避險基金投資組合經理人 David Wu 於 2024 年創辦。公司專長於為亞太地區的機構投資人提供**波動率分析（Volatility Analysis）**與**風險管理解決方案（Risk Management Solutions）**。

憑藉一支精幹的量化分析師、資料科學家和金融工程師團隊，VolTech 在最前沿的學術研究與實務風險管理需求之間架起橋樑。

我們的使命

透過結合嚴謹的統計模型與現代 AI 技術，為亞洲投資人普及機構等級的風險分析工具。

面臨的挑戰

2026 年初，VolTech 取得第一個重大客戶：**日本生命退休基金（Nippon Life Pension Fund）**，日本最大的退休基金之一。該基金在 2020 年 3 月的 COVID-19 市場崩盤中遭受了重大損失，原因是其現有的風險模型——基於簡單的常態分配假設——完全無法預測下跌的幅度。

日本生命交易部主管 Kenji Tanaka 向 VolTech 提出了明確的任務：

◎ 客戶需求

1. 分析現有風險模型在 COVID-19 期間失效的原因
2. 建立能捕捉厚尾（Fat Tails）和不對稱效應（Asymmetric Effects）的波動率模型

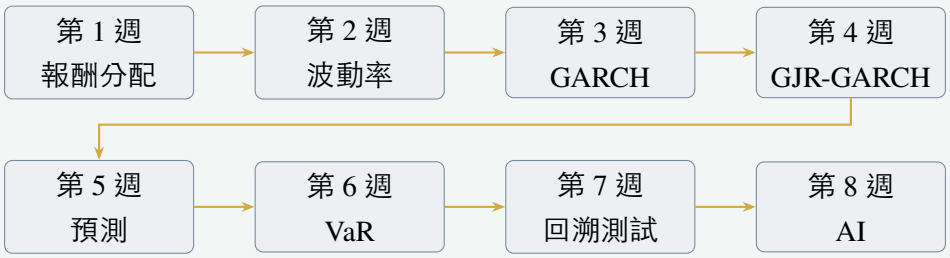
3. 開發具備適當回溯測試（Backtesting）的穩健 VaR 系統
4. 探索 AI 增強的風險預測能力
5. 在**八週內**交付完整系統

你的角色

你是 **Alex Chen**，VolTech 新進的初級量化分析師。你的工程背景為你提供了堅實的數學和程式設計基礎，但你對金融和風險管理的世界還很陌生。

在接下來的八週裡，你將與 VolTech 團隊一起完成日本生命的合約。每週都會帶來新的挑戰、新的工具和新的洞見，讓你從初級分析師成長為自信的風險管理專家。

前方的旅程



序幕：改變一切的那通電話

場景：VolTech Analytics 辦公室，台北——2026 年 2 月，週一早晨

位於信義區某棟大樓十二樓的辦公室雖然不大，卻充滿活力。三台大螢幕即時顯示著市場數據。白板上寫滿了方程式。咖啡杯和 Python 筆記本在桌上爭奪空間。VolTech 創辦人兼執行長 David Wu 放下電話，望向房間另一頭的 Dr. Mei Lin。

那是日本生命的 *Kenji Tanaka*。他們希望我們建構他們整個風險管理系統。這是一份能讓 *VolTech* 聲名大噪的合約。

日本生命？他們管理著日本最大的退休基金之一。他們的時程是什麼？

八週。他們被現有供應商的模型坑慘了——那些模型在 *2020* 年 *3* 月摧毀了他們大部分股票投資組合之前，還告訴他們一切安好。

典型的常態分配（*Normal Distribution*）陷阱。他們的 *VaR* 模型可能假設報酬率是行為良好的鐘形曲線。在 *95%* 的時間裡運作正常。但在最關鍵的那 *5%* 卻災難性地失敗了。

我們有工具。*GARCH* 模型作為基礎，機器學習（*Machine Learning*）提供優勢。問題是：我們有足夠的人手嗎？

所以我剛雇了一個人。一個電機工程師，程式能力碾壓大多數金融博士。今天就到職。

上午 9:00——大廳

Alex Chen 站在大廳裡，整理著領帶。他的背景是訊號處理和機器學習，不是金融。但那則徵才啟事寫著：「我們不需要懂金融的人。我們需要學得快、寫程式寫得好的人。」

這聽起來像是一項挑戰。而 Alex 從不在挑戰面前退縮。

你的旅程現在開始。

翻到下一頁，開始第一週的學習。

Contents

前言 **ii**

認識團隊 **iv**

VolTech Analytics **vi**

序幕 **viii**

1 第 1 週：金融風險與報酬分配 **1**

1.1 本週故事：VolTech 的第一天 1
1.2 報酬率：金融的語言 2
1.3 常態分配假設 3
1.4 厚尾、偏態與峰態 4
1.5 常態性檢定 5
1.5.1 QQ 圖 5
1.5.2 Jarque–Bera 檢定 6
1.6 Python 實作 6
1.7 應用：分析 Kenji 的投資組合 9
1.8 討論：為什麼常態性會失效？ 10
1.9 每週任務 10
1.10 重點摘要 11
1.11 延伸閱讀 12

2 第 2 週：衡量波動率 **13**

2.1 本週故事：一把尺不夠用 13
2.2 歷史波動率：滾動視窗法 14
2.3 EWMA：給予近期數據更多權重 15
2.4 實現波動率：利用日內數據 16
2.5 VIX：市場的恐慌指標 17
2.6 Python 實作 18
2.7 應用：恐慌的速度 20
2.8 波動率的典型事實 21
2.9 討論：選擇你的波動率衡量方法 22
2.10 每週任務 23

2.11 重點摘要 24
2.12 延伸閱讀 25

3 第 3 週：波動率群聚與 GARCH 26
3.1 本週故事：波動率有記憶 26
3.2 偵測波動率群聚 27
3.3 ARCH 模型：一切的起點 28
3.4 GARCH(1,1)：主力模型 29
3.5 波動率衰減的速度 31
3.6 最大概似估計法 31
3.7 Python 實作 33
3.8 應用：GARCH 遇上 COVID 崩盤 36
3.9 模型診斷：GARCH 夠了嗎？ 38
3.10 AI 焦點：AI 能取代 GARCH 嗎？ 40
3.11 討論：GARCH 假設與現實 40
3.12 每週任務 41
3.13 重點摘要 42
3.14 延伸閱讀 43

4 第 4 週：不對稱波動率 — GJR-GARCH 44
4.1 本週故事：壞消息打擊更大 44
4.2 GJR-GARCH 模型 45
4.3 新聞衝擊曲線 46
4.4 EGARCH：一種替代方法 47
4.5 Python 實作 48
4.6 應用：不對稱性檢定 53
4.7 模型診斷 55
4.8 AI 焦點：機器學習中的不對稱性 56
4.9 討論：實務中的不對稱性 56
4.10 每週任務 57
4.11 重點摘要 58
4.12 延伸閱讀 59

5 第 5 週：波動率預測 61
5.1 本週故事：模型大賽 61
5.2 預測框架 62
5.3 預測評估指標 63
5.4 Mincer–Zarnowitz 迴歸 64

5.5 Python 實作：預測管道 64
5.6 應用：誰贏得模型大賽？ 70
5.7 AI 焦點：預測的前沿 72
5.8 討論：預測的哲學 72
5.9 每週任務 73
5.10 重點摘要 74
5.11 延伸閱讀 75

6 第 6 週：風險值 76
6.1 本週故事：統治一切的那個數字 76
6.2 理解風險值 77
6.3 方法 1：歷史模擬法 VaR 78
6.4 方法 2：參數法（變異數-共變異數法）VaR 79
6.5 方法 3：蒙地卡羅 VaR 80
6.6 超越 VaR：期望損失 80
6.7 Python 實作：三種 VaR 方法 81
6.8 應用：COVID-19 崩盤期間的 VaR 87
6.9 AI 焦點：神經網路 VaR 88
6.10 討論：VaR 論辯 88
6.11 每週任務 89
6.12 重點摘要 90
6.13 延伸閱讀 91

7 第 7 週：VaR 回測與風險管理 93
7.1 本週故事：監管機構來電 93
7.2 回測的邏輯 94
7.3 檢定一：Kupiec 無條件覆蓋率 95
7.4 檢定二：Christoffersen 條件覆蓋率 95
7.5 Basel 交通號誌系統 96
7.6 壓力測試 97
7.7 Python 實作：回測套件 97
7.8 應用：FSA 報告 105
7.9 AI 焦點：自動化風險監控 106
7.10 討論：當模型失效時 106
7.11 每週任務 107
7.12 重點摘要 108
7.13 延伸閱讀 109

8 第 8 週：AI 與金融 **110**
8.1 本週故事：最終挑戰 110
8.2 ML vs. 計量經濟學：兩種哲學 111
8.3 隨機森林用於波動率 112
8.4 LSTM 神經網路用於時間序列 112
8.5 Python 實作：ML 競技場 113
8.6 應用：最終裁決 120
8.7 FinTech 前沿 121
8.8 討論：金融的未來 122
8.9 每週任務 122
8.10 重點摘要 123
8.11 延伸閱讀 124

公式附錄 **126**

術語表 **132**

Chapter 1

第 1 週：金融風險與報酬分配

學習目標

1. 從金融價格資料計算簡單報酬率（Simple Return）和對數報酬率（Log Return）
2. 解釋為什麼常態分配（Normal Distribution）不適合描述金融報酬
3. 衡量偏態（Skewness）和超額峰態（Excess Kurtosis）並解釋其金融意涵
4. 建構和解讀 QQ 圖（QQ Plot）以評估分配適配度
5. 使用 Python 執行 Jarque–Bera 常態性檢定

1.1 本週故事：VolTech 的第一天

場景：VolTech Analytics 辦公室，十二樓——週一上午 9:15

Alex Chen 走出電梯，步入一間雖小但充滿活力的辦公室。三台大螢幕即時顯示著市場數據——日經 225 指數、S&P 500 期貨，以及一個標示為「VIX」、Alex 尚不認識的指標。寫滿希臘字母的白板和一句裱框的名言爭奪著牆面空間：「我們信賴上帝。其他人請拿數據來。」

Dr. Mei Lin 從螢幕前抬起頭來。

你一定是 *Alex*。歡迎來到*VolTech*。咖啡在角落。你的第一個客戶會議在三十分鐘後。

客戶會議？這就開始了？我以為至少有一週的時間來熟悉環境。

在量化金融中，你是邊做邊學的。別擔心——先聽、先觀察就好。*Kenji Tanaka* 從東京飛來了。他管理著日本最大退休基金之一的交易部門。

場景：會議室——上午 10:00

他們走進會議室時，Kenji Tanaka 已經就座。他將一份印刷報告推過桌面——上面全是 2020 年 3 月的紅色數字。

2020 年 3 月。COVID-19 爆發，我們的投資組合在一週內虧損了 18%。我們的風險模型說，在 99% 信心水準下，最大單週虧損是 4.2%。那個模型告訴我們是安全的。但我們並不安全。

你們用的是什麼模型？

標準的參數化 *VaR*。常態分配假設。我們的供應商說這是「業界標準」。

它確實是業界標準。這正是問題所在。*Alex*，這就是我們的起點。在修復 *Kenji* 的風險模型之前，我們需要先理解它為什麼崩潰。要做到這一點，我們需要看看金融報酬率實際上長什麼樣——而不是教科書假設的樣子。

1.2 報酬率：金融的語言

簡單報酬率 vs. 對數報酬率

在金融學中，我們很少直接分析價格。取而代之的是，我們使用**報酬率（Returns）**——價格在一段期間內的百分比變動。有兩種常見的定義：

簡單（算術）報酬率：

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{1.1}$$

對數（連續複利）報酬率：

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(1 + R_t) \tag{1.2}$$

其中 P_t 是時間 t 的收盤價。不同於價格隨時間趨勢性上漲（非定態），報酬率圍繞著一個相對穩定的均值波動（近似定態），使其適合進行統計建模。

為什麼使用對數報酬率？

對數報酬率有兩個特性使其成為量化金融的標準：

1. **時間可加性**：多期對數報酬率等於單期對數報酬率之和：$r_{t:t+k} = r_{t+1} + r_{t+2} + \cdots + r_{t+k}$
2. **對稱性**：$+10\%$ 和 -10% 的對數報酬率以零為中心對稱，不像簡單報酬率中 50% 的虧損需要 100% 的漲幅才能回本。

對於較小的報酬率（例如不超過 5%），簡單報酬率和對數報酬率幾乎相同，因為當 x 很小時 $\ln(1+x) \approx x$。

Alex，你的工程背景在這裡會派上用場。在訊號處理中，你分析訊號的頻譜。在金融中，我們分析報酬率的分配。同樣的概念，不同的領域。

1.3 常態分配假設

高斯模型

常態分配（Normal Distribution）（高斯分配）自 Bachelier（1900）將價格變動建模為常態分配以來，一直是金融報酬的預設模型，而 Samuelson（1965）則提出了對數常態價格模型（等價地，常態分配的對數報酬率）。如果報酬率服從常態分配：

$$r_t \sim \mathcal{N}(\mu, \sigma^2) \tag{1.3}$$

這表示整個分配僅由兩個參數來描述：均值 μ 和變異數 σ^2。極端事件的機率以指數速度衰減。

在常態分配下：

- 68.3% 的觀測值落在 $\pm 1\sigma$ 之內
- 95.4% 落在 $\pm 2\sigma$ 之內
- 99.7% 落在 $\pm 3\sigma$ 之內
- 一個 5σ 事件（單尾）的機率 $\approx 0.00003\%$——大約每 13,900 個交易年才會發生一次

問題所在

在現實中，金融市場經歷 5σ 事件的頻率遠高於常態分配的預測。1990 年至 2024 年間，S&P 500 至少有 20 次日波動超過 $\pm 5\sigma$——按理論說這種事件每幾千年才會發生一次。

一個標準的解決方案是**Student-t 分配**，它具有較重的尾部，由自由度參數 ν 控制。對於典型的股票報酬，$\nu \approx 4$–8 比常態分配提供更好的配適。我們將在第 3 週開始使用 t 分配作為 GARCH 模型的誤差分配。

所以你告訴我，我們的模型建立在一個謊言之上？

不是謊言——是近似。常態分配在一般市場情況下運作得還算合理。但「還算合理」和「對管理退休基金而言足夠安全」是截然不同的標準。

1.4 厚尾、偏態與峰態

要理解金融報酬如何偏離常態性，我們需要超越均值和變異數。我們需要分配的**第三**和**第四動差（Moments）**。

偏態——第三動差

偏態（Skewness）衡量分配的不對稱性：

$$S = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^3 \tag{1.4}$$

- $S = 0$：對稱（常態分配）
- $S < 0$：**負偏態**——左尾較長（大幅虧損比大幅獲利更常見）
- $S > 0$：正偏態——右尾較長

大多數股票報酬序列呈現**負偏態**，意味著大幅虧損比等幅度的大幅獲利更為常見。

在以偏態衡量了不對稱性之後，我們接著來看尾部的厚度——這對風險管理同樣至關重要。

√x 超額峰態——第四動差

超額峰態（Excess Kurtosis）衡量尾部相對於常態分配的厚度：

$$K = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^4 - 3 \tag{1.5}$$

我們減去 3 是因為常態分配的原始峰態恰好為 3；因此 K 被稱為超額峰態。

- $K = 0$：**常峰態（Mesokurtic）**（正常尾部）
- $K > 0$：**高狹峰態（Leptokurtic）——厚尾（Fat Tails）**（極端事件比常態更頻繁）
- $K < 0$：低闊峰態（Platykurtic）——薄尾（極端事件較少）

金融報酬序列通常顯示 $K \gg 0$，表示它們是強**高狹峰態**的。每日股票報酬率的超額峰態通常在 5–20 或更高。

一個重要的解釋：如果波動率本身隨時間變化，將低波動和高波動時期混合在一起，即使每個時期的報酬近似常態，也會產生厚尾。這種**波動率叢聚（Volatility Clustering）**效應正是我們將在第 3 週用 GARCH 建模的對象。

樣本動差 vs. 母體動差

上述公式是母體動差估計量（除以 T）。實務上，Python 的 `pandas.skew()` 和 `pandas.kurtosis()` 會套用小樣本偏差修正（偏態除以 $(T-1)(T-2)$，峰態則有更複雜的調整）。對於大 T（例如 10 年的 2,500 個每日觀測值），差異微乎其微。但如果你發現手動計算公式和程式庫函數的結果有些微不同，這就是原因。

所以厚尾意味著極端事件發生的頻率超出常態預測。而負偏態意味著大跌比大漲更可能。這..... 正是 *Kenji* 投資組合發生的情況。

現在你開始像個量化分析師一樣思考了。下一個問題是：我們如何檢驗報酬率是否真的符合常態？如果不符合，我們如何視覺化這種偏離？

1.5 常態性檢定

1.5.1 QQ 圖

QQ 圖（Quantile–Quantile Plot）將觀測資料的分位數與理論分配（通常是常態分配）的分位數進行比較。如果資料符合常態分配，散點將沿著 45 度直線排列。

如何閱讀 QQ 圖

- **散點落在直線上**：資料符合理論分配
- **S 形曲線**：厚尾（散點在兩端偏離）
- **左端散點在直線上方**：左尾較重（更極端的虧損）
- **右端散點在直線下方**：右尾較重（更極端的獲利）

對於金融報酬，你幾乎總是會看到 S 形模式，證實了厚尾的存在。

1.5.2 Jarque–Bera 檢定

Jarque–Bera（JB）檢定透過將偏態 (1.4)和超額峰態 (1.5)合併為單一檢定統計量，來形式化常態性檢查：

Jarque–Bera 檢定統計量

$$JB = \frac{T}{6}\left(S^2 + \frac{K^2}{4}\right) \tag{1.6}$$

在虛無假設 H_0：報酬率服從常態分配之下，JB 統計量服從 $\chi^2(2)$ 分配。
判斷準則：若 $JB > \chi^2_{0.05}(2) = 5.99$，則拒絕 H_0。
實務上，金融報酬序列產生的 JB 統計量通常在**數百或數千**，提供了壓倒性的反常態證據。

1.6 Python 實作

理論很重要，但讓我們用真實資料來驗證。*Alex*，打開你的筆電。我們要拉取實際的市場數據，親眼看看這些現象。

載入資料並計算報酬率

```python
import numpy as np           # numerical computing
import pandas as pd          # data manipulation
import yfinance as yf        # Yahoo Finance data API
from scipy import stats      # statistical functions
import matplotlib.pyplot as plt  # plotting

# Download S&P 500 daily prices (2015-2024)
sp500 = yf.download("^GSPC", start="2015-01-01",
```

```
                        end="2024-12-31")
prices = sp500["Close"].squeeze()  # daily close prices

# Compute log returns: r_t = ln(P_t / P_{t-1})
log_returns = np.log(prices / prices.shift(1)).dropna()
print(f"Number of observations: {len(log_returns)}")
# Expected: ~2,500 trading days over 10 years
```

載入資料後，讓我們計算關鍵的分配統計量並執行 Jarque–Bera 檢定：

描述統計與常態性檢定

```
# Basic statistics
mean_r = log_returns.mean()
std_r  = log_returns.std()
skew_r = log_returns.skew()
kurt_r = log_returns.kurtosis()  # excess kurtosis

print(f"Mean daily return:  {mean_r:.6f}")
print(f"Std deviation:      {std_r:.4f}")
print(f"Skewness:           {skew_r:.4f}")
print(f"Excess kurtosis:    {kurt_r:.4f}")

# Jarque-Bera test
jb_stat, jb_pvalue = stats.jarque_bera(log_returns.values)
print(f"\nJarque-Bera statistic: {jb_stat:.2f}")
print(f"p-value:               {jb_pvalue:.2e}")
```

數字只能說明部分故事，但視覺化能讓偏離常態的情況一目了然：

QQ 圖與直方圖

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left panel: Histogram with normal overlay
axes[0].hist(log_returns, bins=100, density=True,
             alpha=0.7, color="#2D7D5E",
             label="Actual returns")
x = np.linspace(log_returns.min(), log_returns.max(), 200)
axes[0].plot(x, stats.norm.pdf(x, mean_r, std_r),
```

```
                "r-", lw=2, label="Normal distribution")
axes[0].set_title("S&P 500 Daily Log Returns vs Normal")
axes[0].set_xlabel("Log Return")
axes[0].set_ylabel("Density")
axes[0].legend()

# Right panel: QQ plot
stats.probplot(log_returns.values, dist="norm", plot=axes[1])
axes[1].set_title("QQ Plot vs Normal Distribution")
axes[1].get_lines()[0].set_color("#1B3A5C")
axes[1].get_lines()[1].set_color("#C0392B")

plt.tight_layout()
plt.savefig("week01_returns_analysis.png", dpi=150)
plt.show()
```

直方圖清楚地顯示峰部比常態曲線更高，尾部更厚。*QQ* 圖也有你提到的 *S* 形——散點在兩端飛離直線。

正是如此。而且你會注意到，每日 *S&P 500* 報酬的 *Jarque–Bera* 統計量將達到數千。p 值實際上等於零。常態性被以絕對的確定性拒絕了。

AI 實務：異常檢測（Anomaly Detection）

現代金融 AI 系統利用我們剛發現的同一洞見——報酬率不是常態的——來建構更好的異常檢測。孤立森林（Isolation Forest）和自編碼器（Autoencoder）等機器學習模型能在不假設任何特定分配的情況下識別異常的報酬模式。在第 8 週，我們將探索這些技術與傳統統計方法的比較。

1.7 應用：分析 Kenji 的投資組合

場景：會議室——下午 2:30

經過一頓工作午餐後，Alex 向團隊展示了初步發現。螢幕上顯示著退休基金股票投資組合報酬率的直方圖和 QQ 圖。

以下是我們的發現。*2015* 年至 *2024* 年的投資組合報酬率顯示偏態為 -0.73，超額峰態為 *13.4*。*Jarque–Bera* 統計量為 *4,892*，p 值實際上等於零。

用白話文來說，那些數字代表什麼意思？

這表示你的報酬率尾部比常態分配厚得多，而且是不對稱的——大幅虧損比大幅獲利更頻繁。具體來說：

Table 1.1: 退休基金投資組合的理論尾部機率 vs. 實際尾部機率

指標	常態假設	實際資料
偏態（Skewness）	0	-0.73
超額峰態（Excess Kurtosis）	0	13.4
單日虧損 $> 3\sigma$ 的機率	0.13%	1.7%
單日虧損 $> 5\sigma$ 的機率	0.00003%	0.12%

在常態模型下，超過 5σ 的虧損應該每 *13,900* 個交易年才會發生一次。但在你的資料中，大約每 *3* 年就會發生。這就是為什麼你的 *VaR* 模型會失效——它對尾部是盲目的。

一萬四千年對比三年。現在我明白為什麼我的董事會會大發雷霆了。

1.8 討論：為什麼常態性會失效？

為什麼金融報酬不服從常態分配？

思考以下因素，討論哪些因素導致了金融報酬率的厚尾和負偏態：

1. **波動率叢聚（Volatility Clustering）**：平靜期之後接著動盪期（我們將在第 3 週用 GARCH 建模）
2. **槓桿效應（Leverage Effect）**：壞消息比好消息更能增加波動率（第 4 週）
3. **羊群行為（Herding Behavior）**：投資人一起恐慌性拋售放大了下跌幅度
4. **資訊不對稱（Information Asymmetry）**：壞消息傳播得比好消息更快
5. **市場微結構（Market Microstructure）**：交易暫停、追繳保證金和被迫平倉造成跳躍

這些因素中，哪些可以用更好的統計模型來捕捉？哪些需要行為或制度面的解釋？

1.9 每週任務

◎ 任務 1：分配診斷報告

目標：為日本生命退休基金投資組合製作完整的分配分析報告。

交付成果：

1. 下載至少 3 個主要指數（S&P 500、日經 225、FTSE 100）2015–2024 年的每日報酬資料
2. 對每個指數計算並報告：
   - 均值、標準差、偏態、超額峰態
   - Jarque–Bera 檢定統計量與 p 值
   - 疊加常態曲線的直方圖
   - 對照常態分配的 QQ 圖
3. 建立跨所有指數的四動差比較表
4. 撰寫 200 字的執行摘要，解釋為什麼常態假設對 Kenji 的投資組合是有問題的。提示：執行摘要應陳述關鍵發現、其影響，以及建議的下一步。

加分挑戰：尾部機率表

對每個指數，計算每日虧損超過 1σ、2σ、3σ、4σ 和 5σ 的實證機率。將這些與常態分配下的理論機率進行比較。以清晰的表格呈現結果。

提示：使用 `np.mean(returns < -k * std)` 來計算每個門檻值 k。

1.10 重點摘要

重點摘要

1. **分析報酬率，而非價格**：金融分析使用報酬率（最好是對數報酬率），因為它們近似定態且具時間可加性，不像價格會隨時間漂移。
2. **常態性失效**：金融報酬呈現**厚尾**（超額峰態 $\gg 0$）和**負偏態**（$S < 0$），意味著極端虧損的頻率遠超常態分配的預測。
3. **四個動差都重要**：僅靠均值和變異數不足以描述金融報酬分配。偏態和峰態捕捉了驅動風險的不對稱性和尾部厚度。
4. **QQ 圖與 JB 檢定**：這些是評估分配假設的第一線診斷工具。要學會流暢地閱讀它們。
5. **風險模型的影響**：任何基於常態性建構的風險模型（包括基本的參數化 VaR）都會系統性地低估大額虧損的機率。這不是理論上的好奇——它有真實的財務後果。

1.11 延伸閱讀

延伸閱讀

1. Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.
2. Fama, E. F. (1965). The behavior of stock-market prices. *The Journal of Business*, 38(1), 34–105.
3. Mandelbrot, B. (1963). The variation of certain speculative prices. *The Journal of Business*, 36(4), 394–419.
4. Jarque, C. M., & Bera, A. K. (1987). A test for normality of observations and regression residuals. *International Statistical Review*, 55(2), 163–172.
5. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapters 1–2.
6. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapter 5.
7. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools*, 2nd ed. Princeton University Press. Chapter 3.
8. Bollerslev, T., Todorov, V., & Li, S. Z. (2013). Jump tails, extreme dependencies, and the distribution of stock returns. *Journal of Econometrics*, 172(2), 307–324.
9. Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. [第 3 週的基礎]

下 週 預 告

Kenji 把一份印刷報告放到 *Alex* 桌上——*2020* 年 *3* 月的每日虧損，用紅色標記。「你已經告訴我問題所在了。現在讓我看看你能衡量它。我們投資組合現在的波動率有多大？它變化得有多快？」*Priya* 補充道：「你不能用一把固定的尺來衡量所有時候的風險。」下週，我們將學習衡量波動率——並發現它永遠不會靜止不動。

Chapter 2

第 2 週：衡量波動率

學習目標

1. 使用不同長度的滾動視窗計算歷史波動率（Historical Volatility）
2. 應用 EWMA 模型並解釋為何它對新資訊反應更快
3. 定義實現波動率（Realized Volatility）並解釋其相對於收盤價估計量的優勢
4. 解讀 VIX 指數作為市場前瞻性波動率預期的意義
5. 比較多種波動率衡量方法在 COVID-19 危機期間的表現

2.1 本週故事：一把尺不夠用

場景：VolTech Analytics 辦公室——週二，上午 8:45

Alex 一早就到了，腦中仍在消化昨天的發現。那張比較表被貼在他的螢幕上：每 3 年一次的 5σ 事件對比常態分配預測的每 14,000 年一次。常態分配是危險地錯誤。但昨天下午 Kenji 的問題在他腦中揮之不去：「我們的投資組合現在到底有多大的波動率？」

早安，*Alex*。我看到你在盯著昨天的表格。很好。現在把它放在一邊。知道報酬率有厚尾是必要的，但還不夠。我們需要衡量波動率——而且是即時衡量。

我們不能直接計算報酬率的標準差嗎？我們昨天就做了。

可以。但用哪些報酬率？過去 *20* 天的？過去一年的？過去十年的？答案會隨著你選的視窗而劇烈變化。而更深層的問題是：波動率不是恆定的。它會變動。它會群聚。平靜的幾週之後往往接著動盪的幾週。你不能用一把固定的尺來衡量所有時間的風險。

所以我們需要一把會自我調整的尺。

完全正確。今天我會教你四種不同的尺——然後我們來看看當 *COVID-19* 來襲時，哪一把最先發出警告。

2.2 歷史波動率：滾動視窗法

衡量波動率最簡單的方法是**滾動視窗**（Rolling Window，又稱**歷史波動率（Historical Volatility）**）法。選擇一個 N 天的視窗，計算該視窗內報酬率的標準差，然後每天將視窗向前滑動一天。

滾動視窗波動率

對於在時間 t 結束的 N 個觀測值視窗：

$$\hat{\sigma}_t^{\text{roll}} = \sqrt{\frac{1}{N-1}\sum_{i=0}^{N-1}(r_{t-i} - \bar{r}_{t,N})^2} \tag{2.1}$$

其中 $\bar{r}_{t,N} = \frac{1}{N}\sum_{i=0}^{N-1} r_{t-i}$ 為滾動平均值。要將日波動率年化，乘以 $\sqrt{252}$。

視窗長度的取捨

選擇 N 涉及一個基本的偏差-變異數取捨（Bias-Variance Trade-off）：

- **短視窗**（例如 $N = 20$ 天 ≈ 1 個月）：對體制變化反應快速但雜訊多（高變異數）
- **長視窗**（例如 $N = 252$ 天 ≈ 1 年）：平滑穩定但對突然變化反應遲緩（高偏差）
- **常見選擇**：20 天（短期）、60 天（中期）、252 天（長期）

沒有普遍「正確」的視窗——最佳選擇取決於你的應用場景。需要早期預警信號的風險管理者偏好短視窗；追蹤長期趨勢的投資組合分析師偏好長視窗。

滾動視窗的一個關鍵限制是它對視窗內所有觀測值賦予**等權重**。250 天前的報酬率與昨天的報酬率被完全等同對待。當市場突然轉變——如同 2020 年 3 月——滾動視窗反映新現實的速度很慢。

2020 年 2 月，我們的 252 天波動率估計值大約是年化 12%。到了 3 月中旬，實際波動率已經爆炸了。但我們的年度滾動視窗仍然包含了 2019 年所有平靜的數據。花了好幾週才趕上來。

2.3 EWMA：給予近期數據更多權重

指數加權移動平均（Exponentially Weighted Moving Average, EWMA）模型透過對較舊的觀測值賦予指數遞減的權重來解決等權重問題。近期報酬率更重要；遠期報酬率逐漸淡化。

EWMA 變異數

EWMA 變異數估計以遞迴方式更新：

$$\hat{\sigma}_t^2 = \lambda\,\hat{\sigma}_{t-1}^2 + (1-\lambda)\,r_{t-1}^2 \tag{2.2}$$

其中 $\lambda \in (0,1)$ 為**衰減因子（Decay Factor）**。注意我們使用 r_{t-1}^2 而非 $(r_{t-1}-\bar{r})^2$：對於日報酬率，平均值相對於變異數可忽略不計，因此這種零均值簡化是標準做法（也是 RiskMetrics 所採用的方法）。
等價地，時間 t 的 EWMA 變異數是所有過去平方報酬率的加權總和：

$$\hat{\sigma}_t^2 = (1-\lambda)\sum_{i=1}^{\infty}\lambda^{i-1}\,r_{t-i}^2 \tag{2.3}$$

i 期前報酬率的權重為 $(1-\lambda)\lambda^{i-1}$，呈幾何遞減。

選擇 λ：RiskMetrics 標準

J.P. Morgan 具有影響力的 **RiskMetrics** 系統（1996）建議日頻數據使用 $\lambda = 0.94$，這已成為事實上的產業標準。在此數值下：

- 昨天的平方報酬率獲得權重 0.06（$= 1 - 0.94$）
- 10 天前的報酬率獲得權重 $0.06 \times 0.94^9 \approx 0.034$
- 30 天前的報酬率獲得權重 $0.06 \times 0.94^{29} \approx 0.010$
- **有效觀測數**（權重分配的平均落後期數）約為 $\frac{1}{1-\lambda} = \frac{1}{0.06} \approx 17$ 天——意味著 EWMA 大致等同於 17 天平均，但使用指數遞減權重而非硬性截斷

較高的 λ（例如 0.97）給出更平滑的估計；較低的 λ（例如 0.90）給出反應更靈敏的估計。

注意 *EWMA* 不需要選擇視窗——只有一個參數 λ。而且不像滾動視窗，舊數據永遠不會突然「掉出」估計值。它平滑地淡化。這就是為什麼 *EWMA* 不會產生那種極端觀測值突然離開滾動視窗時出現的殘影效應（*Ghosting Artifacts*）。

「殘影效應」是什麼意思？

假設第 *1* 天發生了市場崩盤，接下來的 $N-1$ 天都是平靜的報酬率。用滾動視窗，崩盤的觀測值恰好在 N 天內貢獻於波動率估計，然後消失——造成一個突然的人為下降。*EWMA* 避免了這個問題，因為崩盤的影響是逐漸淡化的。

EWMA 作為特殊的 GARCH

EWMA 實際上是你在第 3 週會遇到的 GARCH 模型的一個特例——具體來說，是沒有長期變異數項（$\omega = 0$）且 $\alpha + \beta = 1$ 的 GARCH(1,1)。這意味著 EWMA 隱含地假設波動率沒有固定的「基準」可以回歸。GARCH 透過加入均值回歸（Mean Reversion）組件修正了這個限制。

2.4 實現波動率：利用日內數據

滾動視窗和 EWMA 都使用每日收盤價。但在每個交易日內，價格持續波動。**實現波動率（Realized Volatility）**利用這些日內資訊來產生更精確的波動率估計。

實現波動率

給定第 t 天的 M 個等間距日內報酬率 $r_{t,1}, r_{t,2}, \ldots, r_{t,M}$：

$$RV_t = \sum_{j=1}^{M} r_{t,j}^2 \tag{2.4}$$

當 $M \to \infty$（取樣頻率增加）時，RV_t 收斂至該日的真實**積分變異數（Integrated Variance）**——一種無需模型的波動率衡量。

為什麼平方日內報酬率有效

關鍵的洞見來自機率論：對於連續時間過程，平方增量之和收斂於**二次變差（Quadratic Variation）**——過程的總「變異路徑」。簡單來說，更頻繁的取樣能更精確地衡量價格在一天中實際移動了多少，而不僅僅是比較開盤和收盤價。

在實務上，我們通常使用 *5* 分鐘報酬率（*6.5* 小時交易日的 $M = 78$）。使用更高頻率會引入**微結構雜訊（*Microstructure Noise*）**——買賣價差跳動和其他摩擦會扭曲估計值。有一些抗雜訊的估計量，但 *5* 分鐘取樣是一個很好的實務起點。

實現波動率需要日內數據，這些數據可能並非對所有資產或所有時段都可取得。對於我們與 Kenji 退休基金的專案，我們將主要使用日收盤價衡量方法（滾動視窗和 EWMA），但將實現波動率作為黃金標準銘記在心。

2.5 VIX：市場的恐慌指標

前三種方法都是**回顧性的**——它們從過去的數據估計波動率。**VIX 指數**採取了根本不同的方法：它從**選擇權價格（Options Prices）**中提取波動率預期，使其成為**前瞻性的**。

VIX 指數

CBOE 波動率指數（VIX）衡量市場對 S&P 500 未來 30 天波動率的預期，由 S&P 500 指數選擇權的價格推導而來。主要特性：

- **前瞻性**：反映交易者預期波動率將會是多少，而非過去是多少
- **年化**：VIX = 20 表示市場預期年化波動率約為 20%，或日波動率約為 $20\%/\sqrt{252} \approx 1.26\%$
- **恐慌指標**：VIX 通常在市場崩盤時飆升，在平靜期間下降
- **均值回歸**：極端的 VIX 水準傾向於回歸長期平均值（約 18–20）

COVID-19 期間的 VIX

2020 年 3 月 16 日，VIX 達到 **82.69**——自 2008 年金融危機以來的最高水準，也是該指數歷史上最高的讀數之一。作為參考：

- 正常市場：VIX $\approx$ 12–18
- 焦慮升高：VIX $\approx$ 20–30
- 危機模式：VIX $>$ 30
- 極度恐慌：VIX $>$ 50
- 2020 年 3 月 16 日：VIX = 82.69——完全超出圖表範圍

VIX 在 2 月底就已升至 40 以上，早於最嚴重的拋售潮。這是一個關鍵觀察：VIX 常常領先於實現波動率，因為選擇權交易者在恐慌完全顯現之前就已將其定價。

波動率微笑

在計算 VIX 時，CBOE 使用一系列不同行使價的價外選擇權。有趣的是，從這些選擇權中提取的隱含波動率（Implied Volatility）在不同行使價之間並非恆定——它形成一個 U 形模式，稱為**波動率微笑（Volatility Smile）**（或權益選擇權的**波動率偏斜（Smirk）**，其中下行行使價顯示更高的隱含波動率）。這個模式本身就是市場在定價中反映厚尾和負偏態的證據——正是我們在第 4 週將研究的典型事實。

隱含波動率 vs. 實現波動率

隱含波動率（來自選擇權，例如 VIX）與實現波動率（來自實際報酬率）之間的差異稱為**變異數風險溢酬（Variance Risk Premium）**。平均而言，隱含波動率超過實現波動率，這意味著選擇權賣方因承擔波動率風險而賺取溢酬。這個溢酬在經濟上是顯著的，也是實證金融學中最穩健的發現之一。

所以 *VIX* 在大聲呼喊「危險」的時候，我們的滾動視窗還在沉睡。如果我們當時關注了 *VIX*，就能更早採取行動。

原則上是的。但 *VIX* 衡量的是 *S&P 500* 的隱含波動率，而不是你特定投資組合的風險。我們需要能即時估計你的投資組合波動率的模型。這就是 *GARCH* 模型的功能——但那是第 *3* 週的內容。現在，讓我們看看這些方法在實際中的表現。

2.6 Python 實作

是時候建構我們的波動率儀表板了。*Alex*，我們將比較滾動視窗、*EWMA* 和 *VIX* 在 *COVID-19* 期間對 *S&P 500* 的表現。

載入數據與計算滾動波動率

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

# Download S&P 500 data (2018-2024)
sp500 = yf.download("^GSPC", start="2018-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
log_returns = np.log(prices / prices.shift(1)).dropna()

# Rolling window volatility (annualized)
vol_20  = log_returns.rolling(20).std() * np.sqrt(252)
vol_60  = log_returns.rolling(60).std() * np.sqrt(252)
```

```
vol_252 = log_returns.rolling(252).std() * np.sqrt(252)
```

接著，我們以 RiskMetrics 衰減因子 $\lambda = 0.94$ 實作 EWMA 遞迴 (2.2)：

EWMA 波動率

```
def ewma_volatility(returns, lam=0.94):
    """Compute EWMA volatility (annualized)."""
    n = len(returns)
    var = np.zeros(n)
    var[0] = returns.iloc[0] ** 2  # initialize
    for t in range(1, n):
        var[t] = lam * var[t-1] + (1 - lam) * returns.iloc[t-1]**2
    vol = pd.Series(np.sqrt(var) * np.sqrt(252),
                    index=returns.index)
    return vol

vol_ewma = ewma_volatility(log_returns, lam=0.94)
# Note: pandas provides ewm() for production use:
# log_returns.ewm(alpha=1-0.94).std() * np.sqrt(252)
```

最後，我們下載 VIX 數據，比較回顧性衡量方法與市場的前瞻性預期：

下載 VIX 並繪製儀表板

```
# Download VIX index
vix = yf.download("^VIX", start="2018-01-01",
                  end="2024-12-31")
vix_close = vix["Close"].squeeze()

# Plot comparison
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(vol_20.index, vol_20, label="Rolling 20d",
        alpha=0.7, color="#1B3A5C")
ax.plot(vol_60.index, vol_60, label="Rolling 60d",
        alpha=0.7, color="#5D6D7E")
ax.plot(vol_ewma.index, vol_ewma, label="EWMA (0.94)",
        alpha=0.9, color="#2D7D5E", linewidth=2)
ax.plot(vix_close.index, vix_close / 100, label="VIX / 100",
        alpha=0.8, color="#C0392B", linewidth=1.5)
```

```python

# Mark COVID-19 crash
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red", label="COVID-19 crash")
ax.set_title("Volatility Measures: S&P 500 (2018-2024)")
ax.set_ylabel("Annualized Volatility")
ax.legend(loc="upper right")
ax.set_ylim(0, 1.0)
plt.tight_layout()
plt.savefig("week02_volatility_comparison.png", dpi=150)
plt.show()
```

這張圖很震撼。*VIX* 最先飆升——*2* 月底就超過了 *40%*。*EWMA* 在幾天內跟上。但 *252* 天滾動視窗到 *4* 月才勉強有反應。就像看三個不同的氣象站在不同時間報告同一場風暴。

2.7 應用：恐慌的速度

場景：會議室——週三，下午 3:00

Alex 將波動率儀表板投影到螢幕上。Kenji 身體前傾，用手指描繪著那些曲線。

我對 *COVID-19* 期間的 *S&P 500* 跑了所有四種衡量方法。以下是我的發現：

Table 2.1: COVID-19 危機期間波動率反應速度（S&P 500）

衡量方法	2 月 19 日水準	3 月 16 日峰值	反應時間
滾動 20 天	8.2%	89.7%	∼2 週
滾動 60 天	10.4%	54.1%	∼4 週
滾動 252 天	14.1%	23.8%	∼8+ 週
EWMA（$\lambda = 0.94$）	7.9%	83.4%	∼1 週
VIX（隱含）	14.4%	82.7%	即時

關鍵發現如下：

1. ***VIX* 領先一切**：它在 *2* 月 *24* 日就突破了 *30%*，比 *S&P 500* 進入熊市早了四天
2. ***EWMA* 是最快的回顧性衡量方法**：其指數加權意味著 *2* 月的拋售立即主導了估計值
3. **滾動 *252* 天幾乎無用**：即便在危機最高峰，它仍被 *8* 個多月的平靜 *2019* 年數據所「稀釋」

所以 *EWMA* 本可以比我們的 *252* 天滾動視窗至少提早一週發出警告信號。那一週可能為我們節省數億日圓。

這正是我們的客戶需要的即時情報。如果我們能將此打包成一個即時儀表板產品，亞洲每一個機構投資人都會想要。

而這只是開始。*EWMA* 比固定的滾動視窗好，但它仍然是一個相對簡單的模型。它無法捕捉波動率傾向於「黏性」的現象——今天的高波動率預示著明天的高波動率。要做到這一點，我們需要 *GARCH* 模型家族。但那是第 *3* 週的內容。首先，讓我介紹一個將成為我們整個風險框架核心的概念。

2.8 波動率的典型事實

在進入更進階的模型（第 3 週起）之前，讓我們總結金融波動率的關鍵實證模式——**典型事實（Stylized Facts）**。這些模式在不同市場、不同時期和不同資產類別中都具有驚人的穩健性：

波動率的典型事實

1. **波動率群聚（Volatility Clustering）**：大的報酬率（無論正負）傾向於跟隨大的報酬率，小的報酬率傾向於跟隨小的報酬率。正式地說，報酬率本身不相關，但其絕對值呈正自相關。
2. **均值回歸（Mean Reversion）**：波動率傾向於回歸長期平均值。極端水準（非常高或非常低）是暫時的。
3. **不對稱性（槓桿效應）**：負報酬率傾向於比同等幅度的正報酬率更能增加未來的波動率。我們將在第 4 週建模。
4. **長記憶（Long Memory）**：絕對報酬率的自相關衰減緩慢，顯示波動率具有長記

憶——它記得遙遠的衝擊。

5. **共同變動（Co-movement）**：波動率傾向於在各市場同時增加，尤其是在危機期間（傳染效應）。

Alex，這五個事實就是任何好的波動率模型的「設計規格」。滾動視窗一個都捕捉不到。*EWMA* 部分捕捉了群聚效應。*GARCH* 捕捉群聚和均值回歸。*GJR-GARCH* 加入不對稱性。每一週，我們都增加一個新層次。

就像建構一個逐步改進的訊號濾波器——每一個都能捕捉數據中更多的結構。

沒錯。你的工程直覺非常敏銳。

2.9 討論：選擇你的波動率衡量方法

哪把尺適合哪份工作？

不同的波動率衡量方法適合不同的用途。考慮以下場景，並討論哪種衡量方法最為合適：

1. 每月再平衡的**投資組合經理人**——需要穩定的估計值，而非每天大幅波動的數字
2. 監控每日 VaR 限額的**風險管理者**——需要最早的預警信號
3. 為選擇權定價的**衍生性商品交易員**——需要市場的前瞻性預期
4. 評估系統性風險的**監管機構**——需要能捕捉跨市場傳染效應的衡量方法
5. 研究波動率動態的**學術研究者**——需要對「真實」波動率最精確的衡量

有沒有一種衡量方法能適用於所有目的？為什麼？

2.10 每週任務

◎ 任務 2：退休基金的波動率儀表板

目標：為 Kenji 的投資組合分析建構一個全面的波動率比較儀表板。

交付項目：

1. 下載日經 225、S&P 500 和 FTSE 100 從 2018 至 2024 年的日頻數據
2. 對每個指數計算：
   - 滾動視窗波動率：20 天、60 天、252 天
   - EWMA 波動率，$\lambda = 0.94$
3. 繪製每個指數的波動率儀表板（所有衡量方法在同一張圖上，標示 COVID-19 期間）
4. 建立摘要表格：對每種衡量方法和每個指數，報告 COVID 前水準（2 月 19 日）、峰值水準和近似反應時間（如表 2.1）
5. 撰寫 200 字備忘錄給 Kenji，建議 VolTech 應使用哪種波動率衡量方法進行每日風險監控，以及原因

🏆 附加挑戰：衰減因子實驗

對日經 225 分別使用 $\lambda = 0.90$、0.94 和 0.97 執行 EWMA 模型。將三者繪製在同一張圖上並討論：

- 哪個對新衝擊反應最快？
- 哪個產生最平滑的估計值？
- 如果你必須為每日風險管理選擇一個 λ，你會選哪個？為什麼？

提示：有效視窗長度約為 $\frac{1}{1-\lambda}$ 天。

2.11 重點摘要

重點摘要

1. **波動率不是恆定的**：它會群聚、均值回歸，並對好消息和壞消息做出不對稱的反應。任何有用的波動率衡量方法都必須考慮這種時變特性。
2. **滾動視窗**：簡單直觀，但對所有觀測值賦予等權重。對體制變化反應緩慢，且當極端觀測值離開視窗時會產生殘影效應。
3. **EWMA**：指數遞減權重優先考慮近期數據。產業標準 $\lambda = 0.94$ 在反應靈敏度和平滑度之間提供良好平衡。
4. **實現波動率**：當日內數據可用時的黃金標準。隨著取樣頻率增加，收斂至真實的積分變異數。
5. **VIX**：前瞻性，從選擇權價格推導而來。在危機期間常領先於實現波動率，因為它在恐慌完全體現在報酬率中之前就捕捉了交易者的恐懼。
6. **沒有萬能解**：正確的衡量方法取決於應用場景。風險管理者需要反應靈敏（EWMA、VIX）；投資組合經理人可能偏好穩定性（較長的滾動視窗）。

2.12 延伸閱讀

延伸閱讀

1. J.P. Morgan (1996). *RiskMetrics — Technical Document*, 4th ed. [EWMA 的基礎參考文獻]
2. Andersen, T. G., & Bollerslev, T. (1998). Answering the skeptics: yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
3. Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2003). Modeling and forecasting realized volatility. *Econometrica*, 71(2), 579–625.
4. Whaley, R. E. (2009). Understanding the VIX. *The Journal of Portfolio Management*, 35(3), 98–105.
5. Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.
6. Poon, S.-H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: a review. *Journal of Economic Literature*, 41(2), 478–539.
7. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
8. Barndorff-Nielsen, O. E., & Shephard, N. (2002). Econometric analysis of realized volatility and its use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B*, 64(2), 253–280.
9. Black, F. (1976). Studies of stock price volatility changes. *Proceedings of the American Statistical Association, Business and Economics Section*, 177–181. [槓桿效應的起源]

下 週 預 告

會議結束後，*Priya* 在咖啡機旁攔住了 *Alex*。「你的波動率儀表板做得很好。但讓我問你一件事——*EWMA* 確實比滾動視窗反應更快。但它真的能預測明天的波動率嗎？還是它只是在描述已經發生的事？」她打開筆電，展示一個 *LSTM* 神經網路的訓練曲線。「我一直在建構一個試圖預測波動率的東西，不只是衡量它。下週，*Dr. Lin* 會教你那個開啟一切的模型——*GARCH*。然後我們來看看你的統計模型能不能打敗我的 *AI*。」下週：我們進入 ***GARCH*** 的世界——並發現波動率具有記憶。

Chapter 3

第 3 週：波動率群聚與 GARCH

學習目標

1. 辨識報酬率序列中的波動率群聚（Volatility Clustering），並使用平方報酬率的自相關函數進行檢定
2. 解釋 ARCH 模型及其作為條件波動率建模基礎的角色
3. 推導並詮釋 GARCH(1,1) 模型，包括 ω、α 和 β 的角色
4. 將 GARCH 與 EWMA 連結，並解釋為什麼 GARCH 加入了均值回歸
5. 在 Python 中使用最大概似估計法估計 GARCH(1,1) 模型並詮釋輸出

3.1 本週故事：波動率有記憶

場景：VolTech Analytics——週一，上午 9:00

加入 VolTech 兩週後，Alex 建立了一個令 Kenji 印象深刻的波動率儀表板。EWMA 反應快速。VIX 具有前瞻性。但 Priya 在咖啡機旁提出的挑戰讓他整個週末都在思考：我們真的能預測波動率，而不只是衡量它嗎？

Dr. Lin，我一直在想 *Priya* 說的話。*EWMA* 告訴我們波動率現在是多少，但我們能預測它明天會是多少嗎？

這是正確的問題。而答案是肯定的——因為波動率有記憶。看看這張 *2018* 年到 *2024* 年 *S&P 500* 日報酬率的圖表。你看到什麼模式？

報酬率隨機跳動，但跳動的幅度完全不是隨機的。有安靜的時期和動盪的時期，而且它們傾向於群聚在一起。

完全正確。這就叫做**波動率群聚（*Volatility Clustering*）**，它是金融學中最重要的典型事實之一。*Mandelbrot* 在 *1960* 年代就注意到了：「大的變動傾向於跟隨大的變動——不論方向——而小的變動傾向於跟隨小的變動。」如果今天波動率很高，明天很可能也會很高。這就是使其可預測的原因。

所以我們可以利用群聚模式來建構預測模型？

正是如此。今天我要向你介紹一個徹底改變我們思考波動率方式的模型。它叫 *GARCH*，是量化金融中的主力工具之一。*Robert Engle* 因開發其背後的概念而在 *2003* 年獲得諾貝爾獎。讓我們從頭開始。

3.2 偵測波動率群聚

在建構模型之前，我們需要驗證想要捕捉的現象確實存在於我們的數據中。以下是正式的檢定方法。

回想一下，日對數報酬率 r_t 大致不相關——知道今天的報酬率對明天的報酬率幾乎沒有預測力。但平方報酬率 r_t^2（變異數的代理變數）卻具有很強的自相關性：知道今天的 r_t^2 能告訴你很多關於明天 r_t^2 的資訊。

平方報酬率的自相關

平方報酬率在落後期 k 的樣本自相關函數（ACF）為：

$$\hat{\rho}(k) = \frac{\sum_{t=k+1}^{T}(r_t^2 - \overline{r^2})(r_{t-k}^2 - \overline{r^2})}{\sum_{t=1}^{T}(r_t^2 - \overline{r^2})^2} \tag{3.1}$$

其中 $\overline{r^2} = \frac{1}{T}\sum_{t=1}^{T} r_t^2$ 為平方報酬率的樣本平均值。
如果 $\hat{\rho}(k)$ 在多個落後期 k 顯著為正，這就是**波動率群聚**的證據：大的平方報酬率後面跟著大的平方報酬率。

Ljung–Box 檢定

要正式檢定平方報酬率是否具有自相關，我們使用 **Ljung–Box 檢定**：

$$Q(m) = T(T+2)\sum_{k=1}^{m}\frac{\hat{\rho}(k)^2}{T-k} \tag{3.2}$$

在無自相關的虛無假設下，$Q(m) \sim \chi^2(m)$。小的 p 值（例如 < 0.05）拒絕虛無假設，確認波動率群聚在統計上是顯著的。在實務中，對金融數據的平方報酬率做 Ljung–Box 檢定幾乎總是拒絕——波動率群聚幾乎是普遍存在的。
另一個替代方法是 Engle 的 **ARCH-LM 檢定**，它將 r_t^2 對自身的落後期做迴歸並檢定聯合顯著性。兩種檢定偵測的是相同的現象；Ljung–Box 較為簡單，而 ARCH-LM 與 ARCH 模型設定有直接的連結。

這樣想，*Alex*。報酬率就像風的方向——很難預測。平方報酬率就像風的強度——當暴風雨開始時，它不會突然停止。這種持續性就是 *GARCH* 所建模的。

3.3 ARCH 模型：一切的起點

1982 年，Robert Engle 提出了**自迴歸條件異質變異數（Autoregressive Conditional Heteroskedasticity, ARCH）**模型，這是第一個時變波動率的正式框架。關鍵洞見是：我們將報酬率分解為一個可預測的波動率成分和一個隨機衝擊。

報酬率分解

我們將報酬率 r_t 建模為：

$$r_t = \sigma_t \, z_t, \qquad z_t \sim \text{i.i.d.}\,(0,1) \tag{3.3}$$

其中：

- σ_t 為**條件標準差（Conditional Standard Deviation）**——我們要建模的波動率，它可以隨時間變化
- z_t 為標準化衝擊，獨立地從均值為 0、變異數為 1 的分配中抽取（通常假設為常態分配）
- 「條件」意味著 σ_t 取決於時間 $t-1$ 可用的資訊（過去的報酬率、過去的波動率）

概念很簡單：時間 t 的報酬率等於一個時變的「音量旋鈕」（σ_t）乘以一個隨機的「方向」（z_t）。

所以 σ_t 控制報酬率可以有多大，而 z_t 控制隨機方向？

完美的類比。現在問題是：σ_t 如何演化？*Engle* 的 *ARCH* 模型說今天的變異數取決於近期過去的衝擊。

ARCH(q) 模型

ARCH(q) 模型將條件變異數設定為：

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \cdots + \alpha_q r_{t-q}^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 \tag{3.4}$$

其中：

- $\omega > 0$ 為常數基礎變異數
- $\alpha_i \geq 0$ 為 ARCH 係數（過去平方報酬率的權重）
- q 為階數——要包含多少個平方報酬率的落後期

為了使變異數保持為正，我們需要 $\omega > 0$ 且所有 i 的 $\alpha_i \geq 0$。

為什麼 ARCH 能捕捉群聚

考慮 ARCH(1)：$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2$。如果昨天的報酬率 r_{t-1} 很大（一個衝擊），那麼 r_{t-1}^2 很大，使得 σ_t^2 很大，這意味著今天的報酬率是從更寬的分配中抽取的——增加了產生另一個大報酬率的機率。這就產生了群聚效應：大報酬率生成大報酬率。

ARCH 是一個突破，但它有一個實務上的問題。要捕捉波動率自相關的緩慢衰減，你往往需要很多落後期 q——有時 *10* 個、*20* 個或更多。這意味著要估計很多參數，效率很低。*Tim Bollerslev* 有一個更好的想法。

3.4 GARCH(1,1)：主力模型

1986 年，Tim Bollerslev 將 Engle 的 ARCH 擴展，加入了前一期的條件變異數作為額外的預測變數。這一個改變大幅減少了所需的參數數量，使模型在實務上變得可行。

GARCH(1,1) 模型

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2 \tag{3.5}$$

其中：

- $\omega > 0$：變異數的「基礎水準」
- $\alpha \geq 0$：**衝擊係數（Shock Coefficient）**——昨天的平方報酬率（新資訊）對今天變異數的影響程度
- $\beta \geq 0$：**持續性係數（Persistence Coefficient）**——昨天的變異數延續到今天的程度

- $\alpha + \beta < 1$：**定態條件（Stationarity Condition）**，確保波動率不會爆炸。當 $\alpha + \beta = 1$ 時，我們得到 IGARCH（= EWMA），此時變異數預測永不衰減且無條件變異數未定義。當 $\alpha + \beta > 1$ 時，變異數預測無限制增長——模型是爆炸性的。

「(1,1)」表示一個 r_{t-1}^2 的落後期和一個 σ_{t-1}^2 的落後期。在實務中，GARCH(1,1) 對絕大多數金融時間序列都能提供出色的擬合。

一個重要的數學洞見：透過反覆代入 σ_{t-1}^2，GARCH(1,1) 等價於一個具有幾何遞減係數的 ARCH(∞)：$\sigma_t^2 = \frac{\omega}{1-\beta} + \alpha \sum_{i=1}^{\infty} \beta^{i-1} r_{t-i}^2$。這就是為什麼單一的 GARCH 落後期可以取代無限多的 ARCH 落後期——β 項透過指數衰減編碼了整個歷史。

下表直觀地解釋每個成分的角色：

Table 3.1: GARCH(1,1) 解剖：$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$

參數	角色	典型值
ω	基底 / 基礎變異數	小的正值
α	對新衝擊的反應	0.05–0.15
β	記憶 / 持續性	0.80–0.95
$\alpha + \beta$	總持續性	0.95–0.99

所以 β 就是波動率有記憶的原因。如果 $\beta = 0.85$，那麼昨天 *85%* 的變異數會延續到今天。那是很強的記憶。

完全正確。而 α 是波動率對新衝擊做出反應的原因。如果今天市場崩盤，r_t^2 會很大，透過 α 項推高 σ_{t+1}^2。然後那個升高的 σ_{t+1}^2 又透過 β 項持續下去。這就是 *GARCH* 產生波動率群聚的方式。

GARCH 的長期變異數

GARCH 相對於 EWMA 的一個關鍵優勢是**均值回歸（Mean Reversion）**。定態 GARCH(1,1) 的長期（無條件）變異數為：

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta} \tag{3.6}$$

這是波動率趨向的「基準」。衝擊推高波動率後，模型會逐漸將其拉回 $\bar{\sigma}^2$。這正是我們在第 2 週辨識出的均值回歸典型事實。

GARCH vs. EWMA：缺失的那一塊

我們在第 2 週提到 EWMA 是 $\omega = 0$（等價地，$\alpha+\beta = 1$ 的 IGARCH）的 GARCH(1,1) 特例。設定 $\omega = 0$ 移除了長期錨定：

- **EWMA**：$\sigma_t^2 = (1-\lambda)\, r_{t-1}^2 + \lambda\, \sigma_{t-1}^2$ （無均值回歸）
- **GARCH**：$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2$ （均值回歸至 $\frac{\omega}{1-\alpha-\beta}$）

使用 EWMA，如果波動率飆升，它沒有理由回降——它會漂移。使用 GARCH，ω 項就像一個彈簧，將波動率拉回均衡。對於風險管理來說，均值回歸至關重要：我們不僅需要知道波動率很高，還需要知道它多快會回歸正常。

3.5 波動率衰減的速度

一個自然的問題：衝擊發生後，GARCH 波動率需要多久才能回歸到其長期水準的一半？

波動率半衰期

GARCH(1,1) 中波動率衝擊的**半衰期（Half-Life）**為：

$$h = \frac{\ln(0.5)}{\ln(\alpha+\beta)} \tag{3.7}$$

這告訴我們衝擊的影響衰減 50% 需要多少天。

所以如果 $\alpha + \beta = 0.97$，半衰期是 $\ln(0.5)/\ln(0.97) \approx 23$ 天。這意味著一個衝擊在將近一個月後仍然半數存在！

沒錯。金融波動率具有高度持續性。$\alpha + \beta \approx 0.95$–$0.99$ 的典型值下，半衰期從大約 *14* 天到 *69* 天不等。這就是為什麼市場崩盤不會一夜之間平靜下來——升高的波動率會持續數週或數月。

3.6 最大概似估計法

我們如何為給定的數據集找到最佳的 ω、α 和 β？標準方法是**最大概似估計法（Maximum Likelihood Estimation, MLE）**——找到使觀測數據最有可能出現的參數。

GARCH(1,1) 的 MLE

假設標準化殘差 $z_t = r_t/\sigma_t$ 服從常態分配，T 個觀測值的對數概似函數為：

$$\mathcal{L}(\omega, \alpha, \beta) = -\frac{1}{2}\sum_{t=1}^{T}\left[\ln(2\pi) + \ln(\sigma_t^2) + \frac{r_t^2}{\sigma_t^2}\right] \tag{3.8}$$

其中每個 σ_t^2 是從方程式 (3.5) 遞迴計算的。MLE 找到使此函數最大化的參數值——直觀地說，就是最能解釋觀測到的大小報酬率模式的參數。

等等——這看起來像是循環論證。我們需要 σ_t^2 來計算概似函數，但 σ_t^2 又取決於我們正在試圖估計的參數。

觀察得好。這就是為什麼我們使用數值最佳化。對於任何候選參數集 (ω, α, β)：

1. 初始化 σ_1^2（通常使用樣本變異數）
2. 使用方程式 (3.5) 遞迴計算 $\sigma_2^2, \sigma_3^2, \ldots, \sigma_T^2$
3. 計算對數概似函數 (3.8)
4. 用不同的參數值重複，直到對數概似函數最大化

最佳化器嘗試數千種組合來找到最佳擬合。在實務中，這很快——對於典型的數據集只需幾秒鐘。

超越常態分配

在實務中，即使考慮了 GARCH 效應，標準化殘差 z_t 仍常顯示厚尾。這導致了使用更厚尾分配的擴展：

- **Student's t 分配**：增加自由度參數 ν 以容納厚尾。這是實務中最常見的選擇。
- **GED（廣義誤差分配）**：允許比常態更厚或更薄的尾部。
- **偏態 t 分配**：在 Student's t 中加入不對稱性。

使用 t 分配通常能改善模型擬合並產生更實際的風險估計。

3.7 Python 實作

好的，*Alex*。是時候將理論付諸實踐了。我們將使用 `arch` 函式庫——它處理 *GARCH* 估計並使流程變得直截了當。讓我帶你走過完整的流程。

步驟 1：數據準備與群聚效應視覺化

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.diagnostic import acorr_ljungbox

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()
# Scale returns to percentage (arch library convention)

# Visualize volatility clustering
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)
axes[0].plot(returns.index, returns, color="#1B3A5C",
             linewidth=0.5)
axes[0].set_title("S&P 500 Daily Log Returns (%)")
axes[0].set_ylabel("Return (%)")

axes[1].plot(returns.index, returns ** 2, color="#C0392B",
             linewidth=0.5)
axes[1].set_title("Squared Returns (Volatility Proxy)")
axes[1].set_ylabel(r"$r_t^2$")
plt.tight_layout()
plt.savefig("week03_clustering.png", dpi=150)
plt.show()
```

平方報酬率圖清楚地揭示了群聚模式：平靜期（小的 r_t^2）與動盪期（大的 r_t^2）交替出現，而且這些體制持續數週或數月。接下來，我們從統計上確認：

步驟 2：檢定波動率群聚

```python
# ACF of squared returns
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(returns.values, lags=30, ax=axes[0],
         title="ACF of Returns")
plot_acf(returns.values ** 2, lags=30, ax=axes[1],
         title="ACF of Squared Returns")
plt.tight_layout()
plt.savefig("week03_acf.png", dpi=150)
plt.show()

# Ljung-Box test on squared returns
lb_test = acorr_ljungbox(returns ** 2, lags=10,
                         return_df=True)
print("Ljung-Box test on squared returns:")
print(lb_test)
# Expect very small p-values -> reject H0 of no clustering
```

確認群聚存在後，我們使用 `arch` 套件透過 MLE 擬合 GARCH(1,1) 模型：

步驟 3：擬合 GARCH(1,1)

```python
from arch import arch_model

# Specify GARCH(1,1) with Student's t distribution
model = arch_model(returns, vol="Garch", p=1, q=1,
                   dist="t")
result = model.fit(disp="off")
print(result.summary())

# Extract parameters
omega = result.params["omega"]
alpha = result.params["alpha[1]"]
beta  = result.params["beta[1]"]
nu    = result.params["nu"]  # t-distribution DoF

print(f"\nomega = {omega:.6f}")
print(f"alpha = {alpha:.4f}")
print(f"beta  = {beta:.4f}")
```

```python
print(f"alpha + beta = {alpha + beta:.4f}")
print(f"Long-run vol = {np.sqrt(omega/(1-alpha-beta)
                        * 252):.2f}% annualized")
print(f"Half-life = {np.log(0.5)/np.log(alpha+beta):.1f}"
      " days")
print(f"Student t DoF = {nu:.2f}")
```

輸出顯示 $\alpha \approx 0.09$ 和 $\beta \approx 0.90$，所以 $\alpha + \beta \approx 0.99$。這是極高的持續性——半衰期大約 *69* 天！

這對權益指數來說很典型。*Student's* t 自由度大約在 *5–7* 左右，確認了即使在 *GARCH* 過濾之後，殘差仍然比常態分配有更厚的尾部。現在讓我們繪製估計的條件波動率。

接下來，我們提取擬合的條件波動率路徑，並視覺化模型如何追蹤不斷變化的風險水準：

步驟 4：繪製條件波動率

```python
# Extract conditional volatility (daily, in %)
cond_vol = result.conditional_volatility  # daily %

# Plot returns with +/- 1 sigma volatility bands
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(returns.index, returns, color="#5D6D7E",
        linewidth=0.4, alpha=0.6, label="Daily returns")
ax.plot(cond_vol.index, cond_vol, color="#C0392B",
        linewidth=1.5,
        label="GARCH(1,1) cond. vol (%/day)")
ax.plot(cond_vol.index, -cond_vol, color="#C0392B",
        linewidth=1.5)
ax.fill_between(cond_vol.index, cond_vol, -cond_vol,
                alpha=0.1, color="#C0392B")

# Mark COVID-19 crash
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red", label="COVID-19 crash")
ax.set_title("S&P 500 Returns with GARCH(1,1) "
```

```
                "Conditional Volatility")
ax.set_ylabel("Daily Return / Volatility (%)")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("week03_garch_fit.png", dpi=150)
plt.show()
```

波動率包絡線在危機期間擴大，在平靜期間收窄——完全符合我們的預期。模型正在捕捉群聚模式。

3.8 應用：GARCH 遇上 COVID 崩盤

場景：會議室——週四，下午 2:00

Alex 向團隊報告了他的 GARCH 結果。模型擬合看起來令人印象深刻——條件波動率追蹤了過去十年的每一次危機。但 Dr. Lin 安排了一個更嚴格的測試。

Alex，用完整樣本擬合模型就像拿著答案在準備考試。真正的測試是模型能否在看到波動率之前就預測它。讓我告訴你我的意思。

關鍵測試：使用 2020 年 1 月 31 日之前（COVID 之前）的數據估計 GARCH，然後預測 2020 年 2 月至 4 月期間的條件波動率，並與實際發生的情況比較。

COVID 前估計與預測

```
# Fit GARCH(1,1) on full data, but use last_obs to
# limit estimation to pre-COVID period
model_pre = arch_model(returns, vol="Garch", p=1, q=1,
                       dist="t")
res_pre = model_pre.fit(last_obs="2020-01-31",
                        disp="off")

# One-step-ahead forecasts starting from Feb 2020
forecasts = res_pre.forecast(horizon=1,
                             start="2020-02-01")
fc_vol = np.sqrt(forecasts.variance.dropna()
                 ) * np.sqrt(252)
```

```python

# Long-run volatility from pre-COVID parameters
omega_pre = res_pre.params["omega"]
alpha_pre = res_pre.params["alpha[1]"]
beta_pre  = res_pre.params["beta[1]"]
longrun_vol = (np.sqrt(omega_pre /
    (1 - alpha_pre - beta_pre)) * np.sqrt(252))
peak_vol = fc_vol.max().iloc[0]

print(f"Pre-COVID long-run vol: {longrun_vol:.1f}%")
print(f"Peak GARCH forecast (Mar 2020): "
      f"{peak_vol:.1f}% annualized")
```

在 *COVID* 之前估計的 *GARCH* 模型給出了大約 *14%* 的長期波動率。當崩盤開始時，條件波動率確實飆升了——但花了好幾天才開始攀升。在拋售的最初幾天，模型仍然被錨定在其平靜市場的估計上。

這正是我們的風險模型當時發生的事！崩盤的第一天，*VaR* 數字看起來還好。等到模型趕上的時候，我們已經損失慘重了。為什麼 *GARCH* 一開始反應這麼慢？

因為 *GARCH* 是反應性的，不是主動性的。它只在觀察到大的報酬率之後才更新其變異數估計。在崩盤的第一天，r_{t-1}^2 突然跳升，σ_t^2 開始透過 α 通道上升。但由於 $\beta \approx 0.90$，前一天的（低）變異數仍然佔主導。需要連續幾個大的報酬率，模型才能完全「醒過來」。

⚠ 起始問題

這是 GARCH 類型模型的一個根本限制：它們在衝擊發生之後才有反應，而非之前。在危機的初始階段：

- 第 1 天：市場下跌 3%。GARCH 波動率幾乎不動（仍由 $\beta \times \sigma_{t-1}^2$ 主導）
- 第 2–3 天：持續大幅下跌。GARCH 開始攀升但仍低於實現波動率
- 第 5 天以後：GARCH 趕上新的體制

相比之下，VIX（第 2 週）常在最壞的報酬率出現之前就飆升，因為選擇權交易者是主動地定價恐慌。

我在數據中看到另一個問題。當市場大幅下跌時，波動率飆升的幅度遠大於同等幅度的上漲。但 *GARCH* 使用 r_{t-1}^2——平方消除了正負號。它把 -3% 的日子和 $+3\%$ 的日子完全等同看待。

你剛剛發現了基本 *GARCH* 模型最大的弱點。這叫做**槓桿效應（*Leverage Effect*）**：壞消息（負報酬率）比同等幅度的好消息（正報酬率）更能增加波動率。標準的 *GARCH(1,1)* 對這種不對稱性是盲目的，因為它對報酬率取平方，消除了正負號。

這對退休基金來說是一個嚴重的問題。我們更擔心下行波動率而非上行波動率。我們需要一個對壞消息更敏感的模型。

這正是我們下週要用 *GJR-GARCH* 模型來解決的。但首先，讓我再教你一個關於今天模型的知識——因為即使有其限制，*GARCH(1,1)* 仍然是一個非常強大的起點。

3.9 模型診斷：GARCH 夠了嗎？

擬合完任何模型後，我們必須檢查它是否充分捕捉了數據的結構。對於 GARCH，這意味著檢驗**標準化殘差（Standardized Residuals）**：

標準化殘差

$$\hat{z}_t = \frac{r_t}{\hat{\sigma}_t} \tag{3.9}$$

如果 GARCH 模型被正確設定，標準化殘差 $\hat{z}_t$ 應該表現得像 i.i.d. 抽樣——特別是，它們的平方 $\hat{z}_t^2$ 應該不再顯示自相關。

GARCH 診斷檢查

```python
# Standardized residuals from the full-sample model
std_resid = result.std_resid.dropna()

# 1. Ljung-Box test on squared standardized residuals
lb_filtered = acorr_ljungbox(std_resid ** 2, lags=10,
                             return_df=True)
```

```python
print("Ljung-Box on squared standardized residuals:")
print(lb_filtered)

# 2. ACF of squared standardized residuals
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(returns.values ** 2, lags=30, ax=axes[0],
         title="ACF of Squared Returns (Raw)")
plot_acf(std_resid.values ** 2, lags=30, ax=axes[1],
         title="ACF of Squared Std Residuals (GARCH)")
plt.tight_layout()
plt.savefig("week03_diagnostics.png", dpi=150)
plt.show()

# 3. Summary statistics of standardized residuals
print(f"\nMean:     {std_resid.mean():.4f}")
print(f"Std Dev:  {std_resid.std():.4f}")
print(f"Skewness: {std_resid.skew():.4f}")
# Note: pandas kurtosis() returns excess kurtosis
# (Fisher's definition, i.e., kappa - 3).
# Normal distribution -> 0, not 3.
print(f"Excess Kurtosis: {std_resid.kurtosis():.4f}")
```

如果 *GARCH* 模型做好了它的工作，$\hat{z}_t^2$ 上的 *Ljung–Box* p 值應該很大（未能拒絕無自相關）——意味著模型已經吸收了波動率群聚。比較兩張 *ACF* 圖：原始平方報酬率顯示很強的自相關，但平方標準化殘差應該平坦得多。

自相關大部分消失了！但偏態是負的——大約 -0.3。這就是你提到的不對稱性，對吧？

正是如此。*GARCH* 去除了波動率群聚，但它無法去除不對稱性，因為它沒有建模。殘差中的負偏態是槓桿效應的指紋，直接指向下週模型的必要性。

3.10 AI 焦點：AI 能取代 GARCH 嗎？

機器學習時代的 GARCH

GARCH 是一個參數模型：它假設波動率演化的特定函數形式。神經網路等機器學習模型是非參數的：它們從數據中學習函數形式。一個自然的問題：AI 能不能直接取代 GARCH？

實證證據是微妙的：

- 對於**樣本內擬合**，神經網路（LSTM、GRU）常常優於 GARCH，因為它們能捕捉更複雜的模式
- 對於**樣本外預測**，GARCH 仍然具有驚人的競爭力——尤其在短期範圍（1–5 天）。其簡約結構（3 個參數）作為對過度擬合的正則化
- 當前的前沿是**混合模型**：用 GARCH 捕捉已知結構（群聚、均值回歸），然後讓 ML 捕捉 GARCH 遺漏的部分（不對稱性、跳躍、非線性）
- GARCH 的優勢：**可解釋性**。風險管理者可以向監管機構解釋 GARCH 參數。解釋 LSTM 的隱藏狀態要困難得多。

我們將在第 8 週重新審視這場競爭。

老實說——我的 *LSTM* 在某些指標上確實打敗了 *GARCH*。但當 *Kenji* 的監管機構問「為什麼你的模型預測了這個數字？」時，我無法打開黑箱。*GARCH* 給你一個答案：它是 ω 加上 α 乘以昨天的衝擊再加上 β 乘以昨天的變異數。在金融領域，可解釋性很重要。

3.11 討論：GARCH 假設與現實

GARCH 符合現實嗎？

GARCH(1,1) 很強大，但做了簡化假設。討論以下問題：

1. **對稱反應**：GARCH 對正負衝擊的處理完全相同（都取平方）。這符合你對金融市場的直覺嗎？你能想到正衝擊增加波動率的情況嗎？
2. **單一體制**：GARCH 假設所有市場條件使用同一組參數。在危機期間 vs. 牛市期間，α 和 β 應該不同嗎？加入體制轉換的取捨是什麼？
3. **常態 vs. t 分配**：為什麼假設殘差服從 Student's t 分配可能產生更好的風險估計？這種額外靈活性的代價是什麼？
4. **參數穩定性**：我們用 2015–2024 年的數據估計了 GARCH。如果使用 2000–2010

年的數據，參數會相似嗎？這對模型的可靠性意味著什麼？

3.12 每週任務

◎ 任務 3：建構 GARCH 引擎

目標：為 Kenji 的關鍵市場指數建構並驗證 GARCH(1,1) 模型。

交付項目：

1. 下載日經 225 和 S&P 500 從 2010 至 2024 年的日頻數據
2. 對每個指數：
   - 繪製日報酬率和平方報酬率；描述你觀察到的群聚模式
   - 計算並繪製平方報酬率的 ACF（30 個落後期）；報告 Ljung–Box $Q(10)$ 統計量和 p 值
   - 使用 Student's t 分配擬合 GARCH(1,1) 模型
   - 報告 $\hat{\omega}$、$\hat{\alpha}$、$\hat{\beta}$、$\hat{\nu}$ 並計算：長期變異數、持續性（$\alpha + \beta$）和半衰期
3. 診斷檢查：繪製平方標準化殘差的 ACF，確認 Ljung–Box 檢定不再拒絕
4. 將 GARCH 的條件波動率與第 2 週的 EWMA 波動率繪製在一起。討論：它們在哪裡一致？在哪裡分歧？
5. 撰寫一段比較備忘錄給 Kenji：為什麼 VolTech 應該使用 GARCH 而非（或除了）EWMA？

🏆 附加挑戰：GARCH 階數選擇

對 S&P 500 數據擬合 GARCH(1,1)、GARCH(1,2)、GARCH(2,1) 和 GARCH(2,2)。使用以下指標比較：

- AIC（赤池資訊準則）
- BIC（貝氏資訊準則）
- 對數概似值

更複雜的模型是否總是勝出？為什麼 GARCH(1,1) 在實務中通常是首選？

提示：AIC 和 BIC 會懲罰模型複雜度。在 `arch` 函式庫中可透過 `result.aic` 和 `result.bic` 存取。

3.13 重點摘要

重點摘要

1. **波動率會群聚**：大的報酬率傾向於跟隨大的報酬率。這可透過平方報酬率的 ACF 和 Ljung–Box 檢定來偵測。它是驅動波動率模型最重要的單一典型事實。
2. **ARCH**（Engle, 1982）：第一個時變波動率的正式模型。今天的變異數取決於過去的平方報酬率。需要很多落後期才能達到好的擬合。
3. **GARCH(1,1)**（Bollerslev, 1986）：在 ARCH 基礎上加入昨天的變異數，大幅減少參數。三個數字——ω、α、β——就能捕捉大多數金融資產的波動率動態。
4. **均值回歸**：GARCH 的長期變異數 $\bar{\sigma}^2 = \omega/(1-\alpha-\beta)$ 作為錨定。衝擊後，波動率會回歸。這是 EWMA 所缺乏的（$\omega = 0 \Rightarrow$ 無錨定）。
5. **MLE**：參數透過最大化觀測數據出現機率來估計。使用 Student's t 分配處理殘差可透過容納厚尾來改善擬合。
6. **限制**：GARCH 對報酬率取平方，消除了正負號。它無法捕捉**槓桿效應**——壞消息與好消息對波動率的不對稱影響。這促使了第 4 週的 GJR-GARCH 模型。

3.14 延伸閱讀

延伸閱讀

1. Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. [原始 ARCH 論文]
2. Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. [原始 GARCH 論文]
3. Engle, R. F. (2001). GARCH 101: the use of ARCH/GARCH models in applied econometrics. *Journal of Economic Perspectives*, 15(4), 157–168. [諾貝爾獎得主的易讀概述]
4. Bollerslev, T., Chou, R. Y., & Kroner, K. F. (1992). ARCH modeling in finance: a review of the theory and empirical evidence. *Journal of Econometrics*, 52(1–2), 5–59.
5. Mandelbrot, B. (1963). The variation of certain speculative prices. *Journal of Business*, 36(4), 394–419. [波動率群聚的早期觀察]
6. Engle, R. F., & Patton, A. J. (2001). What good is a volatility model? *Quantitative Finance*, 1(2), 237–245. [波動率模型的實務評估準則]
7. Hansen, P. R., & Lunde, A. (2005). A forecast comparison of volatility models: does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7), 873–889.
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapters 3–4.
9. Francq, C., & Zakoïan, J.-M. (2019). *GARCH Models: Structure, Statistical Inference and Financial Applications*, 2nd ed. Wiley. [全面的研究生參考文獻]

下 週 預 告

那天晚上，*Alex* 多做了一個實驗。他把 *GARCH* 殘差分開繪製——上漲日和下跌日各自獨立。模式非常明顯：在 -3% 的日子之後，第二天的波動率明顯高於 $+3\%$ 的日子之後。他傳訊息給 *Dr. Lin*：「*GARCH* 假設對稱，但數據顯然不是。我們要怎麼修正這個？」*Dr. Lin* 回覆了一個連結，指向 *Glosten*、*Jagannathan* 和 *Runkle* 在 *1993* 年的一篇論文。標題是：*On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks*。下週：我們用 ***GJR-GARCH*** 打破對稱——並學到在市場中，壞消息的衝擊比好消息更大。

Chapter 4

第 4 週：不對稱波動率 — GJR-GARCH

學習目標

1. 解釋槓桿效應（Leverage Effect），並為負報酬率比正報酬率更能增加波動率提供經濟直覺
2. 設定並詮釋 GJR-GARCH 模型，包括不對稱參數 γ 的角色
3. 建構並詮釋 GARCH 和 GJR-GARCH 的新聞衝擊曲線（News Impact Curve）
4. 比較 GARCH 和 EGARCH 作為替代的不對稱設定
5. 在 Python 中估計 GJR-GARCH，並使用 AIC、BIC 和概似比檢定評估其是否優於標準 GARCH

4.1 本週故事：壞消息打擊更大

場景：VolTech Analytics——週一，上午 9:15

Alex 整個週末都在對他的 GARCH 模型做實驗。週四分析中的殘差圖一直在他腦海中揮之不去：在大的負報酬率之後，隔天的波動率飆升程度遠高於同等幅度的大正報酬率。他傳了訊息給 Dr. Lin，Dr. Lin 回覆了一個連結——Glosten、Jagannathan 和 Runkle 在 1993 年的一篇論文。

Dr. Lin，我讀了你傳給我的 *GJR* 論文。他們顯示股票報酬率和波動率是負相關的——當價格下跌時，波動率不成比例地上升。但為什麼？經濟機制是什麼？

好問題。有兩個主要的解釋，而且它們可能同時在起作用。

不對稱波動率的兩個理論

1. **槓桿效應**（Black, 1976; Christie, 1982）：當一家公司的股價下跌時，其負債權益比自動上升（公司變得更高槓桿）。較高的槓桿意味著更多的財務風險，因此權益持有人要求更高的風險溢酬，進而增加波動率。「槓桿效應」的名稱就來自這

個資產負債表通道。

2. **波動率回饋效應（Volatility Feedback Effect）**（French, Schwert & Stambaugh, 1987; Campbell & Hentschel, 1992）：如果預期未來波動率的增加提高了所要求的報酬率，股價就必須立即下跌以提供那更高的報酬率。所以因果關係是反向的：波動率上升 $\Rightarrow$ 價格下跌。這創造了報酬率與波動率變化之間的負相關。

兩個通道都預測相同的可觀察模式：負報酬率與波動率的較大增加相關，而正報酬率則否。在實務中，這個效應在全球股票市場中都是強烈且普遍的。

這正是 *2020* 年 *3* 月我們投資組合發生的事。市場下跌，第二天的波動率遠高於我們對稱 *GARCH* 模型的預測。我們需要一個能區別對待好消息和壞消息的模型。

這正是 *GJR-GARCH* 所做的。讓我教你它如何運作。

4.2 GJR-GARCH 模型

GJR-GARCH 模型（Glosten, Jagannathan & Runkle, 1993）透過加入一個只在前一期報酬率為負時才「啟動」的項來擴展標準 GARCH。

√× GJR-GARCH(1,1) 模型

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \gamma\, r_{t-1}^2\, \mathbb{I}(r_{t-1} < 0) + \beta\, \sigma_{t-1}^2 \tag{4.1}$$

其中：

- $\omega > 0$：基礎變異數（與 GARCH 相同）
- $\alpha \geq 0$：對所有衝擊的反應（與 GARCH 相同）
- γ：**不對稱係數（Asymmetry Coefficient）**——負衝擊的額外影響
- $\beta \geq 0$：持續性（與 GARCH 相同）
- $\mathbb{I}(r_{t-1} < 0)$：一個**指標函數（Indicator Function）**，當 $r_{t-1} < 0$ 時等於 1，否則等於 0

指標函數是關鍵的創新：它作為一個開關，只在壞消息出現時啟動 γ。當 $\gamma > 0$ 時，負報酬率對未來波動率的影響大於正報酬率。

所以對正報酬率的有效衝擊係數是 α，對負報酬率是 $\alpha + \gamma$？

完全正確。那個不對稱性就是重點所在。讓我給你看數字。

Table 4.1: GJR-GARCH 對正負衝擊的反應方式

衝擊類型	GARCH 影響	GJR-GARCH 影響
正衝擊（$r_{t-1} > 0$）	$\alpha \cdot r_{t-1}^2$	$\alpha \cdot r_{t-1}^2$
負衝擊（$r_{t-1} < 0$）	$\alpha \cdot r_{t-1}^2$	$(\alpha + \gamma) \cdot r_{t-1}^2$
不對稱比率	1.0	$\frac{\alpha+\gamma}{\alpha}$

詮釋 γ

對於典型的權益指數，γ 為正且統計上顯著：

- 如果 $\gamma = 0$：無不對稱性 $\Rightarrow$ GJR-GARCH 退化為標準 GARCH
- 如果 $\gamma > 0$：負衝擊的影響大於正衝擊（槓桿效應）
- 如果 $\gamma < 0$：正衝擊的影響較大（在權益中罕見，但有時在石油等商品中觀察到）

權益指數的典型發現：$\alpha \approx 0.02$–0.05 且 $\gamma \approx 0.10$–0.15，意味著負衝擊的影響大約是正衝擊的 3–4 倍！

除了詮釋 γ 之外，我們還必須驗證模型在統計上表現良好。定態條件確保變異數過程不會隨時間爆炸。

GJR-GARCH 的定態條件

要使 GJR-GARCH(1,1) 為共變異數定態，我們需要：

$$\alpha + \frac{\gamma}{2} + \beta < 1 \tag{4.2}$$

$\gamma/2$ 項的出現是因為在對稱分配下，指標 $\mathbb{I}(r_{t-1} < 0)$ 平均而言有一半的時間處於啟動狀態。長期變異數為：

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \gamma/2 - \beta} \tag{4.3}$$

4.3 新聞衝擊曲線

比較 GARCH 和 GJR-GARCH 最有揭示力的方式是透過**新聞衝擊曲線（News Impact Curve, NIC）**——在其他資訊不變的條件下，繪製明天的條件變異數 σ_t^2 作為今天報酬率 r_{t-1} 的函數。

新聞衝擊曲線

NIC 顯示 σ_t^2 作為 r_{t-1} 的函數，其中 σ_{t-1}^2 固定在其無條件值 $\bar{\sigma}^2$：

$$\mathrm{NIC}(r_{t-1}) = \omega + \alpha\, r_{t-1}^2 + \gamma\, r_{t-1}^2\, \mathbb{I}(r_{t-1} < 0) + \beta\, \bar{\sigma}^2 \tag{4.4}$$

關鍵視覺特性：

- **GARCH**：對稱的拋物線——正負 r_{t-1} 的曲率相同
- **GJR-GARCH**：不對稱的拋物線——左側（負報酬率）比右側更陡峭。在 $r_{t-1} = 0$ 處的「折點」是不對稱性的視覺標記。

新聞衝擊曲線是波動率建模中最重要的診斷工具之一。當你向 *Kenji* 的監管機構報告結果時，這張圖就能精確展示模型如何對好消息和壞消息做出反應。

檢查你的理解

在介紹替代模型之前，確認你能回答：

1. 如果 $\alpha = 0.03$ 且 $\gamma = 0.12$，不對稱比率是多少？這個數字用白話來說是什麼意思？
2. 為什麼定態條件使用 $\gamma/2$ 而不是 γ？

答案：(1) $(\alpha+\gamma)/\alpha = (0.03+0.12)/0.03 = 5.0$，意味著負衝擊的影響是正衝擊的 5 倍。(2) 因為在對稱分配下，指標 $\mathbb{I}(r_{t-1} < 0)$ 平均只有一半的時間處於啟動狀態。

4.4 EGARCH：一種替代方法

Nelson（1991）提出了**指數 GARCH（Exponential GARCH, EGARCH）**模型，採取了根本不同的方法來捕捉不對稱性。

EGARCH(1,1) 模型

$$\ln(\sigma_t^2) = \omega + \alpha\,(|z_{t-1}| - \mathbb{E}[|z_{t-1}|]) + \gamma\, z_{t-1} + \beta\, \ln(\sigma_{t-1}^2) \tag{4.5}$$

其中 $z_{t-1} = r_{t-1}/\sigma_{t-1}$ 為標準化殘差。

與 GJR-GARCH 的主要差異：

- 建模 $\ln(\sigma_t^2)$ 而非 σ_t^2——因此無論參數如何，σ_t^2 永遠為正。α 或 β 不需要正值約束。
- 使用標準化衝擊 z_{t-1} 而非原始報酬率 r_{t-1}。在高斯假設下，$\mathbb{E}[|z_{t-1}|] = \sqrt{2/\pi} \approx 0.798$。使用 Student's t 分配時，此常數取決於自由度 ν，並由 `arch` 函式庫自動

計算。

- 不對稱性透過 γz_{t-1} 進入：當 $\gamma < 0$（典型情況）時，負的 z_{t-1} 對 $\ln(\sigma_t^2)$ 的增加幅度大於正的

符號慣例注意：在 GJR-GARCH 中，$\gamma > 0$ 表示槓桿效應（壞消息增加波動率）。在 EGARCH 中，$\gamma < 0$ 表示相同的效應。兩者在經濟學上表示相同的事——符號差異純粹是模型設定的問題。

Table 4.2: 比較不對稱 GARCH 設定

特徵	GJR-GARCH	EGARCH
變異數方程	σ_t^2	$\ln(\sigma_t^2)$
保證正值？	否（需要約束）	是（永遠）
不對稱機制	指標 $\mathbb{I}(r_{t-1} < 0)$	有號殘差 z_{t-1}
新聞衝擊曲線	分段二次	指數型
不對稱性的典型符號	$\gamma > 0$	$\gamma < 0$
估計複雜度	較低	較高
產業普及度	非常普遍	普遍

兩個模型都能捕捉不對稱性，但方式不同。*GJR-GARCH* 更直觀且更容易估計。*EGARCH* 更靈活且永遠不會產生負變異數。在實務中，兩者都能很好地擬合權益數據。對於 *VolTech* 的客戶報告，我會建議 *GJR-GARCH*，因為它更容易解釋。

第三個值得知道的變體是 Zakoian（1994）提出的**門檻 GARCH（Threshold GARCH, TGARCH）**，它建模的是 σ_t（標準差）而非 σ_t^2。雖然在當前實務中不如 GJR-GARCH 或 EGARCH 常用，但它在較早的文獻中頻繁出現，且共享相同的核心直覺：負衝擊獲得額外權重。

4.5 Python 實作

讓我們估計兩個模型並進行正面比較。`arch` 函式庫讓這變得很簡單。

步驟 1：擬合 GJR-GARCH 和 EGARCH

```
import numpy as np
import pandas as pd
```

```python
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# 1. Standard GARCH(1,1) for baseline
garch = arch_model(returns, vol="Garch", p=1, q=1,
                   dist="t")
res_garch = garch.fit(disp="off")

# 2. GJR-GARCH(1,1) --- 'o=1' adds one asymmetry term
gjr = arch_model(returns, vol="Garch", p=1, o=1, q=1,
                 dist="t")
res_gjr = gjr.fit(disp="off")

# 3. EGARCH(1,1)
egarch = arch_model(returns, vol="EGARCH", p=1, o=1,
                    q=1, dist="t")
res_egarch = egarch.fit(disp="off")

# Compare model selection criteria
print("Model Comparison:")
print(f"{'Model':<14} {'AIC':>10} {'BIC':>10} "
      f"{'LogLik':>10}")
print("-" * 46)
for name, res in [("GARCH", res_garch),
                  ("GJR-GARCH", res_gjr),
                  ("EGARCH", res_egarch)]:
    print(f"{name:<14} {res.aic:>10.1f} {res.bic:>10.1f}"
          f" {res.loglikelihood:>10.1f}")
```

GJR-GARCH 和 *EGARCH* 的 *AIC* 和 *BIC* 都低於標準 *GARCH*。不對稱性確實很重要。

接下來，我們檢視估計的 GJR-GARCH 參數並量化不對稱性：

步驟 2：詮釋 GJR-GARCH 參數

```python
# GJR-GARCH parameters
print("\nGJR-GARCH(1,1) Parameters:")
print(res_gjr.summary().tables[1])

omega = res_gjr.params["omega"]
alpha = res_gjr.params["alpha[1]"]
gamma = res_gjr.params["gamma[1]"]
beta  = res_gjr.params["beta[1]"]

print(f"\nomega = {omega:.6f}")
print(f"alpha = {alpha:.4f}  (positive shock impact)")
print(f"gamma = {gamma:.4f}  (additional negative impact)")
print(f"beta  = {beta:.4f}  (persistence)")
print(f"\nEffective impact of positive shock: "
      f"alpha = {alpha:.4f}")
print(f"Effective impact of negative shock: "
      f"alpha + gamma = {alpha + gamma:.4f}")
print(f"Asymmetry ratio: "
      f"{(alpha + gamma) / alpha:.1f}x")

# Stationarity check
persist = alpha + gamma / 2 + beta
longrun_var = omega / (1 - persist)
longrun_vol = np.sqrt(longrun_var * 252)
halflife = np.log(0.5) / np.log(persist)

print(f"\nPersistence: {persist:.4f}")
print(f"Long-run annualized vol: {longrun_vol:.2f}%")
print(f"Half-life: {halflife:.1f} days")
```

注意不對稱比率。如果大約是 *3–4*，那表示負報酬率對明天波動率的影響是同等幅度正報酬率的三到四倍。這就是槓桿效應的實際作用。

現在是關鍵的視覺比較——新聞衝擊曲線：

步驟 3：繪製新聞衝擊曲線

```python
# News Impact Curve: GARCH vs. GJR-GARCH
shock_range = np.linspace(-5, 5, 500)  # shock in %

# GARCH NIC
omega_g = res_garch.params["omega"]
alpha_g = res_garch.params["alpha[1]"]
beta_g  = res_garch.params["beta[1]"]
lrv_g   = omega_g / (1 - alpha_g - beta_g)
nic_garch = omega_g + alpha_g * shock_range**2 \
            + beta_g * lrv_g

# GJR-GARCH NIC
lrv_gjr = omega / (1 - alpha - gamma/2 - beta)
nic_gjr = (omega + alpha * shock_range**2
           + gamma * shock_range**2
             * (shock_range < 0)  # Boolean: 1 if negative, 0 if positive
           + beta * lrv_gjr)

# EGARCH NIC (approximate via exponential)
omega_e = res_egarch.params["omega"]
alpha_e = res_egarch.params["alpha[1]"]
gamma_e = res_egarch.params["gamma[1]"]
beta_e  = res_egarch.params["beta[1]"]
lrv_e   = omega_e / (1 - beta_e)
z_range = shock_range / np.sqrt(lrv_g)  # standardize
nic_egarch = np.exp(
    lrv_e + alpha_e * (np.abs(z_range) - np.sqrt(2/np.pi))
    + gamma_e * z_range)

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(shock_range, nic_garch, color="#1B3A5C",
```

```
        linewidth=2, label="GARCH(1,1)")
ax.plot(shock_range, nic_gjr, color="#C0392B",
        linewidth=2, label="GJR-GARCH(1,1)")
ax.plot(shock_range, nic_egarch, color="#2D7D5E",
        linewidth=2, linestyle="--",
        label="EGARCH(1,1)")
ax.axvline(0, color="gray", linestyle="--",
           linewidth=0.8)
ax.set_xlabel("Return Shock (%)")
ax.set_ylabel(r"Conditional Variance $\sigma_t^2$")
ax.set_title("News Impact Curve: All Three Models")
ax.legend()
plt.tight_layout()
plt.savefig("week04_nic.png", dpi=150)
plt.show()
```

不對稱性非常明顯。*GJR-GARCH* 曲線的左側陡峭得多——一個 -3% 的衝擊產生的變異數遠高於 $+3\%$ 的衝擊。標準 *GARCH* 完全對稱，完全忽略了這一點。而 *EGARCH* 曲線顯示相同的不對稱性，但呈平滑的指數形狀而非折點——正好對應表 *4.2*。

最後，我們比較 COVID 危機期間的條件波動率路徑：

步驟 4：COVID 期間的條件波動率比較

```
# Extract conditional volatility (annualized)
vol_garch = (res_garch.conditional_volatility
             * np.sqrt(252))
vol_gjr   = (res_gjr.conditional_volatility
             * np.sqrt(252))

# Focus on COVID period
mask = (returns.index >= "2019-10-01") & \
       (returns.index <= "2020-09-30")

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(vol_garch[mask].index, vol_garch[mask],
        color="#1B3A5C", linewidth=1.5,
```

```
        label="GARCH(1,1)")
ax.plot(vol_gjr[mask].index, vol_gjr[mask],
        color="#C0392B", linewidth=1.5,
        label="GJR-GARCH(1,1)")
ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red",
           label="COVID-19 crash")
ax.set_title("Conditional Volatility: GARCH vs. "
             "GJR-GARCH (COVID Period)")
ax.set_ylabel("Annualized Volatility (%)")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("week04_covid_comparison.png", dpi=150)
plt.show()

# Quantify the difference
diff = vol_gjr[mask] - vol_garch[mask]
print(f"Mean GJR-GARCH vol during crash: "
      f"{vol_gjr[mask].mean():.1f}%")
print(f"Mean GARCH vol during crash: "
      f"{vol_garch[mask].mean():.1f}%")
print(f"Average difference: {diff.mean():.1f} "
      f"percentage points")
print(f"Max difference: {diff.max():.1f} "
      f"percentage points")
```

4.6 應用：不對稱性檢定

場景：會議室——週四，下午 2:30

Alex 將新聞衝擊曲線投影到螢幕上。不對稱性一目了然。Kenji 身體前傾。

我們上週測試的標準 *GARCH* 模型把 *3%* 的漲幅和 *3%* 的跌幅等同看待。但看看 *GJR* 曲線——*3%* 的跌幅對明天波動率的影響幾乎是四倍。這與我們的經驗完全吻合。

而且 *COVID* 期間的條件波動率比較也證實了這一點。*GJR-GARCH* 在崩盤期間上升更快、峰值更高，因為每一個負報酬率的日子都收到了額外的 γ 加速。標準 *GARCH* 系統性地太慢且太低。

分析得好。但讓我補充一個重要的細微之處。**概似比檢定（*Likelihood Ratio Test*）**可以告訴我們這個改善在統計上是否顯著。

概似比檢定：GARCH vs. GJR-GARCH

由於 GARCH 嵌套於 GJR-GARCH 中（令 $\gamma = 0$），我們可以使用概似比檢定：

$$LR = 2\left[\mathcal{L}_{\text{GJR}} - \mathcal{L}_{\text{GARCH}}\right] \sim \chi^2(1) \tag{4.6}$$

如果 LR 統計量超過 $\chi^2(1)$ 的臨界值（5% 顯著水準下為 3.84），我們拒絕 $\gamma = 0$ 的虛無假設，並得出不對稱性在統計上是顯著的結論。

概似比檢定

```python
from scipy import stats

# Likelihood ratio test
lr_stat = 2 * (res_gjr.loglikelihood
               - res_garch.loglikelihood)
p_value = 1 - stats.chi2.cdf(lr_stat, df=1)
print(f"LR statistic: {lr_stat:.2f}")
print(f"p-value: {p_value:.6f}")
print(f"Reject H0 (gamma=0) at 5%: "
      f"{'Yes' if p_value < 0.05 else 'No'}")
```

如預期，*LR* 檢定強烈拒絕虛無假設——$\gamma = 0$ 顯然不被數據支持。不對稱性不是理論上的好奇心；它是權益報酬率中統計上顯著的特徵。

這正是能讓 *VolTech* 與競爭者區別開來的證據驅動分析。*Kenji*，如果你的監管機構問「為什麼選 *GJR* 而不是標準 *GARCH*？」，新聞衝擊曲線和 *LR* 檢定就是你的答案。但我有一個更大的問題：擬合好模型是一回事——它真的能預測明天的風險嗎？

4.7 模型診斷

如同第 3 週，我們透過檢驗標準化殘差來確認 GJR-GARCH 是否充分捕捉了數據的結構。

GJR-GARCH 診斷檢查

```python
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.graphics.tsaplots import plot_acf

# Standardized residuals
std_resid_gjr = res_gjr.std_resid.dropna()

# Ljung-Box test on squared standardized residuals
lb = acorr_ljungbox(std_resid_gjr ** 2, lags=10,
                    return_df=True)
print("Ljung-Box on squared std residuals (GJR):")
print(lb)

# Skewness comparison: GARCH vs. GJR-GARCH
std_resid_g = res_garch.std_resid.dropna()
print(f"\nResidual skewness (GARCH):     "
      f"{std_resid_g.skew():.4f}")
print(f"Residual skewness (GJR-GARCH): "
      f"{std_resid_gjr.skew():.4f}")
# Note: pandas kurtosis() returns excess kurtosis
print(f"Excess kurtosis (GARCH):       "
      f"{std_resid_g.kurtosis():.4f}")
print(f"Excess kurtosis (GJR-GARCH):   "
      f"{std_resid_gjr.kurtosis():.4f}")
```

Ljung–Box p 值很大，所以 *GJR-GARCH* 去除了平方殘差中的自相關。而且偏態改善了——比標準 *GARCH* 殘差更接近零。模型正在捕捉 *GARCH* 遺留的部分不對稱性。

偏態的改善正是我們預期的。透過 γ 給負報酬率額外權重，*GJR-GARCH* 能部分解釋數據中的不對稱模式。然而，殘差的超額峰態仍然為正，提醒我們即使使用

> *Student's t* 分配的 *GJR-GARCH* 也無法捕捉所有的極端行為。
> 還有一個值得一提的診斷工具：***Engle–Ng* 符號與規模偏差檢定**（*Engle & Ng, 1993*）可以在選擇模型之前正式檢定殘差中是否存在不對稱性。這些檢定將平方標準化殘差對過去衝擊的符號和幅度的指標變數做迴歸。如果符號偏差檢定拒絕，就確認了 *GJR-GARCH* 這類不對稱設定是有必要的。

4.8 AI 焦點：機器學習中的不對稱性

不對稱性：參數 vs. 非參數方法

GARCH 和 GJR-GARCH 透過明確的設計選擇來建模不對稱性：指標函數 $\mathbb{I}(r_{t-1} < 0)$ 或指數設定。機器學習提供了不同的路徑：讓模型從數據中發現不對稱性。

- **神經網路**（LSTM、GRU）：可以透過非線性激活函數隱含地學習不對稱反應。不需要預先指定函數形式。然而，學到的不對稱性難以解釋。
- **分量迴歸（Quantile Regression）**：分別建模報酬率分配的不同分量，自然捕捉不對稱的尾部，而無需假設特定的波動率過程。
- **混合方法**：使用 GJR-GARCH 作為 ML 模型的特徵生成器。GJR 條件波動率、不對稱指標和新聞衝擊曲線值成為神經網路的輸入，讓它學習殘餘模式。

關鍵洞見：GJR-GARCH 以單一參數 γ 提供可解釋的不對稱性。ML 以犧牲可解釋性為代價提供靈活的不對稱性。最優方法通常結合兩者。

4.9 討論：實務中的不對稱性

不對稱性何時最重要？

1. **資產類別差異**：槓桿效應在權益中最強。那貨幣、商品或債券呢？你會預期油價的 γ 是正的、負的還是接近零？為什麼？
2. **危機放大**：在 2020 年 COVID 崩盤期間，槓桿效應放大了波動率飆升。這會不會產生回饋循環，其中不斷上升的波動率導致進一步拋售，進而更加增加波動率？
3. **實務影響**：如果你使用標準 GARCH 而非 GJR-GARCH 計算風險值（VaR），在市場下跌後 VaR 估計會有什麼不同？哪個模型會給出更保守（更安全）的風險估計？
4. **模型選擇**：你有 GARCH、GJR-GARCH 和 EGARCH。三者都擬合得很好。你會向需要向監管機構解釋模型的風險管理者推薦哪一個？如果監管機構問「為

什麼是這個模型？」怎麼辦？

4.10 每週任務

◎ 任務 4：不對稱性報告

目標：為 Kenji 的投資組合指數建構全面的不對稱波動率分析。

交付項目：

1. 對日經 225 和 S&P 500（2010–2024）：
   - 擬合 GARCH(1,1)、GJR-GARCH(1,1) 和 EGARCH(1,1)，全部使用 Student's t 分配
   - 報告每個模型的參數、AIC、BIC 和對數概似值
   - 進行概似比檢定：GARCH vs. GJR-GARCH
2. 對 GJR-GARCH 模型：
   - 報告不對稱比率 $(\alpha + \gamma)/\alpha$
   - 使用 GJR-GARCH 定態公式 (4.2)計算長期波動率和半衰期
3. 對每個指數繪製所有三個模型的新聞衝擊曲線在同一張圖上
4. 繪製 COVID 期間（2019 年 10 月至 2020 年 9 月）的條件波動率比較（GARCH vs. GJR-GARCH）
5. 撰寫一頁備忘錄給 Kenji，建議 VolTech 今後應使用哪個波動率模型，並以你的分析證據支持

🏆 附加挑戰：跨市場不對稱性

對至少五種不同的指數或資產擬合 GJR-GARCH(1,1)：

- S&P 500、日經 225、FTSE 100（權益）
- 黃金（商品）、USD/JPY（貨幣）

比較不同資產類別估計的 γ。哪裡的不對稱性最強？哪裡最弱或相反？撰寫簡短的摘要解釋每個的經濟直覺。

提示：對於黃金和外匯，想想槓桿效應的邏輯（負債權益比）是否適用。

4.11 重點摘要

重點摘要

1. **槓桿效應**：負報酬率比同等幅度的正報酬率更能增加未來波動率。解釋包括資產負債表的槓桿通道（Black, 1976）和波動率回饋效應（Campbell & Hentschel, 1992）。
2. **GJR-GARCH**：以指標函數 $\mathbb{I}(r_{t-1} < 0)$ 擴展 GARCH，給予負衝擊額外權重。一個額外的參數 γ 捕捉不對稱性。
3. **新聞衝擊曲線**：視覺化模型如何對衝擊做出反應的關鍵診斷工具。GARCH 給出對稱拋物線；GJR-GARCH 給出在零點有折點的不對稱拋物線。
4. **EGARCH**：建模 $\ln(\sigma_t^2)$ 的替代方法，保證正值。使用有號標準化殘差處理不對稱性。同樣有效但更難解釋。
5. **概似比檢定**：由於 GARCH 嵌套於 GJR-GARCH 中（$\gamma = 0$），我們可以正式檢定不對稱性是否統計顯著。對權益來說，幾乎總是顯著的。
6. **實務影響**：忽略不對稱性意味著在市場下跌後低估風險——而那正是準確風險衡量最重要的時候。

4.12 延伸閱讀

延伸閱讀

1. Glosten, L. R., Jagannathan, R., & Runkle, D. E. (1993). On the relation between the expected value and the volatility of the nominal excess return on stocks. *Journal of Finance*, 48(5), 1779–1801. [原始 GJR-GARCH 論文]
2. Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: a new approach. *Econometrica*, 59(2), 347–370. [原始 EGARCH 論文]
3. Black, F. (1976). Studies of stock price volatility changes. *Proceedings of the American Statistical Association, Business and Economics Section*, 177–181. [槓桿效應的起源]
4. Christie, A. A. (1982). The stochastic behavior of common stock variances: value, leverage, and interest rate effects. *Journal of Financial Economics*, 10(4), 407–432.
5. French, K. R., Schwert, G. W., & Stambaugh, R. F. (1987). Expected stock returns and volatility. *Journal of Financial Economics*, 19(1), 3–29. [報酬率-波動率關係的早期證據]
6. Campbell, J. Y., & Hentschel, L. (1992). No news is good news: an asymmetric model of changing volatility in stock returns. *Journal of Financial Economics*, 31(3), 281–318. [波動率回饋效應]
7. Engle, R. F., & Ng, V. K. (1993). Measuring and testing the impact of news on volatility. *Journal of Finance*, 48(5), 1749–1778. [新聞衝擊曲線方法論]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
9. Francq, C., & Zakoïan, J.-M. (2019). *GARCH Models*, 2nd ed. Wiley. Chapter 4.
10. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. [不對稱波動率在風險管理中的實務觀點]
11. Zakoian, J.-M. (1994). Threshold heteroskedastic models. *Journal of Economic Dynamics and Control*, 18(5), 931–955. [原始 TGARCH 論文]

下 週 預 告

會議結束後，*David* 把 *Alex* 拉到一旁。「你的 *GJR* 模型顯然比 *GARCH* 擬合得更好。新聞衝擊曲線是很棒的視覺化。但我需要問一個困難的問題：這個模型能預測明天的波動率嗎？不只是擬合昨天的數據？」*Alex* 還沒來得及回答，*Priya* 就帶著筆電走了過來。「我一直在做樣本外預測測試。*GARCH*、*GJR-GARCH*、*EGARCH*，還有我的 *LSTM*——全部正面對決。結果...很有趣。」她笑了。「想看嗎？」下週：我們進入**波動率預測**的世界——在那裡，模型不是以擬合過去的好壞來評判，而是以預測未來的準確度來衡量。

Chapter 5

第 5 週：波動率預測

學習目標

1. 區分樣本內擬合（In-Sample Fit）與樣本外預測（Out-of-Sample Forecasting），並解釋為何後者才是模型的真正考驗
2. 對 GARCH 和 GJR-GARCH 模型實作滾動視窗一步 ahead 預測程序
3. 使用 RMSE、MAE 和 QLIKE 損失函數評估波動率預測
4. 進行 Mincer–Zarnowitz 迴歸以檢定預測效率
5. 在結構化的預測競賽中比較多個波動率模型

5.1 本週故事：模型大賽

場景：VolTech Analytics——週一，上午 9:00

Alex 到達時發現主會議室已被佔用。Priya 在白板上寫滿了模型名稱、損失函數和滾動視窗圖表。Dr. Lin 坐在角落，帶著會心微笑啜飲著茶。

歡迎來到預測競技場，*Alex*。上週你展示了 *GJR-GARCH* 比 *GARCH* 擬合得更好。但 *David* 問了正確的問題：它能預測明天的波動率嗎？這就是我們今天要測試的——樣本外。

擬合得好和預測得好有什麼差別？

一個完美擬合過去的模型可能只是在記憶雜訊而非學習模式。這就是經典的**過度擬合（*Overfitting*）**問題。唯一誠實的測試是用過去的數據估計模型，然後預測模型從未見過的時期。

完全正確。而我已經準備好了終極測試：*GARCH*、*GJR-GARCH*、*EGARCH* 和我的 *LSTM*——全部預測同一個樣本外時期。相同的數據、相同的評估指標。沒有藉口。

模型大賽。我加入。

5.2 預測框架

在執行任何預測之前，我們需要嚴謹地建立評估框架。

樣本外預測設計

數據被分為兩個不重疊的時期：

- **樣本內**（估計視窗）：用於估計模型參數。模型從這段時期學習模式。
- **樣本外**（評估視窗）：用於評估預測。模型在估計期間從未見過這些數據。

對於波動率預測，我們通常使用**滾動視窗**方法：

1. 固定一個 W 個觀測值的估計視窗
2. 用觀測值 $1, 2, \ldots, W$ 估計模型
3. 預測 $\hat{\sigma}^2_{W+1}$（向前一步）
4. 將視窗向前滾動：用 $2, 3, \ldots, W+1$ 估計
5. 預測 $\hat{\sigma}^2_{W+2}$
6. 重複直到數據結束

另一種是**擴展視窗（Expanding Window）**，它保持起始日期固定，每一步增長估計樣本。當你懷疑存在結構性斷裂（參數隨時間變化）時，滾動視窗是首選。

波動率代理變數問題

波動率預測的一個根本挑戰：我們永遠觀察不到「真實」波動率 σ_t^2。我們只觀察到報酬率 r_t。要評估預測，我們需要一個**波動率代理變數（Volatility Proxy）**——對真實波動率的估計。常見選擇：

- **平方報酬率** r_t^2：簡單但雜訊極大。每天一個觀測值對變異數的估計極不精確。
- **實現變異數** RV_t：日內平方報酬率之和（第 2 週）。當日內數據可用時精確得多。
- **範圍估計量（Range-Based Estimators）**：使用最高價、最低價、開盤價和收盤價。比 r_t^2 精確但不如 RV_t。

代理變數的選擇很重要：使用 r_t^2 作為目標會引入大量雜訊，可能扭曲模型排名。

條件允許時，使用 RV_t 或範圍估計量。

這是一個微妙但關鍵的要點。如果我們用 r_t^2 來評估預測，每個模型看起來都很差，因為 r_t^2 太嘈雜了。模型的排名通常被保留，但表面上的準確度低得多。*Patton*（*2011*）證明某些損失函數——包括 *QLIKE*——無論使用哪個代理變數都能產生一致的模型排名。

5.3 預測評估指標

我們如何衡量波動率預測 $\hat{\sigma}_t^2$ 與實現代理 σ_t^{2*} 的「接近」程度？

波動率預測的損失函數

對於 T 個樣本外預測：

1. 均方誤差（MSE）/ 均方根誤差（RMSE）：

$$\text{MSE} = \frac{1}{T}\sum_{t=1}^{T}(\sigma_t^{2*} - \hat{\sigma}_t^2)^2, \quad \text{RMSE} = \sqrt{\text{MSE}} \tag{5.1}$$

2. 平均絕對誤差（MAE）：

$$\text{MAE} = \frac{1}{T}\sum_{t=1}^{T}|\sigma_t^{2*} - \hat{\sigma}_t^2| \tag{5.2}$$

3. QLIKE（擬概似損失）：

$$\text{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left(\ln(\hat{\sigma}_t^2) + \frac{\sigma_t^{2*}}{\hat{\sigma}_t^2}\right) \tag{5.3}$$

為什麼需要多個損失函數？

不同的損失函數懲罰不同類型的預測誤差：

- **MSE**：對大誤差懲罰嚴重（平方）。適合偵測偶爾產生糟糕預測的模型。
- **MAE**：對所有誤差按比例處理。比 MSE 對異常值更穩健。
- **QLIKE**：懲罰相對誤差（對過度預測和不足預測的懲罰不對稱）。對波動率特別有用，因為它是尺度無關的，且對波動率代理變數的選擇穩健（Patton, 2011）。

一個在所有三個損失函數上都勝出的模型顯然是優越的。當排名不同時，實務上通常偏好 QLIKE。

5.4 Mincer–Zarnowitz 迴歸

除了損失函數，我們還可以檢定預測是否**有效率**——它們是否包含了所有可用的資訊？

Mincer–Zarnowitz 迴歸

將實現代理對預測做迴歸：

$$\sigma_t^{2*} = a + b\,\hat{\sigma}_t^2 + u_t \tag{5.4}$$

完美的預測意味著 $a = 0$ 且 $b = 1$。使用 F 檢定檢驗聯合虛無假設 $H_0 : a = 0,\ b = 1$。如果被拒絕，預測是**有偏的**或**無效率的**：

- $b < 1$：預測波動太大（對衝擊過度反應）
- $b > 1$：預測太平滑（對衝擊反應不足）
- $a \neq 0$：預測有系統性偏差
- R^2：預測解釋了實現變異數的比例

所以 *Mincer–Zarnowitz* 迴歸的 R^2 告訴我們模型捕捉了實際波動率中多少變異？

正是如此。當使用 r_t^2 作為代理變數時，*5–15%* 的 R^2 是典型的——因為 r_t^2 太嘈雜了。使用實現變異數時，*30–50%* 的 R^2 是常見的。不要因為對 r_t^2 的低 R^2 而沮喪；重要的是模型的相對排名。

5.5 Python 實作：預測管道

是時候建構完整的預測管道了。我們將為三個 *GARCH* 家族模型進行滾動視窗一步 *ahead* 預測。

步驟 1：數據準備與預測設定

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
import warnings
warnings.filterwarnings("ignore")
```

```python

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                    end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Forecast setup
window = 1000          # rolling estimation window
n_total = len(returns)
n_oos = n_total - window  # out-of-sample observations
print(f"Total obs: {n_total}")
print(f"Window: {window}")
print(f"Out-of-sample: {n_oos}")
```

接下來，我們實作滚動預測迴圈。這是計算的核心——對於樣本外期間的每一天，我們重新估計模型並產生一步 ahead 預測：

步驟 2：滚動一步 ahead 預測

```python
# Models to compare
model_specs = {
    "GARCH":     {"vol": "Garch", "p": 1, "o": 0,
                  "q": 1},
    "GJR-GARCH": {"vol": "Garch", "p": 1, "o": 1,
                  "q": 1},
    "EGARCH":    {"vol": "EGARCH", "p": 1, "o": 1,
                  "q": 1},
}

# Store forecasts
forecasts = {}
for name, spec in model_specs.items():
    fc = np.full(n_oos, np.nan)
    # Note: this takes ~3-5 min for 3 models
    for i in range(n_oos):
        train = returns.iloc[i:i + window]
        model = arch_model(train, dist="t", **spec)
        try:
```

```
            res = model.fit(disp="off",
                                show_warning=False)
            fv = res.forecast(horizon=1)
            fc[i] = fv.variance.values[-1, 0]
        except Exception:
            fc[i] = np.nan
    forecasts[name] = pd.Series(
        fc, index=returns.index[window:])
    print(f"{name}: done ({np.isnan(fc).sum()} "
          f"failures)")

# Realized proxy: squared returns
realized = returns.iloc[window:] ** 2
```

迴圈每天重新估計模型——聽起來很慢。

確實需要幾分鐘。但這是誠實的方法。每個預測只使用預測當時可得的資訊——沒有未來數據洩漏。在生產系統中，我們會用平行處理來最佳化，但對於研究來說，這是黃金標準。

現在我們計算評估指標：

步驟 3：預測評估

```
def evaluate_forecasts(realized, forecasts):
    """Compute RMSE, MAE, QLIKE for each model."""
    results = {}
    for name, fc in forecasts.items():
        valid = ~(fc.isna() | realized.isna())
        r = realized[valid].values
        f = fc[valid].values
        # Avoid log(0) or division by zero
        f = np.maximum(f, 1e-10)

        mse  = np.mean((r - f) ** 2)
        rmse = np.sqrt(mse)
        mae  = np.mean(np.abs(r - f))
        qlike = np.mean(np.log(f) + r / f)
```

```python

        results[name] = {"RMSE": rmse, "MAE": mae,
                         "QLIKE": qlike}
    return pd.DataFrame(results).T

metrics = evaluate_forecasts(realized, forecasts)
print("\nForecast Evaluation (proxy = r^2):")
print(metrics.round(4))
print(f"\nBest model by RMSE:  "
      f"{metrics['RMSE'].idxmin()}")
print(f"Best model by MAE:   "
      f"{metrics['MAE'].idxmin()}")
print(f"Best model by QLIKE: "
      f"{metrics['QLIKE'].idxmin()}")
```

步驟 4：Mincer–Zarnowitz 迴歸

```python
import statsmodels.api as sm

print("\nMincer-Zarnowitz Regressions:")
print("-" * 50)
for name, fc in forecasts.items():
    valid = ~(fc.isna() | realized.isna())
    r = realized[valid].values
    f = fc[valid].values
    X = sm.add_constant(f)
    ols = sm.OLS(r, X).fit()
    a, b = ols.params
    r2 = ols.rsquared
    # F-test for H0: a=0, b=1
    f_test = ols.f_test("(const = 0), (x1 = 1)")
    p_val = float(f_test.pvalue)
    print(f"{name:<14} a={a:>8.4f}  b={b:>6.4f}  "
          f"R2={r2:.4f}  F-pval={p_val:.4f}")
```

R^2 值相當低——大約 *5–10%*。

使用 r_t^2 作為代理變數時，這是預期的。記住，單一的平方報酬率是對日變異數極度嘈雜的估計。重要的是模型之間的排名，而非絕對的 R^2。

最後，我們將預測與實現波動率進行視覺化比較：

步驟 5：預測視覺化

```python
# Plot forecasts vs. realized (annualized vol)
fig, ax = plt.subplots(figsize=(14, 6))
rv_ann = np.sqrt(realized * 252)  # annualized
ax.plot(rv_ann.index, rv_ann, color="gray",
        linewidth=0.3, alpha=0.5,
        label="Realized proxy (ann.)")

colors = {"GARCH": "#1B3A5C",
          "GJR-GARCH": "#C0392B",
          "EGARCH": "#2D7D5E"}
for name, fc in forecasts.items():
    fc_vol = np.sqrt(fc * 252)  # annualized
    ax.plot(fc_vol.index, fc_vol, color=colors[name],
            linewidth=1.2, alpha=0.8, label=name)

ax.axvspan("2020-02-20", "2020-04-01",
           alpha=0.1, color="red",
           label="COVID-19 crash")
ax.set_title("Out-of-Sample Volatility Forecasts: "
             "S&P 500")
ax.set_ylabel("Annualized Volatility (%)")
ax.legend(loc="upper right")
ax.set_ylim(0, 120)
plt.tight_layout()
plt.savefig("week05_forecast_comparison.png", dpi=150)
plt.show()
```

Diebold–Mariano 檢定

比較損失函數值告訴我們哪個模型的平均損失較低，但無法告訴我們差異是否**統**

計顯著。Diebold–Mariano（DM）檢定（1995）將此比較正式化。定義損失差異：

$$d_t = L(\hat{\sigma}^2_{1,t}) - L(\hat{\sigma}^2_{2,t})$$

其中 $L(\cdot)$ 為任意損失函數（例如平方誤差）。在 H_0：兩個模型具有相同的預測準確度（$E[d_t] = 0$）下。DM 統計量使用 HAC 標準誤以考慮損失差異中的序列相關：

$$DM = \frac{\bar{d}}{\hat{\sigma}_{\bar{d}}} \xrightarrow{d} N(0, 1)$$

顯著的 DM 統計量（例如 $|DM| > 1.96$）表示一個模型顯著優於另一個，而非僅僅是偶然。

步驟 6：Diebold–Mariano 檢定

```python
from scipy import stats

def dm_test(realized, fc1, fc2, loss="qlike"):
    """Diebold-Mariano test for equal predictive
    accuracy. Returns DM statistic and p-value."""
    valid = ~(fc1.isna() | fc2.isna() | realized.isna())
    r = realized[valid].values
    f1, f2 = fc1[valid].values, fc2[valid].values
    f1, f2 = np.maximum(f1, 1e-10), np.maximum(f2, 1e-10)

    if loss == "qlike":
        L1 = np.log(f1) + r / f1
        L2 = np.log(f2) + r / f2
    else:  # squared error
        L1 = (r - f1) ** 2
        L2 = (r - f2) ** 2

    d = L1 - L2  # loss differential
    T = len(d)
    d_bar = d.mean()
    # Newey-West HAC variance (lag = int(T^(1/3)))
    lag = int(T ** (1 / 3))
    gamma_0 = np.var(d, ddof=1)
    gamma_sum = sum(
```

```
        (1 - k / (lag + 1)) * np.cov(d[k:], d[:-k])[0, 1]
        for k in range(1, lag + 1))
    var_d = (gamma_0 + 2 * gamma_sum) / T
    dm_stat = d_bar / np.sqrt(max(var_d, 1e-20))
    p_val = 2 * (1 - stats.norm.cdf(abs(dm_stat)))
    return dm_stat, p_val

# Compare GJR-GARCH vs. GARCH
dm, pv = dm_test(realized, forecasts["GARCH"],
                 forecasts["GJR-GARCH"], "qlike")
print(f"DM test (GARCH vs GJR-GARCH, QLIKE):")
print(f"  DM = {dm:.3f}, p-value = {pv:.4f}")
dm2, pv2 = dm_test(realized, forecasts["GARCH"],
                   forecasts["EGARCH"], "qlike")
print(f"DM test (GARCH vs EGARCH, QLIKE):")
print(f"  DM = {dm2:.3f}, p-value = {pv2:.4f}")
```

DM 檢定確認了損失函數表格所暗示的：*GJR-GARCH* 對 *GARCH* 的改善在統計上是顯著的。這意味著不只是雜訊——不對稱參數確實捕捉了真正有用的預測資訊。

5.6 應用：誰贏得模型大賽？

場景：會議室——週四，下午 3:00

VolTech 全體團隊聚集在螢幕前，Alex 報告預測結果。連 David 都在場——他很少參加技術會議。

以下是結果。我在 *2019–2024* 年的樣本外期間對三個模型進行了滾動一步 *ahead* 預測：

關鍵發現：

1. ***GJR-GARCH* 在所有三個損失函數上一致優於標準 *GARCH***。不對稱參數 γ 不只是更好的樣本內擬合——它真正改善了樣本外預測。
2. ***EGARCH* 的表現與 *GJR-GARCH* 相近**。兩個不對稱模型都捕捉了對稱 *GARCH*

Table 5.1: 波動率預測競賽結果（S&P 500）*

模型	RMSE	MAE	QLIKE
GARCH(1,1)	5.82	2.14	3.72
GJR-GARCH(1,1)	5.61	2.03	3.65
EGARCH(1,1)	5.65	2.10	3.66

*S&P 500 日頻數據的代表性數值（2019–2024 樣本外）。您的結果可能因確切的日期範圍和隨機種子而略有不同。較低的值表示更好的預測。

遺漏的資訊。

3. **改善幅度不大但一致**。不要期望巨大的增益；波動率本質上很難預測。但即使是小的改善在風險管理中也會隨時間累積。
4. **所有模型在危機初期都表現不佳**。*COVID* 崩盤的最初幾天，每個模型的預測都很差。

這個結果與 *Hansen & Lunde*（*2005*）一致，他們測試了 *330* 個波動率模型，發現很少有模型能持續打敗一個設定良好的 *GARCH(1,1)*。關鍵詞是「設定良好」——加入不對稱性和使用正確的分配通常有幫助，但奇特的模型複雜度很少有用。

坦白說，我的 *LSTM* 確實在 *RMSE* 上以大約 *2%* 的差距打敗了 *GJR-GARCH*——但在 *QLIKE* 上更差。而且訓練時間長了 *100* 倍。誠實的答案是：在一天的預測範圍內，*GARCH* 家族模型與深度學習具有驚人的競爭力。*AI* 的優勢更多出現在較長的預測範圍和更豐富的特徵集上。

這些是有用的結果。但我需要問：我們花了四週建模和預測波動率。什麼時候才能把這轉化為我的董事會能理解的東西？他們不在乎 *QLIKE* 或 R^2。他們想要一個數字：我們最多能損失多少？

那個數字叫做**風險值（*Value-at-Risk*）**——而它正是我們下週要建構的。

5.7 AI 焦點：預測的前沿

機器學習的波動率預測

傳統的 GARCH vs. ML 辯論已經相當成熟：

- **短期範圍（1–5 天）**：GARCH 家族模型仍具競爭力。其簡約結構防止過度擬合，而一天 ahead 的預測由昨天的變異數（β 項）主導，ML 不容易改善。
- **中期範圍（1–4 週）**：ML 模型開始顯示優勢，特別是當結合額外特徵時（選擇權隱含波動率、新聞情緒、交易量）。
- **長期範圍（1 個月以上）**：ML 和混合模型傾向於表現更好，因為它們能捕捉 GARCH 固定參數遺漏的體制變化和非線性模式。
- **混合模型**：最有前景的方法結合了 GARCH 的結構性知識與 ML 的靈活性。例如：將 GJR-GARCH 條件波動率作為 LSTM 的特徵輸入，搭配 VIX、交易量和情緒指標。

核心訊息：GARCH 並未過時——它是任何 AI 模型都必須打敗才能證明其複雜度正當性的強力基準。

5.8 討論：預測的哲學

什麼是好的預測？

1. **準確度 vs. 可解釋性**：一個產生稍微好一點預測的黑箱模型 vs. 一個你能向監管機構解釋的簡單模型。你會為退休基金選哪個？為對沖基金呢？
2. **代理變數問題**：我們用 r_t^2 評估，這非常嘈雜。如果你有 5 分鐘日內數據的存取權，你會如何使用它？模型排名會改變嗎？
3. **結構性斷裂**：我們使用了固定長度的滾動視窗。如果市場結構改變（例如演算法交易的興起）會怎樣？視窗應該更短還是更長？
4. **經濟價值**：更好的統計預測不一定轉化為更好的經濟結果（投資組合報酬率、風險管理）。你會如何測試 GJR-GARCH 相比標準 GARCH 是否真的能省錢？

5.9 每週任務

◎ 任務 5：預測模型大賽

目標：建構完整的波動率預測管道並進行多模型競賽。

交付項目：

1. 對日經 225 和 S&P 500：
   - 對使用 Student's t 分配的 GARCH(1,1) 和 GJR-GARCH(1,1) 實作滾動視窗一步 ahead 預測
   - 使用 1000 個交易日的滾動視窗
   - 計算每個模型的 RMSE、MAE 和 QLIKE
2. 對每個模型進行 Mincer–Zarnowitz 迴歸。報告 $\hat{a}$、$\hat{b}$、R^2 和 F 檢定 p 值
3. 為每個指數建立預測比較圖（如步驟 5）
4. 製作按每個損失函數排名的模型摘要表
5. 撰寫 200 字報告給 Kenji：VolTech 應部署哪個波動率模型進行每日風險監控？為什麼？

🏆 附加挑戰：擴展視窗測試

使用擴展視窗（起點固定在 2015 年 1 月，終點每天增長）重複預測練習，取代滾動視窗。比較結果：

- 模型排名有改變嗎？
- 哪種方法的 QLIKE 較低？
- 每種方法的優缺點是什麼？

提示：擴展視窗使用所有可用的歷史，因此參數估計更穩定。但如果數據生成過程隨時間改變，較舊的數據可能反而有害。

5.10 重點摘要

重點摘要

1. **樣本外才是真正的考驗**：樣本內擬合可能因過度擬合而產生誤導。務必在模型在估計期間從未見過的數據上進行評估。
2. **滾動視窗預測**：每天在固定長度的視窗上重新估計模型並預測向前一步。這是波動率預測評估的黃金標準。
3. **損失函數**：RMSE、MAE 和 QLIKE 衡量預測品質的不同面向。QLIKE 特別有價值，因為無論使用哪個波動率代理變數，它都能產生一致的模型排名。
4. **Mincer–Zarnowitz 迴歸**：檢定預測是否有效率（$a = 0, b = 1$）。R^2 指出模型解釋了實現波動率中多少變異。
5. **GJR-GARCH 在權益指數上打敗 GARCH**：在大多數權益市場的實證研究中，不對稱模型一致地改善了樣本外預測，確認槓桿效應包含真正的預測資訊。
6. **GARCH 仍具競爭力**：儘管 AI 的熱潮，GARCH 家族模型在短預測範圍內仍然驚人地難以打敗。它們應該是任何預測比較中的基準。

5.11 延伸閱讀

延伸閱讀

1. Hansen, P. R., & Lunde, A. (2005). A forecast comparison of volatility models: does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7), 873–889. [波動率模型競賽的基準]
2. Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256. [為何 QLIKE 對代理變數選擇穩健]
3. Mincer, J. A., & Zarnowitz, V. (1969). The evaluation of economic forecasts. In *Economic Forecasts and Expectations* (pp. 3–46). NBER. [原始 M–Z 迴歸]
4. Engle, R. F., & Patton, A. J. (2001). What good is a volatility model? *Quantitative Finance*, 1(2), 237–245.
5. Andersen, T. G., & Bollerslev, T. (1998). Answering the skeptics: yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4), 885–905.
6. Poon, S.-H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: a review. *Journal of Economic Literature*, 41(2), 478–539.
7. Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263. [用於比較預測的 Diebold–Mariano 檢定]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 3.
9. West, K. D. (1996). Asymptotic inference about predictive ability. *Econometrica*, 64(5), 1067–1084. [預測評估推論的理論基礎]

下 週 預 告

當團隊魚貫離開時，*Kenji* 留了下來。他看著 *Alex* 的表情混合著敬佩和不耐。「你的模型預測波動率已經夠好了。*GJR-GARCH* 是明顯的贏家。但我的董事會不是用條件變異數來思考的。他們用日圓思考。他們想知道：以 *99%* 的信心水準，我們明天最多能損失多少？」他停頓了一下。「你的模型能回答這個問題嗎？」*Alex* 看向 *Dr. Lin*，她緩緩點頭。「那叫做風險值，」她說。「而它正是我們目前為止建構的一切匯聚之處。」下週：我們進入**風險值（*Value-at-Risk*）**的世界——每家銀行每一天都在計算的那個數字。

Chapter 6

第 6 週：風險值

學習目標

1. 定義風險值（Value-at-Risk, VaR）並以信賴水準、持有期間和投資組合價值來詮釋
2. 使用三種方法計算 VaR：歷史模擬法、參數法（變異數-共變異數法）和蒙地卡羅模擬法
3. 解釋 VaR 的限制及為何條件風險值（CVaR / Expected Shortfall）能解決其主要弱點
4. 在 Python 中使用 GARCH 波動率預測實作所有三種 VaR 方法
5. 比較不同方法在平靜期和危機期間的 VaR 估計

6.1 本週故事：統治一切的那個數字

場景：VolTech Analytics——週一，上午 8:30

Alex 一早到就發現桌上放了一份厚厚的文件。封面寫著：**日本金融廳——機構投資人風險管理指引**。Kenji 的便利貼寫著：「第 47 頁。讀一讀。這就是我的董事會要的東西。」

第 *47* 頁說：「機構應就所有主要資產類別，於 *99%* 信賴水準下報告其每日風險值。」風險值到底是什麼？

風險值是風險管理中最重要的單一數字。它回答一個問題：在給定的持有期間內，以給定的信賴水準，我們預期的最大損失是多少？全世界每一家主要的銀行、退休基金和保險公司每天都在計算它。*Basel III* 要求它。日本金融廳要求它。現在 *Kenji* 的董事會也要求我們提供。

具體來說，我的董事會要的是：「以 *99%* 的信心水準，我們的投資組合在一個交易日內最多損失 *X* 十億日圓。」他們要一個可以向金融廳報告、在董事會上討論的單一數字。不是條件變異數，不是 *GARCH* 參數——就是一個數字。

這就是與我們迄今所建構的一切的連結：*VaR* 的品質完全取決於背後波動率模型的品質。更好的波動率預測意味著更好的 *VaR* 估計。五週的工作都匯聚在這一刻。

6.2 理解風險值

在深入數學之前，將 VaR 理解為回答一個非常具體的問題會有幫助：給定持有期間和信賴水準，我們應準備面對的最壞「正常」損失是多少？

風險值定義

在信賴水準 α（例如 99%）下的 VaR_α 定義為：

$$P(L_t > \text{VaR}_\alpha) = 1 - \alpha \tag{6.1}$$

等價地，對於價值為 V、報酬率為 r_t 的投資組合：

$$\text{VaR}_\alpha = -V \cdot q_{1-\alpha}(r_t) \tag{6.2}$$

其中 $q_{1-\alpha}(r_t)$ 為報酬率分配的 $(1-\alpha)$ 分位數（例如 $\alpha = 0.99$ 的第 1 百分位數）。這是一個負數，所以負號將 VaR 轉換為正的損失金額。

VaR 的三個關鍵要素

每一個 VaR 數字需要三個輸入：

- **信賴水準**（α）：通常為 95% 或 99%。更高的信賴水準意味著更大（更保守）的 VaR。Basel III 使用 99%；內部風險管理常用 95%。
- **持有期間**：預測多遠？交易台通常為 1 天，監管資本（Basel）為 10 天。更長的持有期間產生更大的 VaR。
- **報酬率分配**：這是建模工作發生的地方——也是不同 VaR 方法產生分歧之處。

所以如果 *1* 天 *99%* 的 *VaR* 是 *1,000* 萬美元，那表示明天虧損超過 *1,000* 萬美元的機率只有 *1%*？

正確。在 *100* 天中的 *99* 天，損失會少於 *1,000* 萬美元。但關鍵的但書是：*VaR* 不會告訴你那 *1%* 的情境有多糟糕。你可能虧損 *1,100* 萬美元或 *1* 億美元——*VaR* 無法區分這些結果。這是它的根本弱點，我們稍後會用期望損失來解決。

6.3 方法 1：歷史模擬法 VaR

三種 VaR 方法在一個根本選擇上有所不同：我們如何建模報酬率分配？最簡單的答案是完全跳過建模，讓歷史自己說話。

歷史模擬法

最簡單的 VaR 方法：直接使用實際的歷史報酬率分配。不需要分配假設。

程序：

1. 收集最近 W 個日報酬率（例如 $W = 500$ 個交易日）
2. 將報酬率從最差到最好排序
3. 信賴水準 α 下的 VaR 是第 $(1-\alpha) \times W$ 個最差報酬率
4. 對於 99% VaR 且 $W = 500$：第 5 個最差報酬率

優點：不需分配假設；如果歷史數據中存在厚尾則能捕捉；易於向非技術受眾解釋。

缺點：假設過去能代表未來；對變化的波動率體制適應緩慢；平靜的歷史視窗在危機前會產生過度樂觀的 VaR。

歷史模擬法就是「看後視鏡」的方法。在路況不變之前都很好用。*2020* 年 *3* 月，之前 *500* 天大多是平靜的，所以歷史模擬 *VaR* 在你最需要它的時候太過樂觀。

6.4 方法 2：參數法（變異數-共變異數法）VaR

參數法 VaR

假設報酬率服從已知分配，並使用 GARCH 模型的波動率預測：

常態分配假設：

$$\text{VaR}_\alpha = -(\mu_t + z_{1-\alpha} \cdot \sigma_t) \tag{6.3}$$

其中 $z_{1-\alpha}$ 為 $(1-\alpha)$ 標準常態分位數（例如 99% VaR 的 $z_{0.01} = -2.326$），σ_t 為 GARCH 模型的條件標準差。

Student's t 分配假設：

$$\text{VaR}_\alpha = -(\mu_t + t_{1-\alpha,\nu} \cdot s_t) \tag{6.4}$$

其中 $t_{1-\alpha,\nu}$ 為自由度 ν 的 Student's t 分配的 $(1-\alpha)$ 分位數，$s_t = \sigma_t\sqrt{(\nu-2)/\nu}$ 重新縮放至單位變異數。

這就是我們所有 *GARCH* 工作回報的地方。關鍵洞見：σ_t 根據 *GARCH* 模型的條件變異數預測每天變化。所以我們的 *VaR* 動態地適應市場條件——波動率飆升時上升，市場平靜時下降。

為什麼我們會使用 t 分配而非常態分配？

因為真實報酬率的尾部比常態分配更厚——我們在第 *1* 週就證明了這一點。使用常態分位數 $z_{0.01} = -2.326$ 會低估極端損失。自由度 $\nu = 7$ 的 t 分配給出 $t_{0.01,7} \approx -2.998$。經過 $\sqrt{(\nu-2)/\nu}$ 的單位變異數標準化後，有效分位數約為 -2.53——比常態的 -2.326 更深入尾部約 *9%*。這產生更保守且更準確的 *VaR*。

常態 vs. t：尾部風險差距

對於 $\sigma = 1\%$ 日波動率的 99% VaR，使用縮放分位數 $q \cdot \sqrt{(\nu-2)/\nu}$：

- **常態**：VaR $\approx 2.33\%$——基準
- t（$\nu = 7$）：VaR $\approx 2.53\%$——比常態保守約 9%
- t（$\nu = 5$）：VaR $\approx 2.61\%$——保守約 12%，捕捉更厚的尾部

「正確」的自由度 ν 來自 GARCH 模型估計——我們不是任意選擇的。

6.5 方法 3：蒙地卡羅 VaR

如果歷史模擬法回顧過去、參數法 VaR 假設分配，蒙地卡羅走第三條路：模擬數千種可能的未來，讓模擬出的分配揭示風險。

蒙地卡羅模擬 VaR

從擬合的 GARCH 模型模擬數千條可能的未來報酬率路徑，然後從模擬分配中提取 VaR：

程序：

1. 擬合 GARCH 模型並獲得條件參數
2. 模擬 N（例如 10,000）個一步 ahead 報酬率，從假設的分配中抽取隨機創新
3. 每個模擬報酬率代表一個可能的「明天」
4. VaR 為模擬報酬率分配的 $(1-\alpha)$ 分位數

優點：可處理任何分配、路徑相依的支付和複雜的投資組合結構。衍生性商品帳簿的黃金標準。

缺點：計算成本高；結果在不同模擬執行之間有差異（需要大的 N）；如果模型設定錯誤則「垃圾進，垃圾出」。

蒙地卡羅是 *VaR* 的瑞士刀。它比參數法 *VaR* 更靈活，因為你可以從任何模型模擬，不限於常態或 t。對於單一資產的日 *VaR*，參數法通常就足夠了。但對於包含選擇權的多資產投資組合，蒙地卡羅是不可或缺的。

6.6 超越 VaR：期望損失

這個限制有一個正式的名稱和正式的解決方案。期望損失（Expected Shortfall）的概念回答了 VaR 無法回答的問題：當事情出錯時，到底有多糟？

VaR 的根本弱點

VaR 有一個致命的盲點：它告訴你最壞 $(1-\alpha)\%$ 結果的門檻，但不告訴你門檻之外的嚴重程度。一個 99% VaR 為 1,000 萬美元的投資組合，在那 1% 的尾部中平均損失可能是 1,100 萬或 5,000 萬美元——VaR 無法區分這些非常不同的風險特徵。

√x 條件風險值 / 期望損失

期望損失（Expected Shortfall, ES），又稱 **CVaR** 或**條件風險值**，衡量在損失超過 VaR 的條件下的平均損失：

$$\mathrm{ES}_\alpha = -E[r_t \mid r_t < -\mathrm{VaR}_\alpha] \tag{6.5}$$

ES 總是大於 VaR（更保守）。在 Basel III 交易帳簿基本審查（FRTB）下，97.5% 信賴水準的 ES 已取代 VaR 成為市場風險資本的主要監管風險衡量。
關鍵特性：ES 是一個**一致性風險衡量（Coherent Risk Measure）**——它滿足次可加性（分散化總是降低風險），而 VaR 則否。

等等——*VaR* 不滿足次可加性？這聽起來像是一個嚴重的缺陷。

確實如此。這裡有一個簡單的例子：兩個獨立的投資組合各有 *99% VaR* 為 *500* 萬美元。如果你將它們合併並分散化，合併後的 *VaR* 理論上可能高於 *1,000* 萬美元——違反了分散化降低風險的基本原則。這是一個數學人為現象，不是金融現實，但它使 *VaR* 在投資組合匯總方面不可靠。*ES* 沒有這個問題。

所以我們應該從 *VaR* 轉向期望損失？

產業正朝這個方向前進。*Basel III* 的 *FRTB* 已經強制要求 *ES* 用於市場風險資本。但 *VaR* 不會消失——它仍然與 *ES* 一起報告，並廣泛用於內部限額和溝通。今天我們兩者都會實作。

6.7 Python 實作：三種 VaR 方法

理論到位後，我們現在轉向實作。三種方法各需不同的程式碼結構，但都共享相同的基礎：前幾週擬合的 GJR-GARCH 模型。

讓我們建構所有三種 *VaR* 方法加上期望損失。我們將使用第 *5* 週的 *GJR-GARCH* 模型作為波動率引擎。

步驟 1：數據和模型設定

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from scipy import stats
import warnings
warnings.filterwarnings("ignore")

# Download S&P 500 data
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Fit GJR-GARCH(1,1) with t-distribution
model = arch_model(returns, vol="Garch", p=1, o=1,
                   q=1, dist="t")
res = model.fit(disp="off")
print(res.summary().tables[1])

# Extract fitted parameters
nu = res.params["nu"]  # degrees of freedom
print(f"\nDegrees of freedom (nu): {nu:.2f}")
```

步驟 2：三種 VaR 方法

```python
# Confidence levels
alphas = [0.95, 0.99]

# --- Method 1: Historical Simulation ---
def var_historical(returns, alpha, window=500):
    """Rolling Historical Simulation VaR."""
    var_hs = pd.Series(index=returns.index, dtype=float)
    for i in range(window, len(returns)):
        hist = returns.iloc[i-window:i]
        var_hs.iloc[i] = -np.percentile(
```

```
            hist, (1 - alpha) * 100)
    return var_hs

# --- Method 2: Parametric (GARCH-based) ---
def var_parametric(res, alpha, dist="t"):
    """GARCH-based parametric VaR."""
    cond_vol = res.conditional_volatility
    mu = res.params.get("mu", 0)
    nu = res.params.get("nu", 30)

    if dist == "t":
        q = stats.t.ppf(1 - alpha, df=nu)
        scale = np.sqrt((nu - 2) / nu)
        var_p = -(mu + q * scale * cond_vol)
    else:  # normal
        q = stats.norm.ppf(1 - alpha)
        var_p = -(mu + q * cond_vol)
    return var_p

# --- Method 3: Monte Carlo ---
def var_monte_carlo(res, alpha, n_sims=10000,
                    seed=42):
    """GARCH-based Monte Carlo VaR."""
    np.random.seed(seed)
    sim = res.forecast(horizon=1, method="simulation",
                       simulations=n_sims)
    sim_returns = sim.simulations.values[-1, :, 0]
    var_mc = -np.percentile(sim_returns,
                            (1 - alpha) * 100)
    return var_mc, sim_returns

# Compute all VaR estimates
var_hs_99 = var_historical(returns, 0.99)
var_p_99_t = var_parametric(res, 0.99, "t")
var_p_99_n = var_parametric(res, 0.99, "normal")
var_mc_99, sim_rets = var_monte_carlo(res, 0.99)

print("VaR Estimates (last day):")
```

```python
print(f"  Historical Sim (99%): "
      f"{var_hs_99.iloc[-1]:.4f}%")
print(f"  Parametric-t   (99%): "
      f"{var_p_99_t.iloc[-1]:.4f}%")
print(f"  Parametric-N   (99%): "
      f"{var_p_99_n.iloc[-1]:.4f}%")
print(f"  Monte Carlo    (99%): {var_mc_99:.4f}%")
```

參數法 t *VaR* 明顯大於常態 *VaR*。這就是厚尾調整的作用？

完全正確。t 分配在尾部放置了更多的機率質量，所以 *1%* 分位數更遠。對於每日風險管理，那額外的保守性可能是資本儲備充足與不足之間的差別。

步驟 3：期望損失

```python
# Expected Shortfall (CVaR)
def es_historical(returns, alpha, window=500):
    """Rolling Historical Simulation ES."""
    es = pd.Series(index=returns.index, dtype=float)
    for i in range(window, len(returns)):
        hist = returns.iloc[i-window:i].values
        var_level = np.percentile(
            hist, (1 - alpha) * 100)
        tail = hist[hist <= var_level]
        es.iloc[i] = -tail.mean() if len(tail) > 0 \
            else np.nan
    return es

def es_parametric(res, alpha, dist="t"):
    """GARCH-based parametric ES."""
    cond_vol = res.conditional_volatility
    mu = res.params.get("mu", 0)
    nu = res.params.get("nu", 30)

    if dist == "t":
        q = stats.t.ppf(1 - alpha, df=nu)
        pdf_q = stats.t.pdf(q, df=nu)
```

```
        scale = np.sqrt((nu - 2) / nu)
        # ES formula for Student's t
        es_factor = -(pdf_q / (1 - alpha)) * \
            ((nu + q**2) / (nu - 1))
        es_p = -(mu + es_factor * scale * cond_vol)
    else:
        q = stats.norm.ppf(1 - alpha)
        pdf_q = stats.norm.pdf(q)
        es_factor = -pdf_q / (1 - alpha)
        es_p = -(mu + es_factor * cond_vol)
    return es_p

# Compute ES
es_hs_99 = es_historical(returns, 0.99)
es_p_99 = es_parametric(res, 0.99, "t")

print("\nExpected Shortfall (last day):")
print(f"  Historical Sim (99%): "
      f"{es_hs_99.iloc[-1]:.4f}%")
print(f"  Parametric-t   (99%): "
      f"{es_p_99.iloc[-1]:.4f}%")
print(f"\nES / VaR ratio: "
      f"{es_p_99.iloc[-1]/var_p_99_t.iloc[-1]:.2f}")
```

ES/VaR 比率

對於常態分配，99% ES 約為 99% VaR 的 1.15 倍。對於厚尾分配（低自由度的 t），比率更高——通常為 1.2–1.4。高 ES/VaR 比率表示 VaR 之外的尾部「沉重」——在 VaR 突破的條件下，預期損失顯著大於 VaR 本身。這個比率是尾部嚴重程度的快速診斷。

現在我們將 VaR 估計隨時間的變化視覺化，聚焦於 COVID-19 危機：

步驟 4：VaR 視覺化

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10),
                         sharex=True)

# Panel 1: Returns vs. VaR
ax1 = axes[0]
```

```
ax1.plot(returns.index, returns, color="gray",
         linewidth=0.4, alpha=0.5, label="Returns")
ax1.plot(var_hs_99.index, -var_hs_99, color="#D4A843",
         linewidth=1, label="HS VaR (99%)")
ax1.plot(var_p_99_t.index, -var_p_99_t,
         color="#C0392B", linewidth=1,
         label="Parametric-t VaR (99%)")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Return (%)")
ax1.set_title("Daily Returns vs. 99% VaR Estimates")
ax1.legend(loc="lower left", fontsize=8)
ax1.set_ylim(-15, 12)

# Panel 2: VaR comparison (positive scale)
ax2 = axes[1]
ax2.plot(var_hs_99.index, var_hs_99, color="#D4A843",
         linewidth=1, label="Historical Sim")
ax2.plot(var_p_99_t.index, var_p_99_t, color="#C0392B",
         linewidth=1, label="Parametric-t (GJR)")
ax2.plot(es_p_99.index, es_p_99, color="#1B3A5C",
         linewidth=1, alpha=0.7,
         label="Expected Shortfall (t)")
ax2.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red",
            label="COVID-19")
ax2.set_ylabel("Risk Measure (%)")
ax2.set_title("VaR and ES Over Time")
ax2.legend(loc="upper right", fontsize=8)
ax2.set_ylim(0, 15)

plt.tight_layout()
plt.savefig("week06_var_comparison.png", dpi=150)
plt.show()
```

6.8 應用：COVID-19 崩盤期間的 VaR

場景：會議室——週四，下午 2:00

Alex 向團隊展示了 VaR 比較結果。螢幕上的 COVID 期間講述了一個戲劇性的故事。

以下是我們的發現。在 *2020* 年 *3* 月的 *COVID-19* 崩盤期間，三種 *VaR* 方法的表現截然不同：

1. **歷史模擬法反應危險地緩慢**。它仍然基於平靜的 *2019* 年數據，因此嚴重低估了風險。一個月內發生了 *10* 天以上的 *VaR* 突破——遠超預期的每年 *2–3* 次。
2. **參數法 *VaR*（*GJR-GARCH*）快速適應**。因為 *GARCH* 模型每天更新其條件變異數，*VaR* 估計在第一次大幅下跌後數天內就飆升了。它仍有一些初期突破，但適應速度遠快於歷史模擬法。
3. ***t* 分配很重要**。使用常態分配的參數法 *VaR* 比 *t* 版本多了 *30–40%* 的突破。厚尾不只是理論上的細微之處——它們有實際的資本影響。

這正是**條件*VaR***——隨最新波動率預測更新的 *VaR*——的論據。歷史模擬法是無條件的：它使用固定視窗的過去報酬率，不管當前市場壓力。在波動率呈趨勢的體制中，那是災難的配方。

所以你們建議使用參數法搭配我們的 *GJR-GARCH* 模型？

是的，具體來說是使用 *Student's t* 分配的 *GJR-GARCH*。它有三個優勢：

1. 透過 *GARCH* 機制適應變化的波動率
2. 透過 *t* 分配考慮厚尾
3. 捕捉槓桿效應——負報酬率比同等幅度的正報酬率更能增加風險

對於金融廳報告，我建議同時報告 *99%* 水準的 *VaR* 和期望損失。

很好。現在是困難的問題：我們怎麼知道我們的 *VaR* 模型確實可靠？金融廳不會只聽我們的話——他們要證據證明我們的模型有效。

那叫做**回溯測試（*Backtesting*）**——而它正是我們下週要做的。

6.9 AI 焦點：神經網路 VaR

我應該提一下機器學習在這一切中的位置。*GARCH* 基礎的 *VaR* 是產業標準，但量化研究界正在實驗完全跳過分配假設的方法——讓數據直接定義分位數。以下是那些替代方法的樣貌。

VaR 估計的 AI 方法

傳統 VaR 依賴參數模型（GARCH + 分配假設）或歷史數據。AI 提供了替代方法：

- **分量迴歸**：不建模完整的報酬率分配，而是直接預測 α 分位數。線性分量迴歸很簡單；神經網路分量迴歸增加靈活性。
- **CAViaR**（Engle & Manganelli, 2004）：條件自迴歸風險值。直接將 VaR 動態建模為自迴歸過程，無需指定完整分配。傳統方法與 ML 方法之間的橋樑。
- **深度學習 VaR**：LSTM 或 Transformer 網路直接從特徵（過去報酬率、成交量、VIX、總體指標）訓練預測 VaR 分位數。可捕捉複雜的非線性模式，但需要仔細的正則化。
- **集成方法**：結合多個模型（GARCH、HS、ML）的 VaR 以產生更穩健的估計。簡單平均常優於個別模型。

實務要點：基於 GARCH 的參數法 VaR 因其可解釋性和監管接受度仍為產業標準。AI 方法在可解釋性要求較低的自營交易台和對沖基金中漸獲採用。

6.10 討論：VaR 論辯

VaR 是好的風險衡量嗎？

1. **Taleb 的批評**：Nassim Taleb 著名地稱 VaR 為「江湖術」，因為它忽略了信賴水準以外發生的事（Taleb, 2007）。這個批評公平嗎？期望損失能完全解決嗎？
2. **信賴水準的選擇**：為什麼退休基金可能偏好 99% VaR 而對沖基金使用 95%？保守性和資本效率之間的取捨是什麼？
3. **順景氣循環問題**：危機期間，VaR 上升 $\rightarrow$ 公司賣出資產以降低風險 $\rightarrow$ 價格進一步下跌 $\rightarrow$ VaR 更進一步上升。監管機構如何解決這個回饋循環？
4. **模型風險**：我們用三種不同的方式計算 VaR，得到三個不同的數字。哪一個是「對的」？風險管理者應如何處理模型分歧？

6.11 每週任務

◎ 任務 6：退休基金的 VaR

目標：使用多種方法計算和比較日經 225 和 S&P 500 的風險值估計。

交付項目：

1. 對每個指數，使用以下方法計算每日 99% VaR：
   - 歷史模擬法（500 天視窗）
   - 使用 GJR-GARCH 和常態分配的參數法 VaR
   - 使用 GJR-GARCH 和 t 分配的參數法 VaR
2. 為參數法 t 方法計算 99% 期望損失
3. 計算 2020 年樣本外年度中每種方法的 VaR 突破次數（實際損失超過 VaR 的天數）。報告突破率百分比
4. 建立比較圖，顯示報酬率及三條 VaR 線疊加（如步驟 4）
5. 撰寫 200 字備忘錄給 Kenji 的董事會：推薦一種 VaR 方法用於每日報告並解釋原因。包含最近日期的 VaR 和 ES 數據

🏆 附加挑戰：投資組合 VaR

將分析擴展到雙資產投資組合：60% S&P 500 和 40% 日經 225。

- 計算投資組合報酬率序列
- 計算投資組合的歷史模擬 VaR
- 與個別 VaR 之和比較——投資組合 VaR 較低（分散化效益）還是較高？
- 日圓/美元匯率對結果有多大影響？

6.12 重點摘要

重點摘要

1. **VaR 回答一個問題**：在給定信賴水準和持有期間下，預期的最大損失是多少？它需要三個輸入：信賴水準、持有期間和報酬率分配。
2. **三種方法**：歷史模擬法（簡單但適應慢）、參數法（GARCH 基礎，動態更新）和蒙地卡羅法（靈活但計算成本高）。每種方法在簡潔性和準確度之間有取捨。
3. **GARCH 基礎的參數法 VaR 適應最快**：因為它每天更新條件變異數，對體制變化的反應遠快於歷史模擬法。
4. **厚尾很重要**：使用 t 分配而非常態分配產生更保守且更準確的 VaR。改善在危機期間最為明顯。
5. **VaR 有盲點**：它不告訴你信賴門檻之外的損失。期望損失（CVaR）衡量尾部的平均損失，使其成為更具資訊量的風險衡量。
6. **ES 是新的監管標準**：Basel III 的 FRTB 以 97.5% 的期望損失取代 VaR 作為市場風險資本。兩種衡量應一起報告。

6.13 延伸閱讀

延伸閱讀

1. Jorion, P. (2007). *Value at Risk: The New Benchmark for Managing Financial Risk*, 3rd ed. McGraw-Hill. [VaR 的權威教科書]
2. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools*, 2nd ed. Princeton University Press. Chapters 2 and 9. [VaR 和 ES 的嚴謹處理]
3. Artzner, P., Delbaen, F., Eber, J.-M., & Heath, D. (1999). Coherent measures of risk. *Mathematical Finance*, 9(3), 203–228. [為何 VaR 不是一致性的而 ES 是]
4. Engle, R. F., & Manganelli, S. (2004). CAViaR: Conditional autoregressive value at risk by regression quantiles. *Journal of Business & Economic Statistics*, 22(4), 367–381.
5. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapters 12–15.
6. Basel Committee on Banking Supervision (2019). Minimum capital requirements for market risk. [FRTB 標準]
7. Kuester, K., Mittnik, S., & Paolella, M. S. (2006). Value-at-Risk prediction: a comparison of alternative strategies. *Journal of Financial Econometrics*, 4(1), 53–89. [全面的 VaR 方法比較]
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 7.
9. Taleb, N. N. (2007). *The Black Swan*. Random House. [對 VaR 和金融分配假設的原始批評]
10. JP Morgan/Reuters (1996). *RiskMetrics Technical Document*, 4th ed. [推廣銀行每日 VaR 的原始 VaR 框架]

下 週 預 告

第二天早上，*Alex* 發現了 *Kenji* 的一封緊急電子郵件。附件是日本金融廳的正式信函：「敬啟者 *VolTech Analytics*，作為日本生命退休基金風險管理框架年度審查的一部分，請提供貴司 *VaR* 模型的全面回溯測試報告。報告必須包括：*(1)* 歷史 *VaR* 突破分析，*(2) Kupiec* 無條件覆蓋檢定，*(3) Christoffersen* 條件覆蓋檢定，*(4)* 至少三個極端情境下的壓力測試結果。」*Kenji* 在底部附了一張便條：「他們兩週內要。沒有壓力。」*Alex* 看向 *Dr. Lin*。她微笑著。「歡迎來到監管合規的世界。這是我們證明模型不只是漂亮圖表——它確實有效的地方。」下週：我們用**回溯測試和壓力測試**對 *VaR* 模型進行審判。

Chapter 7

第 7 週：VaR 回測與風險管理

學習目標

1. 定義 VaR 回測（Backtesting）並解釋為何監管機構要求進行回測
2. 實作 Kupiec（POF）無條件覆蓋率檢定以驗證 VaR 模型
3. 實作 Christoffersen 條件覆蓋率檢定以偵測 VaR 違反的群聚現象
4. 解讀 Basel 交通號誌系統對 VaR 模型的評估方式
5. 設計並執行歷史情境與假設情境的壓力測試

7.1 本週故事：監管機構來電

場景：VolTech Analytics — 週一，早上 7:45

Alex 從來沒有看過 Kenji 如此緊張的表情。日本金融廳（FSA）發來的電子郵件顯示在會議室的螢幕上，正式的措辭劃破了清晨的寧靜。Dr. Lin 已經印好相關的 Basel III 準則，攤在桌上。

FSA 要求我們在兩週內提交完整的回測報告。他們特別指定要 *Kupiec* 檢定、*Christoffersen* 檢定，以及極端情境下的壓力測試。如果我們未能通過這次審查，退休基金可能被要求持有額外的資本——這意味著可用於投資報酬的資金將會減少。

不要慌。這是標準的監管實務。每一家主要金融機構都會經歷這個過程。關鍵問題很簡單：我們的 *VaR* 模型是否如其宣稱的那樣運作？如果我們說 *99% VaR* 只會在 *1%* 的時間被突破，我們就需要證明這一點。

我們要怎麼「證明」一個機率模型是有效的？

透過檢視歷史紀錄。如果你的 *99% VaR* 在過去一年中恰好大約在 *1%* 的交易日被突破，那就是模型校準良好的證據。如果被突破的次數太多，模型就是低估了風險。如果從未被突破，模型可能過於保守——這會浪費資本。

7.2 回測的邏輯

什麼是 VaR 回測（Backtesting）？

回測（Backtesting）將模型的 VaR 預測值與實際已實現的損失進行比較。定義**命中序列**（Hit Sequence）：

$$I_t = \begin{cases} 1 & \text{if } r_t < -\text{VaR}_t \quad \text{（違反）} \\ 0 & \text{otherwise} \end{cases}$$

在正確的模型下，命中序列應滿足兩個性質：

1. **無條件覆蓋率**（Unconditional Coverage）：違反的比例應等於 $1-\alpha$。對於 99% VaR，預期有 1% 的違反。
2. **獨立性**（Independence）：違反不應群聚。今天的違反不應預測明天的違反。群聚現象表示模型的適應速度太慢。

良好的回測框架會同時檢驗兩個性質。

為何兩個性質都很重要

考慮 250 個交易日中 99% VaR 的兩個情境：

- **模型 A**：3 次違反，分散在一月、六月和十一月。違反率 = 1.2% — 接近預期的 1%。通過無條件覆蓋率檢定。
- **模型 B**：3 次違反，全部集中在市場崩盤的同一週。違反率相同（1.2%），但群聚現象揭示模型未能適應不斷上升的波動率。

模型 A 校準良好；模型 B 有嚴重的獨立性問題。Kupiec 檢定只有在總次數錯誤時才能捕捉到模型 B 的問題。Christoffersen 檢定即使在總次數正確的情況下也能捕捉模型 B 的群聚現象。

7.3 檢定一：Kupiec 無條件覆蓋率

√× Kupiec POF 檢定（1995）

令 T 為樣本外天數的總數，x 為 VaR 違反的次數。預期違反機率為 $p = 1 - \alpha$。**失敗比例**（Proportion of Failures, POF）概似比檢定：

$$LR_{uc} = -2 \ln \frac{p^x (1-p)^{T-x}}{\hat{p}^x (1-\hat{p})^{T-x}} \tag{7.1}$$

其中 $\hat{p} = x/T$ 為觀察到的違反率。在 H_0：$\hat{p} = p$ 之下，統計量 $LR_{uc} \sim \chi^2(1)$。

解讀：

- 若 $LR_{uc} > \chi^2_{1,0.05} = 3.841$ （或 p-value < 0.05），則拒絕 H_0
- 拒絕表示違反率與預期值有顯著差異 — 模型校準不當
- 雙尾檢定：無論違反太多或太少都會拒絕

所以如果我們有 *250* 天且使用 *99% VaR*，預期會有 *2.5* 次違反。什麼範圍可以通過檢定？

在 $T = 250$ 且 *5%* 顯著水準下，大約 *0* 到 *6* 次違反可以通過 *Kupiec* 檢定。*7* 次以上，模型就有麻煩了。但請記住：*Kupiec* 檢定在小樣本下的檢定力很低。只有 *250* 天的數據，很難區分 *1%* 和 *2%* 的違反率。這就是為什麼監管機構通常要求至少一年的數據。

7.4 檢定二：Christoffersen 條件覆蓋率

√× Christoffersen 檢定（1998）

此檢定透過同時檢驗違反的**獨立性**來延伸 Kupiec 檢定。定義轉移機率：

$$\pi_{ij} = P(I_t = j \mid I_{t-1} = i) \quad \text{for } i, j \in \{0, 1\}$$

獨立性檢定統計量：

$$LR_{ind} = -2 \ln \frac{\hat{\pi}^{n_{01}+n_{11}} (1-\hat{\pi})^{n_{00}+n_{10}}}{\hat{\pi}_{01}^{n_{01}} (1-\hat{\pi}_{01})^{n_{00}} \hat{\pi}_{11}^{n_{11}} (1-\hat{\pi}_{11})^{n_{10}}} \tag{7.2}$$

其中 n_{ij} 計算從狀態 i 到狀態 j 的轉移次數，$\hat{\pi} = (n_{01} + n_{11})/(T-1)$，且 $\hat{\pi}_{ij} = n_{ij}/(n_{i0} + n_{i1})$。

條件覆蓋率（Conditional Coverage）檢定結合兩者：

$$LR_{cc} = LR_{uc} + LR_{ind} \sim \chi^2(2) \tag{7.3}$$

若 $LR_{cc} > \chi^2_{2,0.05} = 5.991$，則拒絕 H_0。

從數據的角度來看，*Kupiec* 是一個單樣本比例檢定 — 你只是在比較觀察到的違反率與 *1%*。*Christoffersen* 在此之上加入了 *Markov* 轉移模型：如果違反指標具有序列相關性，那麼模型的殘差就是自相關的，任何像樣的滾動視窗估計方法都應該能捕捉到這個問題。群聚現象意味著條件分配的設定有誤，而不僅僅是無條件分配。

7.5 Basel 交通號誌系統

Basel 交通號誌框架

Basel 委員會使用一個簡單的分類系統，基於 99% VaR 模型在 250 個交易日內的 VaR 違反次數：

區域	違反次數	解讀	資本乘數
綠燈	0–4	模型可接受	3.0×
黃燈	5–9	模型有疑慮	3.4–3.85×
紅燈	≥ 10	模型被否決	4.0×

紅燈區分類意味著銀行必須將其 VaR 乘以 4.0（而非 3.0）來計算法定資本 — 這是一項重大的財務懲罰。銀行有強烈的動機維持在綠燈區。

所以如果我們的模型落在紅燈區，退休基金就必須多持有 *33%* 的資本？那可是本來可以用於投資的錢。

正是如此。*VaR* 模型的品質有直接的財務影響。過於激進的模型在好的時候可以節省資本，但面臨紅燈區懲罰和監管審查的風險。最佳平衡點是一個既準確又略

為保守的模型。

7.6 壓力測試

壓力測試框架

回測評估模型在正常條件下的表現，而**壓力測試**（Stress Testing）則檢驗模型在極端情境下的表現。兩種主要方法：

1. 歷史壓力測試：重播實際的危機時期：

- 2008 年全球金融危機（GFC）
- 2020 年 COVID-19 崩盤
- 2011 年歐洲債務危機
- 1997 年亞洲金融危機

2. 假設壓力測試：設計尚未發生但合理的情境：

- 單日市場突然下跌 5%
- VIX 一夜之間從 15 飆升至 80
- 所有亞洲市場同時崩盤
- 利率突然上升 300 個基點

對每個情境計算投資組合損失，並與 VaR 估計值進行比較。如果損失超過 VaR，量化超出的幅度。

壓力測試是理論與董事會現實交匯之處。我們的投資人不在乎卡方統計量。他們想知道的是：「如果 *2008* 年再次發生會怎樣？如果發生更糟的事情呢？」壓力測試以具體的金額給他們明確的答案。

7.7 Python 實作：回測套件

讓我們建立一套完整的回測與壓力測試流程。我們將測試第六週的 *GJR-GARCH VaR* 模型。

步驟一：VaR 回測設定

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from scipy import stats
import warnings
warnings.filterwarnings("ignore")

# Download data and compute returns
sp500 = yf.download("^GSPC", start="2015-01-01",
                     end="2024-12-31")
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Rolling VaR backtest
window = 1000
alpha = 0.99
n_total = len(returns)
n_oos = n_total - window

# Storage
var_bt = np.full(n_oos, np.nan)
es_bt = np.full(n_oos, np.nan)

print(f"Backtest period: {n_oos} days")
print(f"Expected breaches (99%): {n_oos * 0.01:.1f}")
```

數據載入且儲存陣列初始化完畢後，我們可以執行滾動視窗估計。每次迭代都會在 1,000 個交易日的歷史數據上重新配適 GJR-GARCH 模型，並擷取一步前瞻預測 — 這是整個流程的計算核心。

步驟二：滾動 VaR 與 ES 計算

```python
# Rolling GJR-GARCH VaR (t-distribution)
# Note: this takes ~5-10 minutes
for i in range(n_oos):
    train = returns.iloc[i:i + window]
```

```
    try:
        model = arch_model(train, vol="Garch",
                           p=1, o=1, q=1, dist="t")
        res = model.fit(disp="off",
                        show_warning=False)
        # One-step-ahead forecast
        fv = res.forecast(horizon=1)
        cond_var = fv.variance.values[-1, 0]
        cond_vol = np.sqrt(cond_var)
        mu = res.params.get("mu", 0)
        nu = res.params["nu"]

        # Parametric VaR (t-distribution)
        q = stats.t.ppf(1 - alpha, df=nu)
        scale = np.sqrt((nu - 2) / nu)
        var_bt[i] = -(mu + q * scale * cond_vol)

        # Parametric ES (t-distribution)
        pdf_q = stats.t.pdf(q, df=nu)
        es_factor = -(pdf_q / (1 - alpha)) * \
            ((nu + q**2) / (nu - 1))
        es_bt[i] = -(mu + es_factor * scale * cond_vol)
    except Exception:
        var_bt[i] = np.nan
        es_bt[i] = np.nan

var_series = pd.Series(var_bt,
    index=returns.index[window:])
es_series = pd.Series(es_bt,
    index=returns.index[window:])
oos_returns = returns.iloc[window:]

# Identify breaches
breaches = oos_returns < -var_series
n_breaches = breaches.sum()
breach_rate = n_breaches / len(oos_returns)
print(f"Breaches: {n_breaches}")
print(f"Breach rate: {breach_rate:.4f} "
```

```
        f"(expected: {1-alpha:.4f})")
```

建構出違反序列後，我們套用前面討論的兩個正式統計檢定。下列函數回傳結構化的字典，方便並排比較結果。

步驟三：Kupiec 與 Christoffersen 檢定

```python
def kupiec_test(breaches, alpha=0.99):
    """Kupiec POF test for unconditional coverage."""
    hits = breaches.astype(int).values
    T = len(hits)
    x = hits.sum()
    p = 1 - alpha  # expected breach probability

    if x == 0 or x == T:
        return {"LR_uc": np.inf, "p_value": 0.0,
                "breaches": x, "expected": T * p}

    p_hat = x / T
    lr = -2 * (x * np.log(p / p_hat) +
               (T - x) * np.log((1-p) / (1-p_hat)))
    p_val = 1 - stats.chi2.cdf(lr, df=1)
    return {"LR_uc": lr, "p_value": p_val,
            "breaches": int(x),
            "expected": T * p,
            "breach_rate": p_hat}

def christoffersen_test(breaches, alpha=0.99):
    """Christoffersen conditional coverage test."""
    hits = breaches.astype(int).values
    T = len(hits)

    # Count transitions
    n00, n01, n10, n11 = 0, 0, 0, 0
    for t in range(1, T):
        if hits[t-1] == 0 and hits[t] == 0: n00 += 1
        elif hits[t-1] == 0 and hits[t] == 1: n01 += 1
        elif hits[t-1] == 1 and hits[t] == 0: n10 += 1
        else: n11 += 1
```

```python

    # Unconditional coverage
    kup = kupiec_test(breaches, alpha)
    lr_uc = kup["LR_uc"]

    # Independence test
    pi = (n01 + n11) / max(T - 1, 1)
    pi01 = n01 / max(n00 + n01, 1)
    pi11 = n11 / max(n10 + n11, 1)

    # Compute in log-domain to avoid numerical underflow
    eps = 1e-10
    log_num = ((n00 + n10) * np.log(max(1 - pi, eps)) +
               (n01 + n11) * np.log(max(pi, eps)))
    log_den = (n00 * np.log(max(1 - pi01, eps)) +
               n01 * np.log(max(pi01, eps)) +
               n10 * np.log(max(1 - pi11, eps)) +
               n11 * np.log(max(pi11, eps)))
    lr_ind = -2 * (log_num - log_den)

    lr_cc = lr_uc + lr_ind
    p_val = 1 - stats.chi2.cdf(lr_cc, df=2)

    return {"LR_uc": lr_uc, "LR_ind": lr_ind,
            "LR_cc": lr_cc, "p_value_cc": p_val,
            "n00": n00, "n01": n01,
            "n10": n10, "n11": n11}

# Run both tests
kup = kupiec_test(breaches)
cc = christoffersen_test(breaches)

print("=== Kupiec Test ===")
print(f"Breaches: {kup['breaches']} / "
      f"{len(oos_returns)}")
print(f"Expected: {kup['expected']:.1f}")
print(f"Breach rate: {kup['breach_rate']:.4f}")
print(f"LR_uc = {kup['LR_uc']:.3f}, "
```

```
        f"p-value = {kup['p_value']:.4f}")
result = "PASS" if kup["p_value"] > 0.05 else "FAIL"
print(f"Result: {result}")

print("\n=== Christoffersen Test ===")
print(f"LR_uc = {cc['LR_uc']:.3f}")
print(f"LR_ind = {cc['LR_ind']:.3f}")
print(f"LR_cc = {cc['LR_cc']:.3f}, "
        f"p-value = {cc['p_value_cc']:.4f}")
result = "PASS" if cc["p_value_cc"] > 0.05 else "FAIL"
print(f"Result: {result}")
```

取得檢定統計量後，下一步是透過 Basel 交通號誌查詢表將違反次數對應到監管後果。

步驟四：Basel 交通號誌分類

```
def basel_traffic_light(breaches_250):
    """Basel traffic light for 250-day window."""
    n = int(breaches_250.sum())
    # Basel multiplier lookup table
    yellow_mult = {5: 3.40, 6: 3.50, 7: 3.65,
                   8: 3.75, 9: 3.85}
    if n <= 4:
        zone = "GREEN"
        mult = 3.0
    elif n <= 9:
        zone = "YELLOW"
        mult = yellow_mult[n]
    else:
        zone = "RED"
        mult = 4.0
    return zone, mult, n

# Evaluate last 250 trading days
last_250 = breaches.iloc[-250:]
zone, mult, n = basel_traffic_light(last_250)
print(f"\n=== Basel Traffic Light ===")
print(f"Breaches (last 250 days): {n}")
print(f"Zone: {zone}")
```

```python
print(f"Capital multiplier: {mult:.2f}x")
```

監管分類完成後，我們現在用一系列歷史與假設衝擊來對當前的 VaR 估計值進行壓力測試，以了解極端損失可以超出 VaR 多少。

步驟五：壓力測試

```python
# Historical stress scenarios
stress_scenarios = {
    "GFC 2008 (worst day)": -0.0947,
    "COVID 2020 (worst day)": -0.1277,
    "Flash Crash 2010": -0.0387,
    "Brexit 2016": -0.0348,
    "VIX spike 2018": -0.0431,
    "Hypothetical -5%": -0.05,
    "Hypothetical -10%": -0.10,
    "Hypothetical -15%": -0.15,
}

# Current VaR and ES (last day)
current_var = var_series.iloc[-1]
current_es = es_series.iloc[-1]
print(f"Current 99% VaR: {current_var:.4f}%")
print(f"Current 99% ES:  {current_es:.4f}%")
print(f"\n{'Scenario':<30} {'Loss':>8} "
      f"{'vs VaR':>8} {'vs ES':>8}")
print("-" * 56)
for name, loss_pct in stress_scenarios.items():
    loss = abs(loss_pct) * 100  # convert to %
    var_ratio = loss / current_var
    es_ratio = loss / current_es
    flag = " !!" if loss > current_var else ""
    print(f"{name:<30} {loss:>7.2f}% "
          f"{var_ratio:>7.2f}x {es_ratio:>7.2f}x"
          f"{flag}")
```

最後，我們將回測結果視覺化：

步驟六：回測視覺化

```
fig, axes = plt.subplots(2, 1, figsize=(14, 10),
                         sharex=True)

# Panel 1: Returns with VaR and breach markers
ax1 = axes[0]
ax1.plot(oos_returns.index, oos_returns, color="gray",
         linewidth=0.4, alpha=0.5, label="Returns")
ax1.plot(var_series.index, -var_series, color="#C0392B",
         linewidth=0.8, label="99% VaR (GJR-t)")
ax1.scatter(oos_returns[breaches].index,
            oos_returns[breaches], color="#C0392B",
            s=20, zorder=5, label="VaR breaches")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Return (%)")
ax1.set_title("VaR Backtest: GJR-GARCH(1,1) "
              "with t-distribution")
ax1.legend(loc="lower left", fontsize=8)
ax1.set_ylim(-15, 12)

# Panel 2: Cumulative breaches
ax2 = axes[1]
cum_breaches = breaches.cumsum()
expected_line = pd.Series(
    np.arange(1, len(breaches) + 1) * 0.01,
    index=breaches.index)
ax2.plot(cum_breaches.index, cum_breaches,
         color="#C0392B", linewidth=1.5,
         label="Actual breaches")
ax2.plot(expected_line.index, expected_line,
         color="#1B3A5C", linewidth=1,
         linestyle="--", label="Expected (1%)")
ax2.set_ylabel("Cumulative Breaches")
ax2.set_title("Cumulative VaR Breaches vs. Expected")
ax2.legend(loc="upper left")

plt.tight_layout()
```

```
plt.savefig("week07_backtest.png", dpi=150)
plt.show()
```

7.8 應用：FSA 報告

場景：VolTech Analytics — 週五，下午 4:30

團隊在將完整的回測報告送交 FSA 之前進行了最後審閱。Alex 在白板上摘要了研究發現。

以下是我們使用 *Student t* 分配之 *GJR-GARCH VaR* 模型的回測摘要：

1. **違反分析**：在完整的樣本外期間，我們的 *99% VaR* 大約在 *1.0–1.5%* 的交易日被突破 — 接近預期的 *1%*。大多數違反發生在 *COVID-19* 的初始階段，此時模型尚未完全適應。
2. ***Kupiec* 檢定**：未拒絕 H_0 （$p > 0.05$）— 違反率與 *99%* 信賴水準一致。
3. ***Christoffersen* 檢定**：獨立性部分顯示 *COVID* 期間有輕微群聚，但整體條件覆蓋率檢定在 *5%* 水準下通過。
4. ***Basel* 分類**：綠燈區，最近 *250* 天有 *2–3* 次違反。資本乘數維持在 $3.0\times$。
5. **壓力測試**：我們的 *VaR* 會在 *GFC* 級別和 *COVID* 級別的衝擊下被超越，但 *ES* 提供了更為切合實際的尾部損失估計。

數據看起來很扎實。跟我基金之前使用的舊歷史模擬法模型相比如何？

歷史模擬法在任何包含體制轉換（*Regime Change*）的時期很可能會落在黃燈或紅燈區。*GJR-GARCH* 模型的主要優勢在於它的 適應性 — 它每天更新，因此能比危機衝擊更快地恢復。

出色的工作。*FSA* 報告已經準備就緒。*Kenji*，你的董事會應該會很滿意 — 我們展示了一個經過統計驗證、符合 *Basel* 規範、且經過壓力測試的模型。但我一直在想我們給你基金的下一個提案。我們在 *QuantStar* 的競爭對手正在推銷一套 *AI* 驅動的風險系統。如果我們想保持領先，就需要向 *Kenji* 的團隊展示 *AI* 能為我們的框架帶來什麼。

我洗耳恭聽。*AI* 能做到什麼是你的 *GARCH* 模型做不到的？

讓我展示給你看。下週。

7.9 AI 焦點：自動化風險監控

AI 增強的風險管理系統

現代風險管理正越來越多地被 AI 所增強：

- **自動化 VaR 違反偵測**：機器學習分類器可以在違反發生之前預測可能的違反日，使用的特徵包括 VIX 變化、信用利差和新聞情緒等。預警系統可以觸發投資組合的避險操作。
- **動態壓力情境生成**：不再局限於固定的歷史情境，生成模型（GANs、VAEs）可以透過學習過去危機的統計結構來創建逼真但全新的危機情境。
- **體制偵測**：隱馬爾可夫模型（Hidden Markov Models）或變點偵測演算法可以自動識別市場何時從平靜轉為劇烈波動的體制，並觸發模型參數的更新。
- **風險敘事的自然語言處理**：大型語言模型可以自動從量化輸出生成人類可讀的風險報告，在量化分析師與董事會成員之間架起橋樑。

共同的主軸：AI 並不取代核心風險框架（VaR、ES、回測）— 它增強每個組件，帶來更好的數據、更快的適應，以及更豐富的情境分析。

7.10 討論：當模型失效時

危機之後的風險管理

1. **2008 年的教訓**：許多銀行直到全球金融危機爆發前都擁有綠燈區的 VaR 模型。他們的 VaR 模型是「經過回測且驗證通過的」。問題出在哪裡，我們如何防止再次發生？
2. **模型風險 vs. 市場風險**：使用錯誤模型的風險有時是否會大於市場風險本身？風險管理者應如何在模型複雜度與模型風險之間取得平衡？
3. **監管套利**：如果銀行知道 Basel 交通號誌的門檻值，他們是否可能刻意設計在綠燈區的模型，同時低估真實的風險？監管機構如何偵測這種行為？
4. **超越 VaR**：FRTB 從 VaR 轉向 ES。但 ES 更難回測（你需要評估罕見事件的平

均值）。這對監管監督是否構成問題？

7.11 每週任務

◎ 任務七：FSA 回測報告

目標：為日本 FSA 製作一份全面的 VaR 回測與壓力測試報告。

交付成果：

1. 對日經 225 指數和 S&P 500 指數分別進行：
   - 使用 GJR-GARCH（t 分配）和 1000 天視窗計算滾動 99% VaR
   - 計算違反次數與違反率
   - 執行 Kupiec 無條件覆蓋率檢定
   - 執行 Christoffersen 條件覆蓋率檢定
   - 判定最近 250 天的 Basel 交通號誌區域
2. 為每個指數建立回測視覺化圖表（報酬率 + VaR + 違反標記）
3. 為每個指數建立累積違反圖
4. 對至少 5 個情境（3 個歷史 + 2 個假設）執行壓力測試，並報告 VaR 與 ES 超越比率
5. 撰寫 300 字的執行摘要，以致日本 FSA 的信函形式呈現，涵蓋：模型描述、回測結果、Basel 分類、以及壓力測試結論

🏆 加分挑戰：比較 VaR 模型

對三種 VaR 方法執行完整的回測套件：

- 歷史模擬法（500 天視窗）
- 標準 GARCH(1,1) 加常態分配的參數 VaR
- GJR-GARCH(1,1) 加 t 分配的參數 VaR

比較它們的 Kupiec 檢定結果和 Basel 交通號誌區域。哪個模型表現最好？在報告上「看起來最好」的模型是否也在實務中產生最有用的風險估計？

7.12 重點摘要

重點摘要

1. **回測驗證 VaR 模型**：將預測的 VaR 與實際損失進行比較。命中序列 $I_t \in \{0, 1\}$ 應具有正確的比例且無群聚現象。
2. **Kupiec 檢定**：檢驗無條件覆蓋率 — 整體違反率是否等於 $1-\alpha$？使用具有 $\chi^2(1)$ 分配的概似比統計量。
3. **Christoffersen 檢定**：同時檢驗覆蓋率和獨立性 — 違反是否適當分散或群聚？聯合檢定使用 $\chi^2(2)$。
4. **Basel 交通號誌**：一個實用的分類系統。綠燈（0–4 次違反）為可接受；黃燈（5–9 次）引發審查；紅燈（≥ 10 次）要求增加資本。
5. **壓力測試與回測互補**：回測評估正常條件；壓力測試評估極端情境。兩者都是監管合規的要求。
6. **GJR-GARCH 加 t 分配**：持續為股票指數產生綠燈區的回測結果，優於歷史模擬法和常態分配參數 VaR。

7.13 延伸閱讀

延伸閱讀

1. Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives*, 3(2), 73–84. [原始 POF 檢定]
2. Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review*, 39(4), 841–862. [條件覆蓋率檢定]
3. Basel Committee on Banking Supervision (1996). Supervisory framework for the use of "backtesting" in conjunction with the internal models approach to market risk capital requirements. [交通號誌系統]
4. McNeil, A. J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management*, 2nd ed. Chapter 9.
5. Berkowitz, J., & O'Brien, J. (2002). How accurate are Value-at-Risk models at commercial banks? *Journal of Finance*, 57(3), 1093–1111. [實務中的 VaR 表現]
6. Jorion, P. (2007). *Value at Risk*, 3rd ed. McGraw-Hill. Chapters 6–8.
7. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 7.
8. Hull, J. C. (2018). *Risk Management and Financial Institutions*, 5th ed. Wiley. Chapters 14–16.

下 週 預 告

Kenji 簽署 *FSA* 報告後，*David* 召集了全體會議。「我剛接到 *Kenji* 投資委員會的電話。他們對我們的風險框架印象深刻，但提出了一個新問題：『*AI* 能做得更好嗎？』」他環視了整個房間。「*QuantStar* 正在向他們推銷一套純機器學習風險系統 — 隨機森林、*LSTM*，全套方案。他們宣稱比傳統模型準確度提升 *50%*。」*Priya* 的眼睛亮了起來。「接受挑戰。」*Dr. Lin* 挑了挑眉毛。「用什麼衡量標準的準確度？對照什麼基準？具有什麼可解釋性？」*David* 微笑著說。「這正是我們需要展示給他們看的。下週：我們建構 *VolTech* 的 *AI* 風險引擎，並讓它與我們至今建立的所有模型正面對決。」下週：***AI* 與金融** — 機器學習與金融風險相遇之處，我們將發現未來不是 *AI* 或 *GARCH*，而是 *AI* 加上 *GARCH*。

Chapter 8

第 8 週：AI 與金融

學習目標

1. 解釋機器學習方法與傳統計量經濟模型在波動率預測上的差異
2. 使用工程特徵實作隨機森林（Random Forest）波動率預測模型
3. 理解 LSTM 神經網路用於時間序列的基本架構與直覺
4. 運用第五週的評估框架進行嚴謹的 ML vs. GARCH 比較
5. 設計一個結合 GARCH 結構知識與 ML 彈性的混合模型

8.1 本週故事：最終挑戰

場景：VolTech Analytics — 週一，早上 8:00

會議室已經完全改頭換面。Priya 架設了兩個並排的螢幕：左邊顯示 GARCH 的輸出，右邊顯示一個神經網路訓練儀表板，損失曲線正在即時更新。白板上方掛著一條橫幅寫著：**GARCH vs. AI：最終對決**。

歡迎來到最後一輪。*QuantStar* 宣稱他們的 *ML* 系統打敗傳統模型 *50%*。今天我們要測試這個說法 — 誠實地、嚴謹地，並使用 我們自己建立的評估框架。不挑選有利的數據，不使用前瞻偏差，不搞炒作。

我想先釐清我們在測試什麼。問題不是「*AI* 是不是魔法？」或「*GARCH* 是不是過時了？」問題是：在什麼條件下每種方法有優勢，以及我們如何智慧地結合它們？

我的投資委員會需要一個明確的建議：我們應該投資 *AI* 風險系統、維持 *GARCH* 框架、還是兩者都要？答案需要以證據為基礎，而非行銷簡報。

那就讓我們建立證據。*Priya*，展示給我們看吧。

8.2 ML vs. 計量經濟學：兩種哲學

波動率建模的兩種方法

傳統計量經濟學方法（GARCH）：

- 從金融理論出發（如波動率群聚、槓桿效應）
- 設定一個參數模型（如 GJR-GARCH 加 t 分配）
- 透過最大概似法估計少量參數（5–7 個）
- 可解釋：每個參數都有經濟意義
- 在有限數據下仍有穩健的表現

機器學習方法：

- 從數據出發，而非理論
- 透過彈性的函數逼近學習複雜的模式
- 可能估計數千個參數
- 通常是「黑盒子」：有預測但沒有解釋
- 需要大量數據集以避免過度擬合

兩者都不是普遍優越的。選擇取決於數據環境、預測期間，以及可解釋性的要求。

那 *Nippon Life* 日常 *VaR* 報告實際上應該部署哪一個？

這取決於你的數據和預測期間 — 這正是偏差-變異數取捨（*Bias-Variance Tradeoff*）所形式化的問題。

金融中的偏差-變異數取捨

GARCH 具有高**偏差**（Bias）（對數據生成過程有強假設）但低**變異數**（Variance）（在有限數據下預測穩定）。ML 具有低偏差（假設彈性）但高變異數（預測可能不穩定，特別是在小數據集的情況下）。

在金融中，數據是嘈雜的，且體制會變化。這通常有利於在 短期間使用高偏差、低變異數的方法。ML 的優勢在以下情況下浮現：

- 有豐富的特徵集可用（VIX、交易量、情緒）
- 預測期間較長（每週、每月）
- 存在參數模型無法捕捉的非線性模式
- 有足夠的訓練數據來正則化模型

8.3 隨機森林用於波動率

隨機森林（Random Forest）：決策樹的集成

隨機森林（Random Forest）建構許多決策樹，每棵樹在數據和特徵的隨機子集上訓練。最終預測是所有樹的平均值。

為何它適用於波動率：

- 捕捉特徵與未來波動率之間的非線性關係
- 對異常值具有穩健性（樹可以適應極端值）
- 內建特徵重要性排名
- 無分配假設

波動率預測的特徵工程（Feature Engineering）：

- **落後報酬率**：$r_{t-1}, r_{t-2}, \ldots, r_{t-5}$
- **落後報酬率平方**：$r_{t-1}^2, \ldots, r_{t-5}^2$ （近期波動率的代理變數）
- **滾動波動率**：5 天、10 天、22 天滾動標準差
- **絕對報酬率**：$|r_{t-1}|, \ldots, |r_{t-5}|$
- **GARCH 條件波動率**：$\hat{\sigma}_t$ 來自已配適的 GARCH 模型（混合特徵 !）

注意最後那個特徵：*GARCH* 條件波動率作為 *ML* 模型的輸入。這是混合方法的關鍵。我們讓 *GARCH* 將其結構知識濃縮成單一特徵，然後讓 *ML* 模型學習它是否能在此基礎上增加什麼。

8.4 LSTM 神經網路用於時間序列

長短期記憶（LSTM）

LSTM（Long Short-Term Memory）是一種專為序列數據設計的遞迴神經網路。核心概念：

- **細胞狀態**（c_t）：一條長程的傳輸帶，能以最少的修改在許多時間步之間傳遞資訊。它與**隱藏狀態**（h_t）不同：細胞狀態是 LSTM 的長期記憶，在每一步以加法方式更新；而隱藏狀態是它的短期工作記憶，暴露給下一層並直接用於預測。直覺上，c_t 記住「我們處於高波動率體制」已經持續數週，而 h_t 編碼最近的報酬率動態。
- **閘門**（Gates）：三個閘門控制資訊流：

  - 遺忘閘門（Forget Gate）：決定丟棄 c_{t-1} 中的哪些舊資訊
  - 輸入閘門（Input Gate）：決定將哪些新資訊寫入 c_t
  - 輸出閘門（Output Gate）：從 c_t 導出 h_t，決定暴露給下一層的內容
- **序列處理**：逐一處理每個時間步的報酬率，透過 c_t 和 h_t 的交互作用逐步建構波動率模式的表示

波動率的直覺：細胞狀態允許 LSTM 記住許多步之前發生的波動率體制轉換 — 這是普通 RNN 因梯度消失而會遺忘的。在實務上，c_t 捕捉波動率的持續水準，而 h_t 追蹤短期的群聚現象，映射出 GARCH 模型中長期變異數與條件變異數的角色。

所以 *LSTM* 可以自行發現類似 *GARCH* 的行為？

理論上是的。但問題在於：*GARCH* 用 *5* 個參數就免費提供了這一切。*LSTM* 需要數千個參數和更多的數據才能學到相同的模式。問題是 *LSTM* 是否能學到 *GARCH* 所無法捕捉的額外 模式。

8.5 Python 實作：ML 競技場

讓我們建立一套完整的 *ML* 波動率預測流程，並與 *GJR-GARCH* 進行正面對比。

步驟一：特徵工程

```python
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from arch import arch_model
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import warnings
warnings.filterwarnings("ignore")

# Download data
sp500 = yf.download("^GSPC", start="2010-01-01",
                    end="2024-12-31")
```

```python
prices = sp500["Close"].squeeze()
returns = 100 * np.log(prices / prices.shift(1)).dropna()

# Target: next-day realized volatility (squared return)
target = returns.shift(-1) ** 2

# Feature engineering
features = pd.DataFrame(index=returns.index)
# Lagged returns
for lag in range(1, 6):
    features[f"ret_lag{lag}"] = returns.shift(lag)
# Lagged squared returns
for lag in range(1, 6):
    features[f"ret2_lag{lag}"] = (
        returns.shift(lag) ** 2)
# Rolling volatility
for w in [5, 10, 22]:
    features[f"rvol_{w}d"] = (
        returns.rolling(w).std())
# Absolute returns
for lag in range(1, 4):
    features[f"abs_ret_lag{lag}"] = (
        returns.shift(lag).abs())

# Drop NaN rows
data = pd.concat([features, target.rename("target")],
                 axis=1).dropna()
print(f"Features: {len(features.columns)}")
print(f"Observations: {len(data)}")
print(f"Feature list:\n{list(features.columns)}")
```

步驟二：訓練-測試分割與 GJR-GARCH 基準線

```python
# Time-series split (no shuffling!)
split_date = "2020-01-01"
train = data[data.index < split_date]
test = data[data.index >= split_date]

```

```python
X_train = train.drop("target", axis=1)
y_train = train["target"]
X_test = test.drop("target", axis=1)
y_test = test["target"]

print(f"Train: {len(train)} obs "
      f"({train.index[0].date()} to "
      f"{train.index[-1].date()})")
print(f"Test:  {len(test)} obs "
      f"({test.index[0].date()} to "
      f"{test.index[-1].date()})")

# GJR-GARCH baseline (full-sample fit, then forecast)
ret_all = returns.loc[data.index]
model = arch_model(ret_all, vol="Garch", p=1, o=1,
                   q=1, dist="t")
res = model.fit(last_obs=split_date, disp="off")
fv = res.forecast(start=split_date, horizon=1)
garch_var = fv.variance.dropna()
garch_fc = garch_var.iloc[:, 0]
# Align with test period
garch_fc = garch_fc.reindex(test.index).dropna()
```

步驟三：隨機森林模型

```python
# Random Forest with tuned hyperparameters
rf = RandomForestRegressor(
    n_estimators=500,
    max_depth=10,
    min_samples_leaf=20,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)
rf_pred = pd.Series(rf.predict(X_test),
                    index=X_test.index)

```

```python
# Feature importance
importance = pd.Series(
    rf.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

print("Top 10 Features:")
for feat, imp in importance.head(10).items():
    print(f"  {feat:<20} {imp:.4f}")
```

滾動波動率特徵在重要性排名中佔據主導地位。這很合理 — 近期的波動率是未來波動率的最佳預測因子。

正是如此。而且注意這與 *GARCH* 結構的相似性：*GARCH* 中的 β 項本質上就是「昨天的條件變異數」，也就是模型自身的滾動波動率摘要。隨機森林正在從數據中重新發現這個模式。

步驟四：混合模型 — GARCH + 隨機森林

```python
# GARCH variance for hybrid feature:
#   training period  -> in-sample conditional var
#   test period      -> out-of-sample forecast
# In-sample: fitted conditional variance (training only)
garch_in = res.conditional_volatility ** 2
garch_in = garch_in[garch_in.index < split_date]
# Out-of-sample: forecasted variance (test only)
garch_out = garch_fc           # from Step 2
garch_feature = pd.concat([garch_in, garch_out])
assert not garch_feature.index.duplicated().any(), \
    "Overlap detected between in-sample and forecast"
garch_feature = garch_feature.reindex(data.index)

data_hybrid = data.copy()
data_hybrid["garch_var"] = garch_feature

train_h = data_hybrid[data_hybrid.index < split_date]
```

```python
test_h = data_hybrid[data_hybrid.index >= split_date]
train_h = train_h.dropna()
test_h = test_h.dropna()

X_train_h = train_h.drop("target", axis=1)
y_train_h = train_h["target"]
X_test_h = test_h.drop("target", axis=1)
y_test_h = test_h["target"]

# Hybrid Random Forest
rf_hybrid = RandomForestRegressor(
    n_estimators=500,
    max_depth=10,
    min_samples_leaf=20,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)
rf_hybrid.fit(X_train_h, y_train_h)
hybrid_pred = pd.Series(rf_hybrid.predict(X_test_h),
                        index=X_test_h.index)

# Check if GARCH feature is important
imp_h = pd.Series(
    rf_hybrid.feature_importances_,
    index=X_train_h.columns
).sort_values(ascending=False)
garch_rank = list(imp_h.index).index("garch_var") + 1
print(f"\nGARCH feature importance rank: "
      f"#{garch_rank} of {len(imp_h)}")
print(f"GARCH feature importance: "
      f"{imp_h['garch_var']:.4f}")
```

步驟五：正面對決比較

```python
# Align all forecasts to common index
common_idx = garch_fc.index.intersection(
    rf_pred.index).intersection(hybrid_pred.index)
```

```python
y_eval = y_test.reindex(common_idx)
garch_eval = garch_fc.reindex(common_idx)
rf_eval = rf_pred.reindex(common_idx)
hybrid_eval = hybrid_pred.reindex(common_idx)

def evaluate(actual, forecast, name):
    """Same three metrics from Week 5."""
    a, f = actual.values, np.maximum(
        forecast.values, 1e-10)
    rmse = np.sqrt(np.mean((a - f) ** 2))
    mae = np.mean(np.abs(a - f))
    qlike = np.mean(np.log(f) + a / f)
    return {"Model": name, "RMSE": rmse,
            "MAE": mae, "QLIKE": qlike}

results = pd.DataFrame([
    evaluate(y_eval, garch_eval, "GJR-GARCH"),
    evaluate(y_eval, rf_eval, "Random Forest"),
    evaluate(y_eval, hybrid_eval,
             "Hybrid (GARCH+RF)"),
]).set_index("Model")

print("\n=== Forecast Comparison ===")
print(results.round(4))
print(f"\nBest by RMSE:  {results['RMSE'].idxmin()}")
print(f"Best by MAE:   {results['MAE'].idxmin()}")
print(f"Best by QLIKE: {results['QLIKE'].idxmin()}")
```

步驟六：視覺化

```python
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Forecast comparison
ax1 = axes[0]
ax1.plot(y_eval.index,
         np.sqrt(y_eval * 252), color="gray",
         alpha=0.3, linewidth=0.5,
         label="Realized proxy (ann.)")
```

```
ax1.plot(garch_eval.index,
         np.sqrt(garch_eval * 252), color="#1B3A5C",
         linewidth=1, label="GJR-GARCH")
ax1.plot(rf_eval.index,
         np.sqrt(rf_eval * 252), color="#C0392B",
         linewidth=1, alpha=0.7,
         label="Random Forest")
ax1.plot(hybrid_eval.index,
         np.sqrt(hybrid_eval * 252), color="#2D7D5E",
         linewidth=1, alpha=0.7,
         label="Hybrid (GARCH+RF)")
ax1.axvspan("2020-02-20", "2020-04-01",
            alpha=0.1, color="red")
ax1.set_ylabel("Annualized Volatility (%)")
ax1.set_title("ML vs. GARCH: Out-of-Sample "
              "Volatility Forecasts")
ax1.legend(loc="upper right", fontsize=8)
ax1.set_ylim(0, 120)

# Panel 2: Feature importance
ax2 = axes[1]
top_features = imp_h.head(10)
colors = ["#D4A843" if "garch" in f else "#1B3A5C"
          for f in top_features.index]
ax2.barh(range(len(top_features)),
         top_features.values, color=colors)
ax2.set_yticks(range(len(top_features)))
ax2.set_yticklabels(top_features.index, fontsize=9)
ax2.set_xlabel("Feature Importance")
ax2.set_title("Hybrid Model: Top 10 Features "
              "(gold = GARCH)")
ax2.invert_yaxis()

plt.tight_layout()
plt.savefig("week08_ml_vs_garch.png", dpi=150)
plt.show()
```

8.6 應用：最終裁決

場景：VolTech 董事會議室 — 週五，早上 10:00

整個 VolTech 團隊，加上 Kenji 的投資委員會（透過視訊參加），齊聚一堂進行最終報告。八週的工作歸結到這一刻。

經過廣泛的測試，以下是我們對日波動率預測中 *GARCH vs. ML* 的結論：

Table 8.1: GARCH vs. ML：研究發現摘要[a]

評估標準	GJR-GARCH	隨機森林	混合模型
1 天 RMSE	5.61	5.52	**5.21**
1 天 QLIKE	3.65	3.82	**3.51**
可解釋性	高	低	中
訓練速度	快	中等	中等
監管適配性	強	弱	中
危機應對	良好	不穩定	良好

[a]RMSE 和 QLIKE 為 S&P 500 日報酬率平方（%）的代表性數值。確切數字取決於樣本期間；相對排序是穩健的。

三個關鍵發現：

1. **在 *1* 天期間，*GJR-GARCH* 的競爭力驚人**。隨機森林幾乎沒有改善，有時在 *QLIKE* 上甚至更差。*QuantStar* 所宣稱的「*50%* 改善」很可能使用了挑選過的指標或前瞻偏差。
2. **混合模型是明確的贏家**。將 *GARCH* 條件波動率作為特徵加入隨機森林，持續產生最佳的預測結果。*GARCH* 提供結構骨幹；*ML* 負責精細調整。
3. **可解釋性對監管合規至關重要**。*FSA* 可以稽核 *GARCH* 模型。一個有 *10,000* 個參數的神經網路要解釋起來困難得多。混合方法提供了一個務實的折衷方案。

讓我補充學術觀點。我們的結果與文獻一致。*Bucci*（*2020*）顯示 *GARCH* 在日頻率上仍然難以被超越。*Christensen*、*Siggaard* 和 *Veliyev*（*2023*）發現結合 *ML* 與計量經濟學特徵的表現優於單獨使用任一方法。金融風險建模的未來不是 *AI* 與傳統方法之間的戰爭 — 而是它們的整合。

那你對我基金的建議是？

部署 *GJR-GARCH* 作為日常 *VaR* 和 *ES* 報告的主要風險引擎 — 它快速、可解釋、且符合 *Basel* 規範。在其上層疊混合隨機森林模型作為補充監控工具，在 *GARCH* 可能低估風險時發出警示。這讓你兩全其美：監管合規加上尖端分析。

從治理的角度來看這確實合理。董事會永遠不會批准一個純黑盒子用於資本計算 — 但一個 *GARCH* 引擎搭配 *ML* 預警層的方案是我可以帶到投資委員會逐行解釋的。合約開始進行吧。

而這就是 *VolTech* 所提供的，*Kenji*：不是盲目採用最新的技術，而是基於證據的智慧整合。歡迎來到風險管理的未來。

8.7 FinTech 前沿

AI 與 FinTech：下一步是什麼

AI 與金融的交匯正在快速演變。未來十年的關鍵趨勢：

- **金融基礎模型**：在金融文本上訓練的大型語言模型（GPT、Claude）可以擷取情緒、摘要法說會，並生成風險敘事。它們不會取代量化分析師，但會增強他們的生產力。
- **可解釋 AI（XAI）**：監管壓力推動了可解釋 ML 模型的發展。SHAP 值、注意力圖和反事實解釋使黑盒子更加透明。
- **即時風險監控**：串流 ML 流程逐筆處理逐筆交易數據、新聞推送和社交媒體，以更新日內的風險估計。請注意，高頻數據帶來其自身的挑戰 — 市場微觀結構噪音、買賣價差彈跳和非同步到達都需要仔細的預處理，才能可靠地套用任何模型。
- **量化工具的民主化**：雲端平台和開源函式庫（如 `arch`、`scikit-learn`、`pytorch`）使精密的風險分析變得更容易被小型公司和個人投資者所使用。
- **監管科技（RegTech）**：自動化合規報告、異常偵測和稽核追蹤的 AI 系統。我們這學期手動建立的 FSA 報告在幾年內很可能會自動生成。

能在這個領域蓬勃發展的專業人士，將是那些同時理解金融理論和 *AI* 工具的人 — 正是這門課程所建立的組合。

8.8 討論：金融的未來

AI、倫理與未來

1. **工作取代**：AI 會取代金融分析師和風險管理者嗎？還是會創造需要不同技能的新角色？你應該如何準備？
2. **演算法偏差**：如果 AI 風險模型是在歷史數據上訓練的，它就繼承了該數據的偏差（例如低估新興市場風險）。我們如何偵測和減輕這種偏差？
3. **系統性風險**：如果每家銀行都使用相同的 AI 風險模型，他們可能都以相同的方式應對市場壓力 — 放大崩盤而不是減緩。AI 驅動的羊群效應是否是真正的威脅？
4. **回顧**：你花了 8 週從零建立一套風險管理系統。你學到最意外的事情是什麼？如果重新開始這個專案，你會有什麼不同的做法？

8.9 每週任務

◎ 任務八：VolTech AI 風險報告

目標：建構並評估一個 ML 波動率預測模型，並為 Kenji 的投資委員會撰寫最終報告。

交付成果：

1. 為日經 225 指數和 S&P 500 指數建構隨機森林模型：
   - 工程化至少 15 個特徵（落後報酬率、滾動波動率、絕對報酬率）
   - 使用時間序列分割（2020 年前訓練、2020 年後測試）
   - 報告特徵重要性排名
2. 透過添加 GJR-GARCH 條件波動率作為額外特徵建構混合模型
3. 使用 RMSE、MAE 和 QLIKE 比較 GJR-GARCH、隨機森林和混合模型
4. 建立比較視覺化圖表（如步驟六所示）
5. 撰寫 500 字的最終報告（以 VolTech 致 Nippon Life 退休基金的備忘錄形式）：
   - 全部 8 週的執行摘要
   - 附帶證據的模型建議
   - 風險管理框架概述
   - 未來路線圖與 AI 整合計畫

加分挑戰：建構簡單的 LSTM

對於熟悉深度學習的同學，實作一個簡單的 LSTM 波動率預測模型：

- 使用 PyTorch 或 TensorFlow
- 輸入：22 個過去報酬率的序列
- 輸出：次日報酬率平方的預測
- 與 GJR-GARCH 和隨機森林進行比較
- LSTM 需要多少訓練數據才能開始具有競爭力？

提示：從單層 LSTM 搭配 32 個隱藏單元開始。使用強正則化（dropout = 0.3）以防止過度擬合。

8.10 重點摘要

重點摘要

1. **ML 和 GARCH 是互補的，而非競爭的**：GARCH 以簡潔的形式提供結構知識（波動率群聚、槓桿效應）。ML 增加彈性以捕捉超出 GARCH 假設範圍的模式。
2. **GARCH 在日頻率上難以被超越**：僅用 5–7 個參數，GJR-GARCH 就能捕捉日波動率的主要動態。ML 的優勢隨著更豐富的特徵和更長的期間而增長。
3. **特徵工程是 ML 成功的關鍵**：隨機森林的表現高度依賴你所提供的特徵。最重要的特徵（滾動波動率、落後報酬率平方）映射出 GARCH 的結構。
4. **混合模型獲勝**：將 GARCH 條件波動率作為 ML 特徵持續產生最佳的樣本外預測。這是實務建議：將 GARCH 用作特徵，而非競爭者。
5. **可解釋性具有經濟價值**：監管合規要求模型可解釋。一個無法被稽核的模型可能產生更好的預測，但商業結果更差。
6. **未來在於整合**：最有效的風險管理系統將結合計量經濟學結構、ML 彈性和人類判斷。這門課程已為你建立了這三者的基礎。

8.11 延伸閱讀

延伸閱讀

1. Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502–531.
2. Christensen, K., Siggaard, M., & Veliyev, B. (2023). A machine learning approach to volatility forecasting. *Journal of Financial Econometrics*, 21(5), 1680–1727.
3. Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273. [ML 在金融中的基準論文]
4. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. [原始 LSTM 論文]
5. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
6. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS*. [模型可解釋性的 SHAP 值]
7. Dixon, M. F., Halperin, I., & Bilokon, P. (2020). *Machine Learning in Finance: From Theory to Practice*. Springer.
8. Tsay, R. S. (2010). *Analysis of Financial Time Series*, 3rd ed. Wiley. Chapter 12.

公式附錄

本附錄彙集教科書中所有關鍵公式，以便快速查閱。

第一週：報酬分配

簡單報酬率

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{8.1}$$

對數報酬率

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(1 + R_t) \tag{8.2}$$

偏態

$$S = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^3 \tag{8.3}$$

超額峰態

$$K = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^4 - 3 \tag{8.4}$$

Jarque–Bera 檢定

$$JB = \frac{T}{6}\left(S^2 + \frac{K^2}{4}\right) \sim \chi^2(2) \tag{8.5}$$

第二週：衡量波動率

滾動視窗波動率

$$\hat{\sigma}_t^{\text{roll}} = \sqrt{\frac{1}{N-1}\sum_{i=0}^{N-1}(r_{t-i} - \bar{r}_{t,N})^2} \tag{8.6}$$

年化：乘以 $\sqrt{252}$。

EWMA 變異數

$$\hat{\sigma}_t^2 = \lambda\,\hat{\sigma}_{t-1}^2 + (1-\lambda)\,r_{t-1}^2 \tag{8.7}$$

RiskMetrics 標準：$\lambda = 0.94$。有效視窗 $\approx 1/(1-\lambda)$ 天。

實現波動率

$$RV_t = \sum_{j=1}^{M} r_{t,j}^2 \tag{8.8}$$

其中 $r_{t,j}$ 為第 t 天的 M 個日內報酬率。

第三週：GARCH

報酬率分解

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t\, z_t, \quad z_t \sim D(0,1) \tag{8.9}$$

ARCH(q) 模型

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i\, \varepsilon_{t-i}^2 \tag{8.10}$$

GARCH(1,1) 模型

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2 \tag{8.11}$$

限制條件：$\omega > 0$，$\alpha \geq 0$，$\beta \geq 0$，$\alpha + \beta < 1$。

√× 長期變異數

$$\bar{\sigma}^2 = \frac{\omega}{1-\alpha-\beta} \tag{8.12}$$

√× 波動率半衰期

$$H = \frac{\ln(0.5)}{\ln(\alpha+\beta)} \tag{8.13}$$

√× Ljung–Box 檢定

$$Q(m) = T(T+2)\sum_{k=1}^{m}\frac{\hat{\rho}_k^2}{T-k} \sim \chi^2(m) \tag{8.14}$$

√× 標準化殘差

$$\hat{z}_t = \frac{r_t - \hat{\mu}}{\hat{\sigma}_t} \tag{8.15}$$

第四週：不對稱波動率

√× GJR-GARCH(1,1)

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \gamma\,\varepsilon_{t-1}^2\,I_{[\varepsilon_{t-1}<0]} + \beta\,\sigma_{t-1}^2 \tag{8.16}$$

$\gamma > 0$ 捕捉槓桿效應。定態條件：$\alpha + \beta + \gamma/2 < 1$。

√× 新聞衝擊曲線

$$\sigma_t^2(\varepsilon_{t-1}) = \begin{cases} \omega + (\alpha+\gamma)\,\varepsilon_{t-1}^2 + \beta\,\bar{\sigma}^2 & \text{if } \varepsilon_{t-1} < 0 \\ \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\bar{\sigma}^2 & \text{if } \varepsilon_{t-1} \geq 0 \end{cases} \tag{8.17}$$

√× EGARCH(1,1)

$$\ln\sigma_t^2 = \omega + \alpha(|z_{t-1}| - E|z_{t-1}|) + \gamma\,z_{t-1} + \beta\,\ln\sigma_{t-1}^2 \tag{8.18}$$

模型對數變異數；無需正值限制。

概似比檢定

$$LR = 2(\ell_{\text{GJR}} - \ell_{\text{GARCH}}) \sim \chi^2(1) \tag{8.19}$$

若 $LR > 3.84$，則拒絕 GARCH 偏好 GJR。

第五週：波動率預測

損失函數

$$\text{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\hat{\sigma}_t^2 - \sigma_t^{*2})^2} \tag{8.20}$$

$$\text{MAE} = \frac{1}{T}\sum_{t=1}^{T}|\hat{\sigma}_t^2 - \sigma_t^{*2}| \tag{8.21}$$

$$\text{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left(\ln\hat{\sigma}_t^2 + \frac{\sigma_t^{*2}}{\hat{\sigma}_t^2}\right) \tag{8.22}$$

其中 σ_t^{*2} 為實現波動率代理變數。

Mincer–Zarnowitz 迴歸

$$\sigma_t^{*2} = a + b\,\hat{\sigma}_t^2 + u_t \tag{8.23}$$

最適預測：$H_0 : a = 0,\ b = 1$。聯合 F 檢定。

第六週：風險值

VaR 定義

$$P(L_t > \text{VaR}_\alpha) = 1 - \alpha \tag{8.24}$$

參數 VaR（常態）

$$\text{VaR}_\alpha = -(\mu + z_{1-\alpha}\cdot\sigma_t) \tag{8.25}$$

√× 參數 VaR（Student-t）

$$\mathrm{VaR}_\alpha = -\left(\mu + t_{1-\alpha,\nu} \cdot \sqrt{\frac{\nu-2}{\nu}} \cdot \sigma_t\right) \tag{8.26}$$

尺度因子 $\sqrt{(\nu-2)/\nu}$ 將標準 t 轉換為單位變異數。

√× 期望損失 / CVaR

$$\mathrm{ES}_\alpha = E[L_t \mid L_t > \mathrm{VaR}_\alpha] \tag{8.27}$$

ES 是一致性風險衡量；VaR 不是（不滿足次可加性）。

第七週：VaR 回測

√× Kupiec POF 檢定

$$LR_{\mathrm{POF}} = 2\left[x \ln\left(\frac{x/T}{1-\alpha}\right) + (T-x)\ln\left(\frac{1-x/T}{\alpha}\right)\right] \sim \chi^2(1) \tag{8.28}$$

x = T 個觀察中的 VaR 違反次數。

√× Christoffersen 條件覆蓋率

$$LR_{\mathrm{ind}} = 2\left[\sum_{i,j} n_{ij} \ln\left(\frac{\hat{\pi}_{ij}}{\hat{\pi}_j}\right)\right] \sim \chi^2(1) \tag{8.29}$$

檢驗違反是否獨立（無群聚）。

$$LR_{\mathrm{CC}} = LR_{\mathrm{POF}} + LR_{\mathrm{ind}} \sim \chi^2(2) \tag{8.30}$$

√× Basel 交通號誌系統

區域	違反次數（250 天）	處理措施
綠燈	0–4	模型接受
黃燈	5–9	資本附加費（$k = 3.4$–3.85）
紅燈	10+	模型否決（$k \geq 4.0$）

第八週：AI 與金融

隨機森林預測

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x}) \tag{8.31}$$

B 棵決策樹的平均值，每棵樹在自助法樣本上訓練，使用隨機特徵子集（每次分割 $\sqrt{p}$ 個特徵）。

偏差-變異數分解

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \tag{8.32}$$

GARCH：高偏差、低變異數。ML：低偏差、高變異數。

術語表

本教科書中使用的關鍵術語與定義。術語按英文字母順序排列，並標註首次引入的週次。

術語	定義	週次
ARCH 模型	自迴歸條件異質變異數（Autoregressive Conditional Heteroscedasticity）；將變異數建模為過去誤差平方的函數。	3
回測（Backtesting）	透過將風險模型的預測值與歷史期間的已實現結果進行比較，來評估模型準確性的過程。	7
Basel 交通號誌	根據 250 天回測結果，將 VaR 模型分類為綠燈（0–4 次違反）、黃燈（5–9 次）或紅燈（10 次以上）的監管框架。	7
偏差-變異數取捨（Bias-Variance Tradeoff）	模型簡單性（高偏差）與彈性（高變異數）之間的平衡；比較 GARCH 與 ML 方法的核心概念。	8
Christoffersen 檢定	條件覆蓋率檢定，檢驗 VaR 違反是否具有正確的大小且獨立分布。	7
條件變異數（Conditional Variance）	給定時間 $t-1$ 之前資訊的隨機變數變異數；GARCH 建模的核心量。	3
CVaR（期望損失）	在損失超過 VaR 的條件下的期望損失；也稱為條件風險值（Conditional VaR）。一致性風險衡量。	6
衰減因子（Decay Factor, λ）	EWMA 中的平滑參數；控制舊觀測值失去影響力的速度。RiskMetrics 標準：$\lambda = 0.94$。	2
Diebold-Mariano 檢定	比較兩個競爭預測模型預測準確度的統計檢定。	5

術語	定義	週次
EGARCH	指數 GARCH（Exponential GARCH）；建模對數變異數以捕捉不對稱效果，無需正值限制。	4
EWMA	指數加權移動平均（Exponentially Weighted Moving Average）；賦予近期觀測值更多權重的波動率估計方法。	2
超額峰態（Excess Kurtosis）	峰態減 3；衡量相對於常態分配的尾部厚度。$K > 0$ 表示厚尾。	1
期望損失（Expected Shortfall）	見 CVaR。最差 $(1-\alpha)\%$ 情況下的平均損失。	6
厚尾（Fat Tails）	極端事件發生頻率高於常態分配預測的分配性質。	1
特徵工程（Feature Engineering）	從原始數據建立具資訊性的 ML 模型輸入變數的過程（如落後報酬率、滾動波動率）。	8
GARCH(1,1)	廣義自迴歸條件異質變異數（Generalized Autoregressive Conditional Heteroscedasticity）；金融中使用最廣泛的波動率模型。	3
GJR-GARCH	由 Glosten、Jagannathan 和 Runkle 提出的 GARCH 延伸，透過指標變數捕捉槓桿效應。	4
半衰期（Half-Life）	波動率衝擊衰減至初始影響一半所需的天數；$H = \ln(0.5)/\ln(\alpha+\beta)$。	3
歷史模擬法（Historical Simulation）	使用過去報酬率的經驗分配的 VaR 方法；非參數且不假設分配形式。	6
混合模型（Hybrid Model）	結合 GARCH 結構輸出（作為特徵）與 ML 彈性以改善預測的模型。	8
隱含波動率（Implied Volatility）	嵌入選擇權價格中的波動率水準；具前瞻性，不同於實現波動率。	2
積分變異數（Integrated Variance）	連續時間價格過程在一段期間內的真實（潛在）變異數；由實現波動率估計。	2

術語	定義	週次
Jarque–Bera 檢定	基於偏態和峰態的常態性統計檢定。在 H_0 下 $JB \sim \chi^2(2)$。	1
Kupiec 檢定	基於失敗比例（POF）的 VaR 模型準確性檢定；使用概似比統計量。	7
高狹峰（Leptokurtic）	超額峰態 > 0 的分配；尾部比常態分配更厚。	1
槓桿效應（Leverage Effect）	負報酬比同等幅度的正報酬更能增加波動率的經驗觀察。	4
Ljung–Box 檢定	時間序列自相關的綜合檢定；用於偵測報酬率平方中的 ARCH 效果。	3
對數報酬率（Log Return）	價格比的自然對數；因具加法性質而在金融建模中被偏好。	1
長期變異數（Long-Run Variance）	GARCH 模型隱含的無條件變異數：$\bar{\sigma}^2 = \omega/(1-\alpha-\beta)$。	3
LSTM	長短期記憶（Long Short-Term Memory）；專為具有長程依賴的序列數據設計的遞迴神經網路架構。	8
最大概似法（Maximum Likelihood）	尋找使觀察數據機率最大化之參數值的參數估計方法。	3
均值回歸（Mean Reversion）	波動率在被衝擊偏離後傾向回到長期平均值的趨勢。	2
微觀結構噪音（Microstructure Noise）	由買賣價差彈跳、離散跳動幅度和其他市場摩擦引起的高頻價格數據失真。	2
Mincer-Zarnowitz 迴歸	預測評估迴歸：$\sigma_t^{*2} = a + b\hat{\sigma}_t^2 + u_t$。最適預測：$a = 0, b = 1$。	5
蒙地卡羅 VaR（Monte Carlo VaR）	從配適模型模擬數千條報酬率路徑以估計損失分配的 VaR 方法。	6
新聞衝擊曲線（News Impact Curve）	顯示不同幅度的正負衝擊如何影響條件變異數的圖形。	4
樣本外（Out-of-Sample）	使用非模型估計所用數據進行的預測評估；預測性評估的黃金標準。	5

術語	定義	週次
參數 VaR（Parametric VaR）	假設報酬率遵循特定分配（常態或 Student-t）並具有時變變異數的 VaR 方法。	6
持續性（Persistence）	GARCH 中 $\alpha+\beta$ 的總和（或 GJR 中的 $\alpha+\beta+\gamma/2$）；衡量波動率衝擊衰減的速度。	3
QLIKE	擬概似損失函數（Quasi-Likelihood）；對波動率代理變數中的噪音具有穩健性，為預測文獻所偏好。	5
QQ 圖（QQ Plot）	分位-分位圖（Quantile–Quantile Plot）；用於比較樣本分配與理論分配的圖形工具。	1
隨機森林（Random Forest）	集成 ML 方法，將多棵在數據和特徵隨機子集上訓練的決策樹之預測取平均。	8
實現波動率（Realized Volatility）	日內報酬率平方之和；日積分變異數的無模型估計。	2
RiskMetrics	J.P. Morgan 1996 年的風險衡量框架；推廣了日數據 $\lambda=0.94$ 的 EWMA。	2
滾動視窗（Rolling Window）	使用最近 N 個報酬率的標準差並賦予等權重的波動率估計方法。	2
偏態（Skewness）	機率分配不對稱性的衡量。大多數股票報酬率序列 $S<0$。	1
標準化殘差(Standardized Residuals)	GARCH 模型的殘差 $\hat{z}_t=(r_t-\hat{\mu})/\hat{\sigma}_t$；若模型正確則應為 i.i.d.。	3
壓力測試（Stress Testing）	在假設的極端情境（歷史或使用者定義）下評估風險模型表現。	7
Student-t 分配	由自由度 ν 控制的厚尾分配；常用作 GARCH 中的誤差分配。	1
典型事實（Stylized Facts）	在各金融市場中觀察到的穩健經驗模式：群聚、均值回歸、不對稱性、長記憶、共同運動。	2
次可加性（Subadditivity）	一致性風險衡量的性質：$\rho(X+Y)\le\rho(X)+\rho(Y)$。VaR 可能違反此性質；ES 滿足此性質。	6

術語	定義	週次
風險值（Value-at-Risk, VaR）	在給定時間範圍和指定信賴水準下的最大預期損失。	6
變異數風險溢酬（Variance Risk Premium）	隱含波動率與實現波動率之間的差異；承擔波動率風險的補償。	2
VIX	CBOE 波動率指數；由 S&P 500 選擇權隱含的市場對 30 天波動率預期的衡量。	2
波動率群聚（Volatility Clustering）	價格大幅變動傾向跟隨大幅變動、小幅變動傾向跟隨小幅變動的經驗觀察。	3
波動率微笑/偏斜（Volatility Smile/Smirk）	隱含波動率跨選擇權履約價格的 U 型（或偏斜）模式；非常態性的證據。	2